

TrekLink Developer Handbook

Conventions, Workflow & the AI-First Doctrine

Version 1.0 · 2026-09-13

MEMBER	CODE	ROLE
Đỗ Đăng Khoa	KhoaDD	Leader / Product Owner
Trần Khải Hoàng	HoangTK	Member — Frontend
Lâm Phi Long	LongLP	Member — Gateway & Data
Nguyễn Bá Tân	TanNB	Member — Reporting & Monitoring
Nguyễn Ngọc Long	LongNN	Member — Backend

Capstone FA26SE159 · TrekLink Team

Supervisor: Đặng Ngọc Minh Đức

Contents

Orientation

TrekLink — Project Charter

- 1. Problem
- 2. What we're building
- 3. Actors & role summary
- 4. Research questions (drive the experiment plan — see roadmap)
- 5. Non-functional targets (binding — used as PR/Definition-of-Done gates)
- 6. Tech stack (as registered — see D-001 in decisions log, resolved)
- 7. Deliverables (map 1:1 to Jira epics and the roadmap's TP packages)
- 8. Related work / gap statement (condensed)

TrekLink Engineering Conventions — Index

- Read in this order
- What changed in v2 (2026-09-13)
- Adopting these conventions in a repo
- Authority

How We Work

Core Engineering Philosophy: Session-Based Development & Living SSOT

- 1. The Core Philosophy
- 2. Pillar I: Living Documentation as SSOT
- 3. Pillar II: Session-Based Development (SBD)
- 4. Crash-Safe Memory Checkpoint Protocol
- 5. Context Window & Token Budget Management (AI Agents)
- 6. Tenets of Iterative Refinement

Spec-Driven Development Workflow (SDSDW)

- 1. Executive Overview
- 2. Directory Structure (specs/{module_name}/)
- 3. Phase-by-Phase Execution Protocol
- 4. Universal Phase Exit Gate Matrix

Engineering Lifecycle & Operational Workflows

- 1. Executive Workflow Matrix
- 2. Lifecycle 1: Normal Feature Development Flow (Spec-First)
- 3. Lifecycle 2: Diagnostic & Debugging Flow (Hypothesis-Driven)
- 4. Lifecycle 3: Pull Request & Code Review Flow

Delivery

GitHub Workflow, Branching, Commits & Pull Requests

1. Repositories
2. Branch Model
3. Commit Conventions
4. Rebase Discipline
5. Pull Requests
6. Language Rule (Hard Requirement)
7. Branch Protection Settings
8. Where Work Is Tracked
9. Quick Reference
10. GitLab → GitHub Concept Map

Jira Tracking, Board Workflow & the Development Loop

1. The Three Places Work Lives
2. Work Item Hierarchy
3. The Board Workflow
4. BUG vs NEEDS HELP — they are not the same thing
5. Where Bugs Actually Go
6. Jira ↔ GitHub Integration
7. Sprints
8. The Development Loop (End to End)
9. Backlog ↔ Jira Key Mapping
10. Quick Reference

Team Communication, Daily Reports & Schedule Awareness

1. Channels — What Goes Where
2. The Two Mandatory Announcements
3. Daily Reports
4. Raising Problems
5. Schedule Awareness — Automated Countdown
6. Weekly Rhythm
7. Quick Reference

AI-First Development

The AI-First Doctrine, Toolchain & Prompting Standard

1. The Doctrine in One Page
2. Model Policy (Hard Requirement)

3. Required Toolchain
4. Agent Auto-Steering
5. The Mandatory Session Workflow
6. Token & Context Discipline
7. How to Prompt Well
8. Starter Prompts
9. Common Failure Modes
10. Checklist — New Member Setup

AI Agent Steering, Governance & Thinking Discipline

1. Operating Principles & Decision Hierarchy
2. The Universal Pre-Action Protocol
3. Power Modes & Token Budget
4. Hallucination & Loop Circuit Breakers
5. Tool Discipline (adapt to whatever tool names your actual agent exposes)
- 5.1 The Mandatory Session Shape
6. Multi-Agent / Subagent Dispatch (if your tool supports it)

Doc-Driven Scaffold & SSOT Conventions

1. TrekLink Application Note (read this first)
2. Universal Documentation-Driven Scaffold & SSOT Architecture

Engineering Standards

Architectural Conventions — NestJS Backend / Node Gateway / React Frontend

1. Module Topology
2. Entity & Audit Conventions
3. Idempotency & Priority Queue (gateway-sync — the module most different from a normal CRUD app)
4. Module Isolation & Parallel Team Scaling
5. Frontend Architecture: Feature-Sliced Design (FSD)

Backend Code & REST API Conventions (NestJS)

1. The Standard Module Structure
2. Input Validation
3. The Canonical Response Envelope (binding — from API_Design_Template.md)
4. Status Code & Error Taxonomy
5. Controller Standards
6. Async, Transactional & Concurrency Standards

Frontend Code & UI/UX Conventions (React + TypeScript)

1. Architectural Layout (Feature-Sliced Design)
2. State Management: The Triad Standard
3. The Core UX Rule: 4 Steps, 6 Fields

4. Reusable Screen Patterns
5. Keyboard Navigation & Accessibility (WCAG 2.1 AA)
6. Client API & Validation Contract Mirroring

Appendices

TrekLink — Roadmap & Milestone Calendar

1. The three things to know before reading anything else
2. Official milestone schedule
3. Week-by-week calendar and deliverable ledger
4. Report 1–7 ↔ engineering artifact map
5. TP1–TP6 ↔ Iteration ↔ Sprint reconciliation
6. ⚠ The Week-6 scope lock — what must be settled before it
7. Review-readiness checklists
8. Parallel workstreams

TrekLink — Decisions Log & Risk Register

Decisions

Risk register (carried from FA26SE159, kept live)

Contributing to TrekLink

1. Orient yourself
2. Where work lives
3. Before writing code
4. Branch, commit, PR
5. Merging
6. Backend response contract (non-negotiable)
7. Language
8. Using an AI coding agent
9. Definition of Done, every PR

Orientation

TrekLink — Project Charter

Duration: 09/2026 – 03/2027 · Class/Specialty: SE · Supervisor: Đặng Ngọc Minh Đức (Assoc. Prof)

Team & skill matrix (used for backlog assignment — see [03-backlog/](#)):

Name	Code	MSSV	Tech Stack	Role
Đỗ Đăng Khoa	KhoaDD	SE192357	Embedded C, Spring Boot, Java, ReactJS, TypeScript, NestJS , PostgreSQL	Leader — only member with NestJS listed; anchors backend architecture/review
Lâm Phi Long	LongLP	SE182769	PostgreSQL, Node.js, ReactJS, TypeScript	Member — Gateway Bridge (Node.js) + data-heavy backend
Trần Khải Hoàng	HoangTK	SE182767	Node.js, ReactJS	Member — frontend-leaning
Nguyễn Ngọc Long	LongNN	SE193490	Java Spring Boot, ReactJS, TypeScript, PostgreSQL	Member — backend CRUD modules (Spring Boot → NestJS transfer)
Nguyễn Bá Tân	TanNB	SE183860	PostgreSQL, MySQL, ReactJS, TypeScript	Member — reporting/DB + monitoring frontend

 **Skill-gap note** (from the matrix above): NestJS is explicitly listed only for Khoa, yet it's the whole backend framework. Budget Sprint 1–2 ramp-up/pairing on NestJS module conventions ([01-conventions/05-backend-conventions.md](#)) for Long L.P., Hoàng, Long N.N., and Tân, mentored by Khoa — tracked as an added risk in [03-decisions-and-risk-register.md](#) .

This charter reconciles [Phieu_FA26SE159.docx](#) (primary, authoritative — has formal RQs/NFRs/experiments) with [Phieu_dang_ky_de_tai_TrekLink_FA26_FINAL.docx](#) (earlier draft, same project). Differences are called out inline and logged as Decision D-000 in [03-decisions-and-risk-register.md](#) ([03-decisions-and-risk-register.md](#)).

1. Problem

Vietnamese trekking routes (Tà Năng–Phan Dũng, Tả Liên Sơn, Bạch Mộc Lương Tử, ...) run through multi-day cellular dead zones. SU26 produced **TrekLink firmware**: an open-source LoRa mesh (ESP32/ESP32-S3, forked from Meshtastic v2.7.19, ~636 files / ~149k LOC, 4 hardware variants v1–v4) with mesh messaging, GPS tracking, IMU-based fall detection, and device-level SOS broadcast — already working and **not being rebuilt**.

Two operational gaps remain, unaddressed by the firmware or by any existing product:

1. **No store-and-forward path to the cloud**. Field events generated during connectivity loss are silently lost — there is no Gateway between the mesh and a backend.
2. **No structured incident/rental operations layer**. A device-level SOS triggers nothing beyond a local mesh alert; there's no fleet, rental, or trip management; agencies coordinate by phone/WhatsApp with no audit trail.

2. What we're building

TrekLink Operations Platform — extends the inherited firmware with:

- **Gateway Bridge** (Node.js/TypeScript): LoRa-serial → SQLite priority queue (P0 SOS / P1 incident-location / P2 GPS / P3 telemetry) → MQTT → cloud, with store-and-forward and reconnection flush. *How a node actually reaches*

the internet (dedicated Wi-Fi/cellular hardware vs. a phone running the Meshtastic app) is still open — see Decision D-005.

- **Idempotent event ingestion:** `eventId = DeviceID + SessionID + SequenceNumber` as the backend idempotency key — N deliveries of the same event ⇒ exactly 1 Incident + 1 notification.
- **Device Fleet & Rental Management:** 7-state device lifecycle `Available → Reserved → Rented → In-Field → Returned → Maintenance → Retired`.
- **SOS-to-Incident pipeline:** 5-state FSM `Detected → Acknowledged → In Progress → Resolved → Closed`, actor+timestamp+note on every transition, WebSocket push to Staff/Guide.
- **Real-time monitoring:** live map (Leaflet.js) of trip positions, device telemetry, gateway connectivity.
- **Trip/Booking/Billing:** packages, bookings, device reservation, check-out/check-in, deposits, damage/late fees, mock/sandbox payment.

Explicitly out of scope: firmware redesign, native mobile apps (responsive web only), production payment gateway, route/trail recommendation, localization, hardware certification (FCC/CE).

3. Actors & role summary

Role	Core capability
Admin	Orgs/users/roles, device types & firmware versions, pricing config, system health, audit logs
Staff	Bookings, device allocation, rental agreements, check-out/in, fees, incident acknowledgment/coordination
Guide	Assigned trips/devices, device handover confirmation, live group view, incident ack + response notes
Customer	Browse/book packages, reserve devices, view booking/rental/payment history

4. Research questions (drive the experiment plan — see roadmap)

- **RQ1:** Gateway reliability — delivery rate, data-loss rate, duplicate rate under intermittent connectivity.
- **RQ2:** Effect of connectivity-loss duration/reconnection pattern on delivery rate, sync latency, priority-ordering compliance.
- **RQ3:** SOS-to-Incident pipeline vs. uncoordinated baseline — MTTA, MTTR, traceability score, completion rate, via controlled drills.

5. Non-functional targets (binding — used as PR/Definition-of-Done gates)

Property	Target
API latency	≤300ms for 95% of requests @ 50 concurrent users
Gateway→cloud sync	≤5s when uplink available
Offline recovery delivery rate	≥99% after reconnection (30s–30min loss, ≥20 trials/condition)
Duplicate prevention	0 duplicate Incidents for repeated <code>eventId</code> delivery (10× replay test)
Priority-ordering compliance	≥99% — all P0 flushed before P2/P3 on reconnect
Concurrency	≥20 simultaneous gateway submissions, 0 loss/duplication
Fleet scale	≥50 devices, multiple concurrent trips, no architecture change
Security	JWT + RBAC + TLS + bcrypt; immutable/append-only incident & device audit logs

6. Tech stack (as registered — see D-001 in decisions log, resolved)

Layer	Technology
Embedded firmware	ESP32/ESP32-S3, C++/PlatformIO, RadioLib, Meshtastic base — inherited, version-locked, not a new deliverable
Gateway bridge	Node.js + TypeScript, SQLite (local queue), MQTT client
Backend	NestJS (TypeScript), REST + WebSocket (Socket.io)
Database	PostgreSQL. ORM: Prisma (resolved via D-001 — FA26SE159 form originally specified TypeORM, team's FINAL draft specified Prisma; Prisma locked as team mandate)
Messaging	MQTT (Mosquitto, gateway↔backend) + Socket.io (backend↔browser)
Frontend	React + TypeScript, role-based views, Leaflet.js map
Auth/AuthZ	JWT + bcrypt; RBAC — CASL (@casl/ability) per the FINAL draft's more specific tech table
DevOps	Docker / Docker Compose, GitHub Actions CI/CD

7. Deliverables (map 1:1 to Jira epics and the roadmap's TP packages)

- TrekLink Operations Platform (web app, 4 role-based views)
- Gateway Bridge module (priority SQLite queue, idempotent MQTT sync, health reporting)
- Event Reliability Layer (eventId scheme, idempotency enforcement, sync audit log, dedup test suite)
- Device Fleet & Rental Management module
- SOS Incident Management module (FSM + WebSocket + audit trail)
- Real-Time Field Monitoring module
- Trip/Booking/Billing module
- Backend API (OpenAPI/Swagger) + WebSocket services
- Technical docs: SRS, 2 UML State Machine diagrams (Device 7-state, Incident 5-state), architecture, DB schema, API reference, test plan, deployment guide, user guide
- Evaluation report: RQ1/RQ2/RQ3 results
- Source + deployment package: gateway/backend/frontend repos, Docker Compose, CI/CD, demo environment

8. Related work / gap statement (condensed)

Meshtastic and consumer satellite messengers (SPOT, Garmin inReach, Zoleo) solve field comms but have no rental/fleet/incident layer. Generic IoT fleet platforms (AWS IoT Core, Azure IoT Hub, Balena) have no LoRa integration or store-and-forward for intermittent connectivity. DTN/store-and-forward research (RFC 4838, MQTT-SN) is protocol-level, not tied to a business workflow. IT incident tools (JIRA SM, PagerDuty, Opsgenie) have no IoT field-event ingestion. TrekLink is the first to combine all five dimensions in one domain-specific platform.

TrekLink Engineering Conventions — Index

This folder is the **TrekLink Developer Handbook**. It is the authoritative, machine-readable source. [TrekLink_Developer_Handbook_v1.0.pdf](#) is a **generated render** of these files — never hand-edited, rebuilt automatically by CI whenever anything here changes. If the PDF and these files ever disagree, **these files win** and the PDF is stale; rebuild it.

Conventions v2 (2026-09-13). Originally ported from a session-driven + spec-driven (Kiro-style) framework used on a .NET project, then adapted to TrekLink's real stack: **NestJS/TypeScript backend, Node.js/TypeScript gateway, React/TypeScript frontend, PostgreSQL, GitHub, Jira**.

Read in this order

#	File	Domain
01	01-session-based-development-and-ssot.md (01-session-based-development-and-ssot.md)	Core philosophy: docs as SSOT, session-based development, crash-safe checkpointing
02	02-spec-driven-development-workflow.md (02-spec-driven-development-workflow.md)	EARS requirements → design → tasks. The spec-before-code gate.
03	03-operational-workflows.md (03-operational-workflows.md)	The four lifecycles: feature dev, debugging, code review, review resolution
04	04-architecture-conventions.md (04-architecture-conventions.md)	Module boundaries, entity conventions, Feature-Sliced frontend
05	05-backend-conventions.md (05-backend-conventions.md)	NestJS modules, DTOs, the response envelope, error handling
06	06-frontend-conventions.md (06-frontend-conventions.md)	FSD, state management, forms/UX, WCAG, Leaflet, WebSocket
07	07-github-workflow-git-conventions.md (07-github-workflow-git-conventions.md)	Branching, commits, PRs, review, merge. The most-referenced chapter.
08	08-ai-agent-steering-and-discipline.md (08-ai-agent-steering-and-discipline.md)	AI agent behaviour: thinking discipline, blast radius, circuit breakers, doc-sync
09	09-doc-driven-scaffold-and-ssot-conventions.md (09-doc-driven-scaffold-and-ssot-conventions.md)	The ignore/ personal garden scaffold and how it reconciles with tracked specs/
10	10-jira-tracking-and-workflow.md (10-jira-tracking-and-workflow.md)	Jira board, statuses, the development loop, backlog↔Jira mapping
11	11-ai-first-doctrine-and-toolchain.md (11-ai-first-doctrine-and-toolchain.md)	Model policy, required toolchain, session workflow, prompting standard
12	12-communication-and-daily-reports.md (12-communication-and-daily-reports.md)	Channels, daily reports, merge announcements, schedule countdown

If you read only three: 07 (git), 10 (Jira), 11 (AI-first). Those three cover everything you touch daily.

What changed in v2 (2026-09-13)

Logged as **Decision D-009**, which supersedes D-003. Every one of these was a live contradiction in v1 — the documented commands did not work.

Was	Now
<code>develop</code> integration branch	<code>dev</code> — no repo ever had a <code>develop</code> branch
<code>[Feature]</code> bracket-tag commits	Conventional Commits — <code>feat(TK-45): add device FSM guard</code>
Dual <code>features/Design_X</code> + <code>features/Implementation_X</code> branches	One branch per unit of work , spec commits first
<code>release/sprint_x</code> branches	Dropped — <code>main</code> is the release
Squash-merge always	Rebase & merge default; squash only when multi-commit; merge commits prohibited
GitHub Issues + Milestones as the tracker	Jira is the single tracker; GitHub Issues = daily reports + bugs
<code>points:*</code> labels	Jira story points, base-5 scale (1/2/3/5/10/15/20/25/30)
Two PR templates	One <code>pull_request_template.md</code> , Design DoD optional
AI agent guidance scattered	Chapters 11 and 12, plus a mandatory session workflow

Adopting these conventions in a repo

All three TrekLink repos use **identical** conventions. There is no per-repo exception.

1. Point the repo's `AGENTS.md` and `CLAUDE.md` at this folder — one canonical file, copied verbatim into `capstone/`, `treklink-docs/`, `treklink-web/`, and `treklink-firmware/`.
2. Copy `../.github/` into the repo root as-is (PR template, issue templates, labels).
3. Run the label setup script in `../.github/labels-and-milestones.md` (`labels-and-milestones.md`) §3.
4. Apply branch protection per `07-github-workflow-git-conventions.md` (`07-github-workflow-git-conventions.md`) §7.
5. Create a tracked `specs/{module}/` folder per module — layout in `02-spec-driven-development-workflow.md` (`02-spec-driven-development-workflow.md`) §2.
6. **Never write production code before the requirements → design → tasks gate has been passed.**

Do **not** vendor a second copy of these files into another repo. A duplicated convention set drifts and then actively teaches the wrong rules — that is exactly what happened with `treklink-web/docs/conventions/`, which has been deleted for this reason. Link to this folder; don't copy it.

Authority

When two documents conflict, the higher tier wins until an ADR in `../00-project-context/03-decisions-and-risk-register.md` (`03-decisions-and-risk-register.md`) supersedes it:

1. **Tier 1** — Project Charter & capstone registration form
2. **Tier 2** — Module specs (`treklink-web/specs/{module}/`) and the backlog (`../03-backlog/`)
3. **Tier 3** — These conventions

4. **Tier 4** — Templates (`../02-templates/`) — starting points, not binding once filled in

5. **Tier 5** — Source code & tests

How We Work

Core Engineering Philosophy: Session-Based Development & Living SSOT

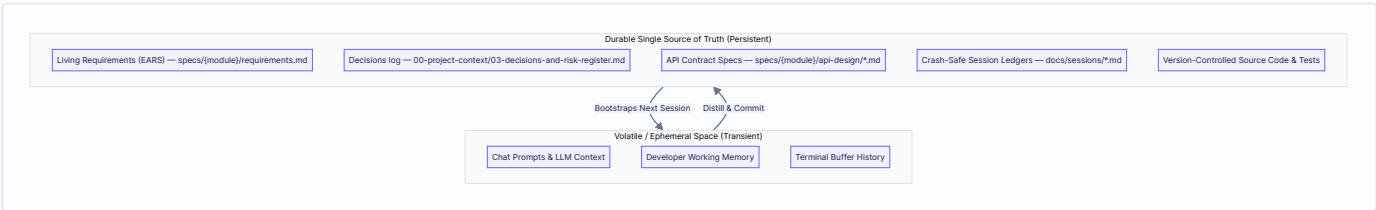
Fundamental Law of Engineering Continuity: In modern software engineering with human-AI collaboration, conversational memory is volatile, ephemeral, and lossy. **The codebase and its living documents are the only durable sources of truth.** Every decision, state transition, and requirement **MUST** be grounded in structured documents, and every work stream **MUST** execute in bounded, crash-resilient sessions.

This applies with extra force on TrekLink: a 5-person team working 2-week sprints across 4 codebases (firmware [frozen], gateway, backend, frontend), reviewed by lecturers at fixed checkpoints (Week 4/8/14) who were not in the room for any implementation decision. If it isn't written down, the reviewer can't grade it and the next session can't resume it.

1. The Core Philosophy

Software projects fail not from lack of code, but from **context drift, ungrounded assumptions, and fragmented memory**. To eliminate this entropy, this framework rests on two pillars:

- 1. **Documents as the Single Source of Truth (SSOT):** If a requirement, architectural choice, or API contract is not written down in an official document, **it does not exist**.
- 2. **Session-Based Development (SBD):** All engineering work is structured into deterministic, self-contained sessions with rigorous pre-flight discovery, milestone checkpoints, and explicit handoffs.



2. Pillar I: Living Documentation as SSOT

2.1 The SSOT Hierarchy

Level	Document Type	Purpose	Mutability
Tier 1	Project Charter, System Architecture & Data Schema	Module boundaries, domain entities, invariants	High ceremony; requires team + supervisor sign-off (esp. at Review 1/2)
Tier 2	Feature Specs (specs/{module}/)	EARS requirements, API contracts, task breakdowns	Updated per module/user story; locked during implementation
Tier 3	Engineering Conventions (docs/conventions/ , this folder)	Architecture rules, code style, Git flow, agent discipline	Living rules; updated on retrospective consensus
Tier 4	Session Ledgers & Worklogs (docs/sessions/)	Current milestone progress, test metrics, open bugs	Frequently updated during active sessions
Tier 5	Source Code & Test Suites	Concrete implementation fulfilling Tiers 1–3	Changes only to satisfy Tiers 1–4

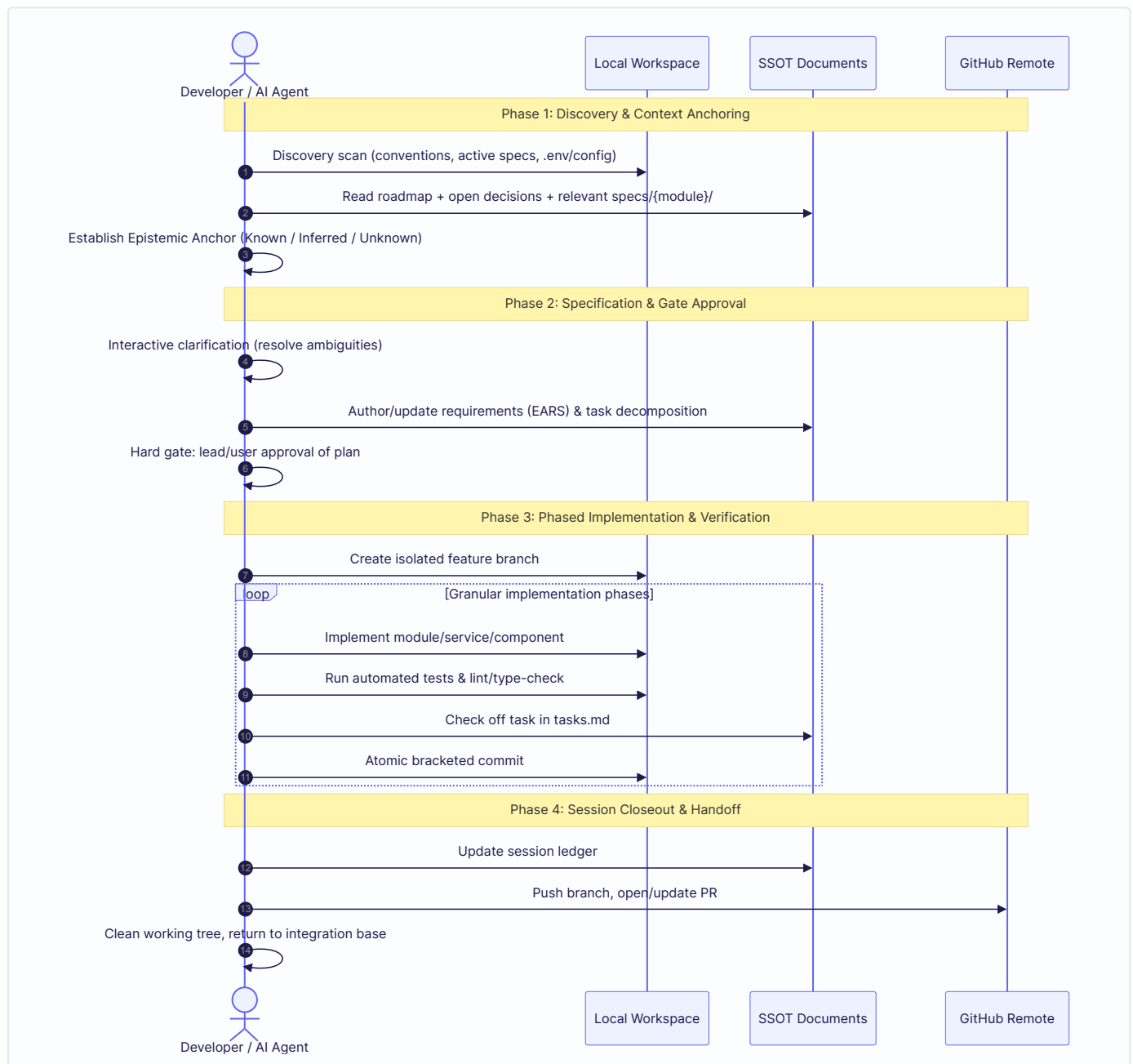
2.2 In-Repo Living Documents

- **Zero third-party dependency for context:** docs live inside the repo, version-controlled, next to the code they describe.
- **Atomic spec-code merges:** when an endpoint changes, its `specs/{module}/api-design/*.md` file is updated in the same PR. This generalizes to every tier above it, not just API endpoints: a finding that touches the charter, a decision, a spec, or a convention gets that document updated in the same working pass it was found in — see `08-ai-agent-steering-and-discipline.md` Stage 2.5b. A doc update queued for "later in the session" is functionally the same failure mode as an endpoint shipped without its spec update — just easier to miss because nothing blocks the PR on it.
- **No stale documentation:** specs gate coding, they aren't written after the fact.

3. Pillar II: Session-Based Development (SBD)

Break open-ended work into bounded execution cycles — 1–3 hours for humans, or one AI-agent context window.

3.1 The 4-Phase Session Lifecycle



Phase 1: Discovery & Context Anchoring

1. **Discovery scan:** conventions, active specs, roadmap position (which sprint/week are we in).
2. **Epistemic Reality Anchor:**
 - **KNOWN:** verified from files read this session.
 - **INFERRED:** derived pattern (state confidence).
 - **UNKNOWN:** missing context — flag **before** proceeding, check `03-decisions-and-risk-register.md` for anything OPEN that blocks you.
3. **Kalama Empirical Proof Standard:** reject "seems reasonable" — accept only what's backed by file evidence or test output.

Phase 2: Specification & Gate Approval

1. Never write production code from a bare chat prompt or vague issue.
2. Author/review the 3-file spec suite in `specs/{module}/`: `requirements.md` → `design.md` → `tasks.md`.
3. Pause and clarify edge cases before coding.

Phase 3: Phased Implementation & Verification

1. Branch off `dev` (or `main` for hotfixes).
2. Work `tasks.md` phase-by-phase.
3. **Read-Before-Write:** never edit a file not opened and inspected this session.
4. Run `npm test` / `npm run lint` / `npm run typecheck` (per-package — see `05-backend-conventions.md`, `06-frontend-conventions.md`) after every atomic phase.

Phase 4: Session Closeout & Crash-Safe Handoff

1. Update the session ledger.
2. Commit with bracket tags (`feat:`, `fix:`, `refactor:`, `docs:` — full list in `07-github-workflow-git-conventions.md`).
3. Rebase on the upstream integration branch.
4. Push and open/update the PR; output a concise handoff so the next session (human or AI) resumes with zero overhead.

4. Crash-Safe Memory Checkpoint Protocol

(Session 4 revision — namespacing for concurrency) The original version of this section allowed a single rolling `docs/sessions/current.md` as an alternative to per-date files. That's unsafe once more than one session (human or AI-agent) can be active on the repo at the same time — a five-person team plus their own AI agents, each possibly running more than one session paper window, is exactly that case. **Per-session-instance files are now mandatory, not optional.** This isn't paperwork: this exact gap caused a real loss this term — a Session 4 audit produced several non-obvious findings (firmware wire-format facts, scaffold bugs) that sat only in chat output, in a repo that already had this protocol on paper, because nothing forced them to disk mid-session. See `08-ai-agent-steering-and-discipline.md` Stage 2.5 for the enforcement rule.

Tracked tier — `docs/sessions/<YYYY-MM-DD>-<HHMM>-<topic-slug>.md`, one file per session instance (e.g. `2026-09-13-1830-gateway-sync-audit.md`). `docs/sessions/current.md` is no longer where work-in-progress gets written — it becomes a short **index**: a few lines per active or recent session file, updated whenever a session file is created or closed out, so a new session can see at a glance what else is/was in flight before re-deriving it.

Personal scratch tier — same pattern, one level down: `ignore/[name]/docs/sessions/<YYYY-MM-DD>-<HHMM>-<topic-slug>.md` per session instance, replacing the old single shared `ignore/[name]/docs/current-progress.md`. That file still exists but changes role: a short rolling pointer/summary ("latest: see `sessions/2026-09-13-1830-gateway-sync-audit.md`"), not the place high-frequency scratch writes go — that would recreate the same collision risk one level down.

Maintain a session file at every major milestone, and — per the enforcement rule in `08-ai-agent-steering-and-discipline.md` Stage 2.5 — immediately on any non-obvious finding, not only at milestones:

```
# Session Checkpoint: 2026-09-14 - gateway-sync

## 1. Active Focus & Objectives
- Module: Gateway Bridge - SQLite priority queue (TP2)
- Branch: feat/TK-45-gateway-priority-queue

## 2. Completed Milestones
- [x] Phase 1: SQLite schema for P0-P3 queue tiers
- [x] Phase 2: Enqueue/dequeue with priority ordering + unit tests

## 3. Immediate Next Steps (for resuming session)
1. Implement MQTT publish-on-reconnect flush routine
2. Add integration test: 30s / 2min / 5min connectivity-loss simulation
3. Wire health-reporting endpoint (queue depth per tier)

## 4. Uncommitted State / Known Blockers
- Working tree: clean (commit `a1b2c3d`)
- Test status: 18/18 passing
- Open questions: none (D-001 ORM decision doesn't block gateway work)
```

5. Context Window & Token Budget Management (AI Agents)

1. **Context decay:** past ~70–80% capacity, reasoning fidelity drops. Compact proactively.
 2. **Proactive compaction:** grep for the specific error, don't dump whole logs; read targeted line ranges, not whole large files; summarize findings into the session ledger instead of re-reading the repo every turn.
 3. **Handoff over compounding:** for multi-day work (e.g. TP2's whole Gateway Bridge), split into clean sessions — Session N ends by committing + updating `tasks.md`; Session N+1 boots by reading the spec, with a full context budget for implementation.
-

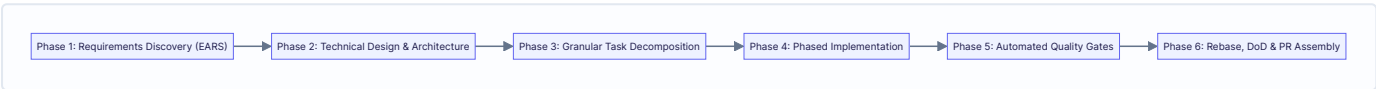
6. Tenets of Iterative Refinement

1. **Working software at every step:** never leave `dev` in a broken/uncompilable state. Incomplete work stays on its feature branch.
2. **Minimal sufficient change:** solve the registered requirement — no speculative abstractions ahead of the TP schedule.
3. **Continuous verification:** a feature is done when tests prove it satisfies the EARS requirements, not when code is typed.

Spec-Driven Development Workflow (SDSDW)

Core Axiom: Specification is the architectural blueprint of software. Writing code without an approved specification is constructing a building without an engineering schematic. This workflow guarantees requirements, interactions, contracts, and tasks are formally verified before a single line of production code is written — this is the "Kiro spec-driven development" pattern (`requirements.md` → `design.md` → `tasks.md`) combined with session-based development from `01-session-based-development-and-ssot.md` .

1. Executive Overview



Universal tenets

1. **Spec before code** (Read → Clarify → Design → Code). Never guess business rules, RBAC edge cases, or error conditions — TrekLink's grading depends on traceable requirements (RQ1–RQ3 must map back to specific FRs).
2. **Standardized directory topology:** specs live in `specs/{module_name}/` inside each repo, next to the code.
3. **EARS-compliant criteria** for every functional requirement.
4. **Mermaid** for all architecture diagrams, state machines (Device 7-state, Incident 5-state — these are graded deliverables), and sequence flows.
5. **In-codebase tracked deliverables:** specs, migrations, and docs are committed, never left in an uncommitted scratchpad.
6. **Docs sync at the moment of discovery, not at phase-exit cleanup** (`08-ai-agent-steering-and-discipline.md` Stage 2.5b). If implementation, testing, or a code-level investigation reveals that a requirement, design assumption, or task in this module's spec was wrong or incomplete, fix `requirements.md` / `design.md` / `tasks.md` in that same working pass — immediately, not queued for a later "update the docs" step. Phase 6's `6.3` below is a final **audit** that sync already happened throughout, not the first time specs get touched.

2. Directory Structure (`specs/{module_name}/`)

TrekLink modules (per the roadmap's TP mapping and `07-github-workflow-git-conventions.md` 's `module:*` labels): `auth` , `devices` , `rentals` , `trips` , `gateway-sync` , `incidents` , `monitoring` , `billing` , `frontend` .

```

specs/{module_name}/
├── requirements.md          # EARS-format business rules & criteria
├── design.md               # Architecture, domain models, sequence flows, state machines
├── tasks.md                # Phased, granular implementation checklist
└── api-design/             # One file per endpoint, using 02-templates/04-api-endpoint-
    template.md
        ├── 00-api-testing-guide.md    # Swagger / Postman / curl walkthrough
        ├── 01-post-devices-register.md
        ├── 02-get-devices-list.md
        └── ...

# Example for the gateway-sync module:
specs/gateway-sync/
├── requirements.md          # eventId scheme, P0-P3 priority rules, idempotency criteria
├── design.md               # SQLite queue schema, MQTT topic design, reconnect/flush sequence diagram
├── tasks.md
└── api-design/
    └── 01-post-events-ingest.md

```

Frontend-only concerns (screens, component architecture) live under `specs/frontend/` with the same 3-file pattern, referencing backend `api-design/` files rather than duplicating them.

3. Phase-by-Phase Execution Protocol

Phase 1: Requirements Discovery & EARS Specification

1. Context ingestion

Review the relevant section of `00-project-context/01-project-charter.md`, existing entities in `design.md` of dependent modules, and upstream/downstream module boundaries (e.g. `incidents` depends on `devices` + `rentals` existing first).

2. Interactive clarification interview (mandatory quality gate)

Before writing specs or code, formulate 3–5 high-value clarifying questions targeting:

- **Domain edge cases:** e.g. what happens to an in-field device's rental record if a trip's Guide reports it lost?
- **Authorization:** which of Admin/Staff/Guide/Customer can trigger this action; ownership checks (a Guide can only see their own assigned trips).
- **Reliability:** for gateway-sync/incidents, does this requirement interact with idempotency or priority ordering?

MANDATORY HARD STOP: stop after presenting the questions and wait for an explicit answer. No specs, designs, tasks, or code before that.

3. Authoring `requirements.md` in EARS syntax

Pattern	Template	TrekLink example
Ubiquitous	<i>The system SHALL [response]</i>	The system SHALL record actor (user ID + role) and UTC timestamp on every incident state transition.
Event-driven	<i>WHEN [trigger], the system SHALL [response]</i>	WHEN a valid SOS event passes the idempotency check, the system SHALL create exactly one Incident and notify Staff + Guide via WebSocket within 2 seconds.
State-driven	<i>WHILE [state], the system SHALL [response]</i>	WHILE a device is in <code>Maintenance</code> status, the system SHALL reject any rental allocation attempt referencing it.
Unwanted behavior	<i>IF [condition/error], THEN the system SHALL [response]</i>	IF the same <code>eventId</code> is delivered more than once, THEN the system SHALL discard the duplicate without creating a second Incident.
Optional feature	<i>WHERE [feature/flag], the system SHALL [response]</i>	WHERE sandbox payment mode is enabled, the system SHALL mark all transactions with a <code>sandbox: true</code> flag.

Phase 2: Technical Architecture & Design (`design.md`)

Backend/gateway modules:

1. Domain & data modeling — entities, enums, FK constraints, state machines (render as Mermaid `stateDiagram-v2` for Device/Incident FSMs).
2. Service & queue architecture — NestJS providers/services, MQTT topics, SQLite queue schema for gateway-sync.
3. Endpoint spec (`api-design/*.md`) using `02-templates/04-api-endpoint-template.md` .

Frontend:

1. Component architecture (Feature-Sliced Design — see `06-frontend-conventions.md`).
2. Server vs. client vs. form state lifecycle.
3. Validation schemas (Zod) mirroring backend `class-validator` DTOs 1:1.

Phase 3: Granular Task Decomposition ([tasks.md](#))

```
# Implementation Tasks: [Module Name]

## Phase 1: Foundation & Domain Modeling
- [ ] 1.1 Define entities/enums (Prisma, per D-001)
- [ ] 1.2 Write migration(s)
- [ ] 1.3 Add DTOs and error codes

## Phase 2: Core Service / Mutation Logic
- [ ] 2.1 Implement service method(s) with class-validator DTO input
- [ ] 2.2 Wrap multi-table writes in a transaction
- [ ] 2.3 Unit test success + failure paths

## Phase 3: Query / Retrieval
- [ ] 3.1 Implement paginated list/query endpoints
- [ ] 3.2 Unit test filter/sort/pagination boundaries

## Phase 4: API Presentation Layer
- [ ] 4.1 Controller + Guards (JWT + RBAC/CASL)
- [ ] 4.2 Swagger/OpenAPI annotations
- [ ] 4.3 Integration tests for the HTTP contract (match api-design/*.md exactly)

## Phase 5: Frontend Integration (if applicable)
- [ ] 5.1 API client + TypeScript types
- [ ] 5.2 Form (RHF + Zod) or view wired to TanStack Query
- [ ] 5.3 Loading/empty/error states

## Phase 6: End-to-End Verification & DoD Audit
- [ ] 6.1 Full test suite green
- [ ] 6.2 Lint + typecheck clean
- [ ] 6.3 Audit: confirm `specs/{module}/api-design/*.md` already matches actual behavior — it
  should have been updated live in Phases 1-5 per tenet #6 above, so this is a check, not
  the first edit. If it's out of sync here, that's a process miss to flag, not routine cleanup.
```

Phase 4: Phased Implementation

Never skip ahead. Atomic commits per subtask with bracket tags ([07-github-workflow-git-conventions.md](#)). Zero magic strings — centralize error codes and route constants.

Phase 5: Automated Testing & Verification Gates

- **Unit tests:** every domain rule, FSM transition, validator branch (esp. idempotency and priority-ordering logic — these are graded NFRs).
- **Integration tests:** DB persistence, unique constraints, transaction rollback.
- **Static analysis:** 0 build warnings, 0 lint errors, strict TypeScript.

Phase 6: GitHub Flow, Rebase & PR Assembly

1. One branch: `feat/TK-nn-short-desc` off `dev`, carrying spec commits first, then code+tests (see [07-github-workflow-git-conventions.md](#) §2.3).
2. `git fetch origin && git rebase origin/dev`.
3. Open the PR — `.github/pull_request_template.md` loads automatically. DoD checked, test output pasted, specs referenced, `Model used:` declared. Fill the optional Design DoD block if this branch introduced or changed a spec. Then move the Jira card to `IN REVIEW` and ping the reviewer in Zalo.

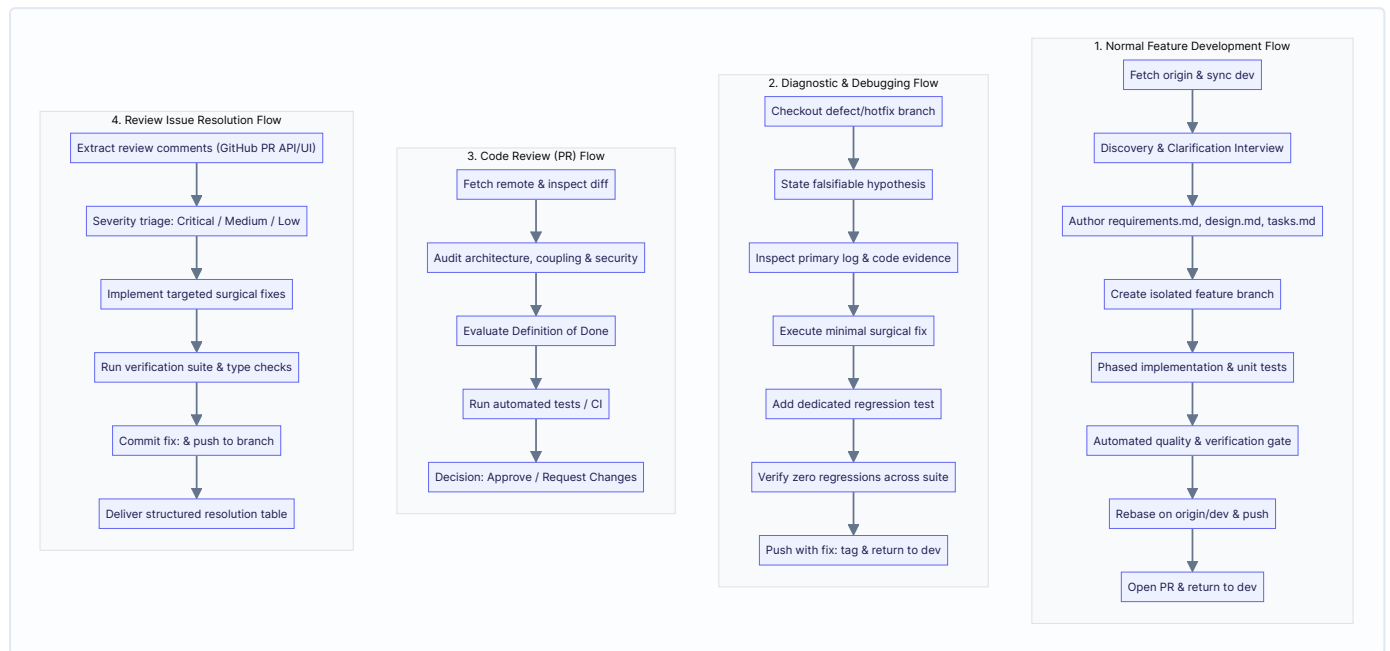
4. Universal Phase Exit Gate Matrix

Phase	Core Deliverable	Mandatory Exit Gate
1A: Clarification	Interview	STOP & WAIT for answers
1B: Requirements	<code>requirements.md</code>	All rules in unambiguous EARS syntax
2: Technical Design	<code>design.md</code> + <code>api-design/</code>	Architecture/schema/Mermaid diagrams reviewed
3: Task Decomposition	<code>tasks.md</code>	Granular checklist approved by lead
4: Implementation	Source code	Matches design, minimal sufficient change
5: Verification	Test suite	100% pass, 0 lint/type errors
6: Delivery & Review	Rebased branch & PR	Linear history, DoD verified, PR description compiled

Engineering Lifecycle & Operational Workflows

Four canonical, repeatable workflows for feature development, defect diagnosis, code review, and review-comment resolution. Applies equally whether the acting party is a teammate or an AI coding agent.

1. Executive Workflow Matrix



2. Lifecycle 1: Normal Feature Development Flow (Spec-First)

Step 1: Baseline Synchronization

```
git fetch origin
git checkout dev
git pull origin dev
```

Step 2: Discovery, Clarification & Spec Authoring

Review the target module's charter section, existing entities, and dependent modules. Run the **Phase 1A Mandatory Gate** clarification interview (see [02-spec-driven-development-workflow.md](#)) — hard stop until answered. Author `specs/{module}/requirements.md` → `design.md` → `tasks.md` → `api-design/`.

Step 3: Branch Creation & Isolation

```
git checkout -b feat/TK-45-device-registration
```

One branch per unit of work. Spec commits and implementation commits live on the **same** branch, spec first — see [07-github-workflow-git-conventions.md](#) §2.3. Branch type is `feat/`, `fix/`, `docs/`, or `chore/` as appropriate.

Step 4: Phased Clean Implementation

Follow `tasks.md` phase-by-phase. Commit small, atomic units using Conventional Commits with the Jira key as scope — `feat(TK-45): ...`, `fix(TK-45): ...`, `test(TK-45): ...` (see `07-github-workflow-git-conventions.md` §3).

Step 5: Quality Gate & Verification

```
# Backend (NestJS) — from the backend package root
npm run lint && npm run typecheck && npm test && npm run test:e2e

# Gateway (Node/TS)
npm run lint && npm test

# Frontend (React)
npm run lint && npm run build && npm test
```

Confirm **100% test pass rate**, 0 lint errors, clean build. Backend endpoint behavior must match `specs/{module}/api-design/*.md` exactly (same response envelope — see `05-backend-conventions.md`).

Step 6: Upstream Rebase & Push

```
git fetch origin
git rebase origin/dev
# re-run Step 5 after resolving any conflicts
git push --force-with-lease origin feat/TK-45-device-registration
```

Step 7: PR Assembly & Return to Base

Open the PR with `.github/pull_request_template.md`, fill the DoD, and **ping the reviewer in Zalo**. Move the Jira card to `IN REVIEW`. Checkout back to `dev`.

3. Lifecycle 2: Diagnostic & Debugging Flow (Hypothesis-Driven)

Step 1: Checkout Target Branch

```
git checkout feat/TK-45-device-registration
# OR for an urgent defect — see 07-github-workflow-git-conventions.md §2.4
# for which branch to cut from (dev vs main):
git checkout -b hotfix/TK-99-jwt-expiry-crash
```

Step 2: Hypothesis Formulation & Investigation

State a single falsifiable hypothesis. Inspect logs/stack traces/source lines with real tool calls, not guesses. Assess blast radius — e.g. a fix to the idempotency key logic touches every consumer of `eventId` across gateway + backend.

Step 3: Minimal Surgical Fix

Address the root cause only. No unrelated refactors or formatting changes bundled in.

Step 4: Dedicated Regression Test

Add a test that reproduces the failure without the fix and passes with it. Run the full suite — zero downstream regressions.

Step 5: Push & Return to Base

```
git commit -m "fix(TK-99): precise description of the resolved defect"
git push --force-with-lease origin hotfix/TK-99-jwt-expiry-crash
git checkout dev
```

Output a diagnostic summary: root cause, fix mechanics, regression-test proof.

4. Lifecycle 3: Pull Request & Code Review Flow

Step 1: Branch Fetch & Diff Inspection

```
git fetch origin
git log --oneline origin/dev..origin/{target_branch}
git diff origin/dev...origin/{target_branch}
```

Or via GitHub CLI: `gh pr diff {number}`, `gh pr checks {number}`.

Step 2: Architectural & Code Quality Audit

- ☐ Module boundaries respected (no cross-module repository access — see `04-architecture-conventions.md §4`)
- ☐ DTOs strictly typed with `class-validator`, matching `api-design/*.md`
- ☐ Errors use the standard envelope (`result` / `isSuccess` / `statusCode` / `message`), not ad-hoc shapes
- ☐ Security: mutating endpoints have both a JWT guard and a role/CASL policy check — e.g. `@UseGuards(JwtAuthGuard, PoliciesGuard)` on any endpoint that mutates device/rental/incident state
- ☐ Performance: no N+1 queries, indexed lookups on `eventId` /device ID/trip ID
- ☐ Hygiene: no debug logs, dead code, hardcoded secrets, or `.env` values committed

Step 3: Definition of Done Verification

Confirm DoD checklist in the PR description is honestly checked and CI is green.

Step 4: Structured Review Output

- Decision:** `[APPROVED]` | `[CHANGES REQUESTED]` | `[NEEDS FIXES]`
 - Findings:** exact file + line references, drop-in diff suggestions.
-

5. Lifecycle 4: Review Issue Resolution Flow

Step 1: Retrieve Review Notes

Pull comments from the GitHub PR conversation thread (`gh pr view {number} --comments` or the UI).

Step 2: Severity Triage

- **Critical:** missing auth/RBAC guard, data corruption risk, broken CI, idempotency/priority-ordering violation.
- **Medium:** performance issue, missing transaction rollback, unhandled edge case.
- **Low/Hygiene:** formatting, naming, doc updates.

Step 3: Implement Targeted Fixes

Address each comment directly, on the same branch, without unrelated scope changes.

Step 4: Verification & Handoff

```
npm test && npm run lint
git commit -m "fix: Address review feedback on {topic}"
git push
```

Output a resolution table:

#	Reviewer Comment	Severity	Applied Fix	Status
1	Missing role guard on device retire endpoint	 Critical	Added <code>@Roles('Admin')</code> + CASL <code>can('retire','Device')</code> check	Resolved
2	Duplicate-SOS test only covers 2 retries, NFR needs 10	 Medium	Extended test to 10× replay of same <code>eventId</code>	Resolved

Delivery

GitHub Workflow, Branching, Commits & Pull Requests

Conventions v2 — supersedes everything written before 2026-09-13. The previous version of this file described a `develop` branch and a dual `Design / Implementation` branch model inherited from the school's `Git_Lab_Guide.pdf`. Neither matches reality: no repository has ever had a `develop` branch, and the dual-branch model doubles PR overhead for a 5-person team on a 13-week term. Decision **D-009** records the change and supersedes **D-003**.

PR = MR. "Pull Request" (GitHub) and "Merge Request" (GitLab) are used interchangeably across this documentation and in team chat. They mean the same thing and follow the same workflow. The school's guide says MR; GitHub says PR; we say either.

1. Repositories

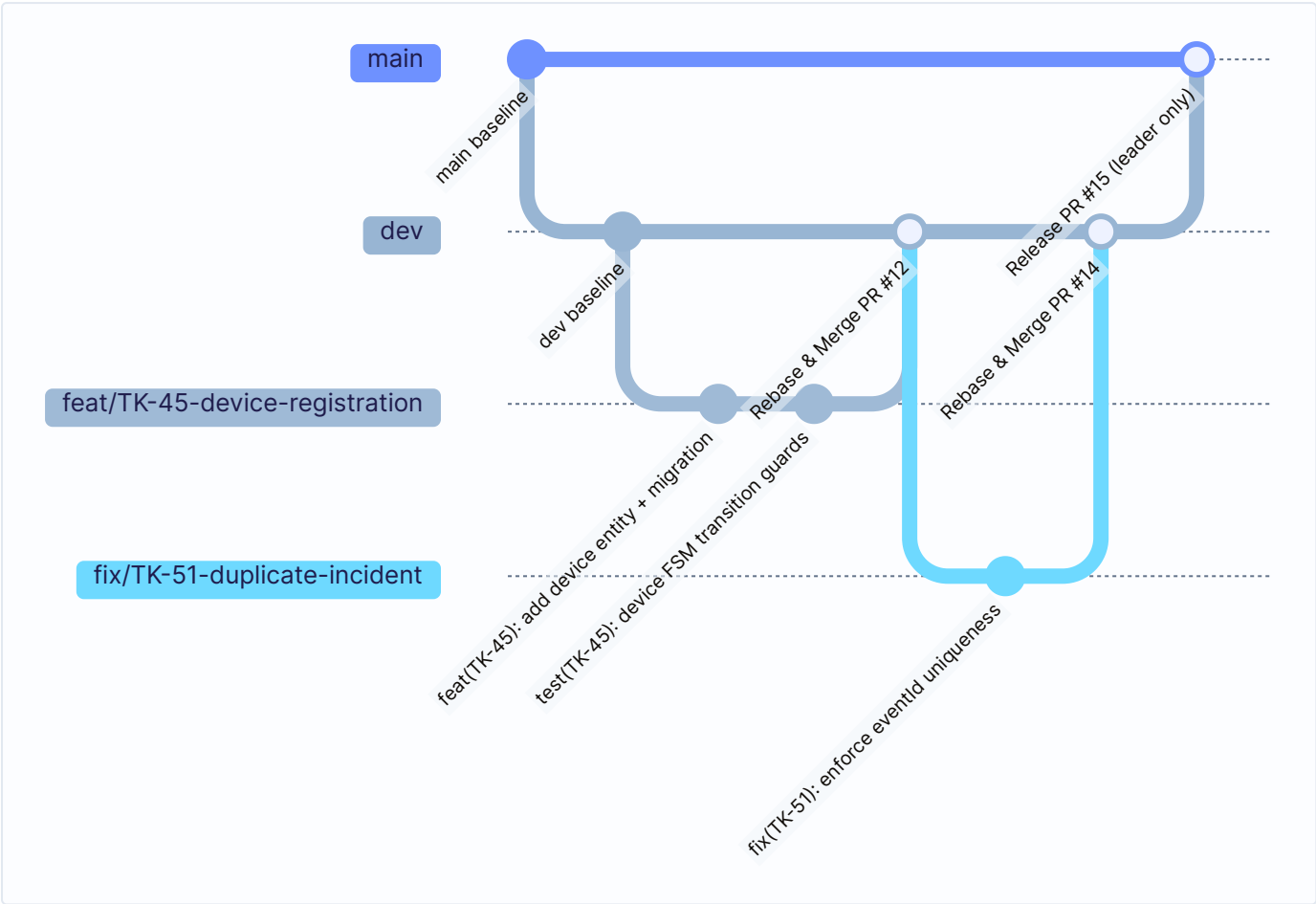
Three repositories under `github.com/TrekLink-Team`, cloned as siblings inside one `capstone/` parent folder. See the root `README.md` §1 for the full layout and why the parent folder matters for AI agents.

Repo	Contents	Branch model
<code>treklink-docs</code>	This documentation set (SSOT). No application code.	Full model below
<code>treklink-web</code>	The active build: <code>gateway/</code> , <code>backend/</code> , <code>frontend/</code> as workspace packages	Full model below
<code>treklink-firmware</code>	Inherited SU26 firmware — editable per D-008, surgical fixes only	Full model below

All three repos use identical conventions. Same branches, same commit format, same PR process, same AI agent rules. There is no "this repo is different" exception. A teammate who learns the workflow in one repo knows it in all three.

The `capstone/` parent folder is also git-versioned but **never published** — it exists so an AI agent opened on `capstone/` can read all three repos plus `Documents/` in one context window.

2. Branch Model



2.1 The branches

Branch	Branches from	Merges into	Purpose
main	—	—	Protected. Production / demo-ready state. This <i>is</i> the release — there are no <code>release/*</code> branches.
dev	main	main	Protected. Active integration branch. Everything lands here first.
feat/*	dev	dev	New capability. One per Jira story or task.
fix/*	dev	dev	Defect in unreleased work (found on <code>dev</code> , or a GitHub Issue bug).
hotfix/*	dev or main	see §2.4	Urgent defect. Branch point depends on where the fault is.
docs/*	dev	dev	Documentation-only changes. Primary branch type in <code>treklink-docs</code> .
chore/*	dev	dev	Dependencies, tooling, config, cleanup. No behaviour change.

No direct pushes. Ever. To any branch. Every change — including a one-line typo fix, including the leader's own changes — is branched, pushed, and merged through a PR. There is no exception for "it's tiny" or "it's urgent". Urgency is what `hotfix/*` is for, and a hotfix is still a PR.

2.2 Branch naming

```
{type}/{JIRA-KEY}-{short-kebab-description}
```

Example	Notes
feat/TK-45-device-registration	Standard case. Jira key, then 2–4 words.
fix/TK-51-duplicate-incident-on-replay	
docs/TK-88-handbook-v1	
chore/TK-12-bump-nestjs-10	
hotfix/TK-99-jwt-expiry-crash	
chore/repo-gitignore-cleanup	No Jira key only when no card exists (misc housekeeping). Rare — prefer creating the card.

Rules:

- **Lower-case, kebab-case** after the type. No underscores, no `CamelCase`, no spaces.
- **Jira key uppercase**, exactly as Jira issued it (`TK-45` , not `tk-45`).
- Keep it under ~50 characters total. The description is a reminder, not a summary.
- One branch per unit of work. Do not accumulate unrelated changes on a long-lived personal branch.

Why the Jira key is in the branch name: the org-level Jira↔GitHub integration scans branch names, commit messages, and PR titles for issue keys. With the key present, the Jira card's Development panel auto-populates with the branch, its commits, and its PR — for free, with zero extra effort from you. That is the traceability the supervisor and the council look for, and it costs nothing but a naming habit.

2.3 Design and implementation live on the same branch

The old dual-branch model (`features/Design_X` + `features/Implementation_X`) is **retired**. A single `feat/*` branch carries the spec commits *and* the implementation commits, in that order.

You still cannot write code before the spec exists — the spec-before-code gate in `02-spec-driven-development-workflow.md` (`02-spec-driven-development-workflow.md`) is unchanged and non-negotiable. The gate is enforced by *commit order within the branch*, not by a second branch: the reviewer reads `git log` and sees `docs(TK-45): requirements + design for devices` landing before `feat(TK-45): implement device entity` . That gives the same review independence at half the PR overhead.

The Design DoD section in the PR template is therefore **optional** — fill it in when the branch introduced or changed a spec, skip it when it didn't.

2.4 Hotfix protocol

Hotfixes are branch-specific. Which branch you cut from depends on where the fault actually is.



Before cutting any hotfix: bring every branch up to date. A hotfix applied to a stale base is how a fix gets silently reverted by the next merge. Concretely:

```
git fetch origin
git checkout main && git pull origin main
git checkout dev && git pull origin dev
# confirm dev contains main's history:
git log --oneline main ^dev      # must print nothing
```

If that last command prints commits, `dev` is behind `main` — sync it before hotfixing, or the fix will only exist on one side.

A hotfix landed on `main` **must also reach** `dev`, always. A fix that lives only on `main` will be undone the next time `dev` is promoted.

3. Commit Conventions

3.1 Format

```
{type}({scope}): {imperative description}
```

The scope is the **Jira key** when the work has a card, which is almost always.

```
git commit -m "feat(TK-45): add device FSM transition guard"
git commit -m "fix(TK-51): reject duplicate eventId before incident creation"
git commit -m "docs(TK-88): add Jira workflow chapter to handbook"
git commit -m "chore: refactor conventions files"           # no card – plain, unscoped
```

3.2 Types

All standard Conventional Commit types are available:

Type	Use for
feat	New capability
fix	Bug fix
docs	Documentation only
test	Adding or fixing tests
refactor	Restructuring with no behaviour change
perf	Performance work
style	Formatting, whitespace, lint fixes — no logic
chore	Dependencies, tooling, config, housekeeping
ci	CI/CD pipeline changes
build	Build system, bundler, compiler config
revert	Reverting a previous commit

3.3 Scope

Mandatory **where meaningful**. Use the Jira key. When the change is genuinely general, cross-cutting, or has no card — repository housekeeping, a sweep across every file, a `.gitignore` tweak — **omit the scope entirely** rather than inventing one:

```
chore: refactor conventions files      # correct — general, no card
chore(misc): refactor conventions files # wrong — "misc" is noise
```

3.4 Quality rules

1. Understandable from the subject line alone.
2. No vague messages. `"update"`, `"fix bug"`, `"wip"`, `"asdf"` tell the next reader nothing.
3. One logical change per commit.
4. Never bundle formatting churn with functional changes — that is what `style:` is for.
5. Imperative mood: "add", not "added" or "adds".
6. English only. See §6.

3.5 Commit messages are a practice, not a gate

Commit hygiene must never block a delivery. There is no commitlint hook and no CI check on commit messages, deliberately. When you are shipping at 23:50 the night before a review, a commit that says `fix: login` is acceptable and nobody will chase you for it.

Adhere to the format whenever you reasonably can — it is what makes `git log --oneline` readable during Review 2 prep and what feeds the Jira Development panel. But a perfect commit history on an unshipped feature is worth nothing. Ship, then tidy if there's time.

4. Rebase Discipline

We keep linear history. No merge commits anywhere, ever.

4.1 Before opening a PR

```
git fetch origin
git rebase origin/dev
# resolve conflicts, preserving business rules – re-read the spec if unsure
npm test # must pass before you push
git push --force-with-lease origin feat/TK-45-device-registration
```

Always `--force-with-lease`, never `--force`. It refuses to overwrite work you haven't seen, which matters when an AI agent or a second machine has pushed to the same branch.

4.2 After *any* PR merges into `dev` — everybody rebases

This is a team-wide obligation, not a courtesy:

- If you have no open PR: `git checkout dev && git pull origin dev` before you branch again.
- If you have an open PR: rebase it onto the new `dev` and force-push. Your PR now reviews against current reality.

```
git fetch origin
git checkout feat/TK-45-device-registration
git rebase origin/dev
npm test
git push --force-with-lease
```

This is why a merge into `dev` must be announced in the Zalo group (see [12-communication-and-daily-reports.md](#) ([12-communication-and-daily-reports.md](#))). The announcement is the signal for everyone to stop, pull, and rebase. Merging silently and letting four teammates discover it through conflicts three hours later is the single most expensive avoidable mistake on this project.

5. Pull Requests

5.1 Opening a PR

```
git push -u origin feat/TK-45-device-registration
gh pr create \
  --base dev \
  --title "feat(TK-45): device registration" \
  --body-file .github/pull_request_template.md \
  --assignee @me \
  --reviewer {reviewer-github-handle}
```

Then ping the reviewer in Zalo with the PR link. GitHub notifications are not reliably read; the Zalo ping is what actually starts the review clock.

Draft PRs are allowed as a work-in-progress signal, and reviewers will not look at them. When you want a review, mark it Ready for Review — that is the explicit handoff. A draft PR sitting for two days is not "waiting for review", it is not in the queue at all.

Stacked PRs (a PR based on another open PR) are allowed. Set the `--base` to the parent branch and say so in the description, so the reviewer knows the diff includes inherited commits.

5.2 Who reviews

The backlog's **Reviewer** field is the default reviewer and it is binding. It is set per story in `../03-backlog/02-user-stories.md` (`02-user-stories.md`).

- The leader (**KhoaDD**) is the Product Owner and default reviewer for most stories.

- **TanNB** and **LongLP** are delegated reviewers — the leader may assign review to either when capacity requires it.
- **The PR author may re-assign the reviewer** if the assigned reviewer is busy, without asking permission first. Tell the new reviewer; you do not need to tell the busy one.

5.3 Self-approval — the narrow exception

A PR author may self-approve and merge **ONLY** if the change meets every one of these:

- under **50 lines** changed, and
- **no behavioural impact** — nothing a test or a user could observe differently, and
- it is housekeeping: a chore, a cleanup, a file move, formatting, a comment, a doc typo.

Everything else requires a second party. Including the leader's own PRs.

The Product Owner is not exempt. When the leader authors a non-trivial PR, the leader **must run a separate AI review pass on it before merging** — a fresh agent session, pointed at the diff, with no memory of having written the code. The output of that pass goes in the PR as a review comment.

The reasoning is not bureaucratic: work is not correct because of who produced it. An author reviewing their own code re-reads their intent, not their output. A fresh reviewer — human or agent — reads what is actually there. That asymmetry is the entire value of review, and it does not disappear when the author happens to be the PO.

5.4 Reviewing a PR

Pull the branch and actually run it. Reading the diff in the browser is not a review.

```
gh pr checkout 42
npm ci

# Run the same gates the author was supposed to run:
npm --prefix backend run lint && npm --prefix backend run typecheck && npm --prefix backend test
npm --prefix gateway run lint && npm --prefix gateway run typecheck && npm --prefix gateway test
npm --prefix frontend run lint && npm --prefix frontend run typecheck && npm --prefix frontend run build

# Bring up the local stack if the change touches runtime behaviour:
docker compose up -d
```

Unit tests are not optional and not negotiable.

- The author runs them and they pass **before** the PR is opened.
- The reviewer runs them **again** on their own machine.
- A feature with no tests is not done. Shipping untested code by simply not writing a test is cheating the gate, and the reviewer should request changes on that basis alone.
- A **failing test is evidence of a flaw** until the author explains otherwise. "That test is flaky" is a claim that needs proof, not a dismissal.

Then decide, using the severity vocabulary in [03-operational-workflows.md](#) ([03-operational-workflows.md](#)) §4: **[APPROVED]** · **[CHANGES REQUESTED]** · **[NEEDS FIXES]** .

On "LGTM". It is a legitimate approval for a genuinely small, genuinely clear change you have actually read and run. It is not a way to clear your review queue. If the diff is 400 lines and your review is four characters, you did not review it — you signed for it. Use it gracefully.

5.5 Review SLA

- A PR opened during working hours (**08:00–17:00**) is reviewed **before 17:00 that day**.
- A PR left unreviewed at 17:00 becomes **top priority the next morning**.

- Absolute ceiling: **1 day**. After that the author escalates in Zalo, or re-assigns per §5.2.

5.6 Mid-review fixes

When changes are requested:

1. Open the PR skill / read the review comments in full before touching anything.
2. `git checkout feat/TK-45-...` — confirm you are on the right branch. Fixing a review on `dev` is a classic and painful mistake.
3. Fix, commit (`fix(TK-45): address review - validate deviceId length`), rebase if `dev` moved.
4. `git push --force-with-lease` .
5. **Comment on the PR** saying what you changed and what you deliberately did not. Then wait. Pushing silently does not re-request review.

If you are stuck on a review comment rather than disagreeing with it, move the Jira card to `NEEDS HELP` and say so in the PR — see `10-jira-tracking-and-workflow.md` (`10-jira-tracking-and-workflow.md`) §4.

5.7 Merging

Situation	Method
Single commit on the branch	Rebase and merge
Multiple commits on the branch	Squash and merge
Anything at all	Never "Create a merge commit"

"Create a merge commit" is prohibited in this project. It produces a non-linear history that makes `git log --oneline` , `git bisect` , and Review-2 traceability materially harder. Disable it in repository settings so the option is not offered.

Branch deletion after merge:

- `feat/*` , `fix/*` , `docs/*` , `chore/*` , `hotfix/*` → **delete the source branch** ("Delete branch after PR"). They are temporary by definition and a stale branch list hides the live work.
- `dev` → `main` → **never delete the source branch**. `dev` is permanent.

Who merges into `main` : the leader only. The supervisor never touches the repository — all review flows through the leader, who opens and merges the `dev` → `main` PR.

6. Language Rule (Hard Requirement)

ENGLISH ONLY in every artifact this project produces: source code, identifiers, comments, commit messages, branch names, PR titles and descriptions, review comments, specs, conventions, documentation, Jira cards, and diagrams.

You may prompt an AI agent in Vietnamese. English is preferred and gets better results, but chatting to your agent in Vietnamese is fine. Regardless of what language you prompted in, **everything the agent writes into the repository must be English.**

The only sanctioned exception is a daily-report entry, where Vietnamese is tolerated when English would cost you clarity — see `12-communication-and-daily-reports.md` (`12-communication-and-daily-reports.md`) §3.

This is not stylistic preference. The council reads the artifacts, the roadmap mandates English (§06 rule 5), and a mixed-language codebase is unreviewable.

7. Branch Protection Settings

Configure once per repository, by the leader, in Settings → Branches.

main

- Require a pull request before merging.
- Require **1 approval**.
- Block force-push **for all members except the leader**.
- Disallow "Create a merge commit"; allow rebase-merge and squash-merge only.
- Do not allow deletions.

dev

- Require a pull request before merging.
- Require **1 approval** from the assigned reviewer (see §5.2, §5.3 for the self-approval carve-out).
- Block force-push **for all members except the leader**.
- Disallow "Create a merge commit"; allow rebase-merge and squash-merge only.

CI is an indicator, not a gate

A red CI does not block merging. CI is advisory on this project — deliberately. A 13-week term with a hardware dependency produces plenty of legitimately-red pipelines (missing device, flaky integration, an unconfigured secret) and a hard gate would mean waiting on infrastructure instead of delivering.

But red CI means merge with caution. Specifically:

- The **leader may merge** a red-CI PR at their discretion.
- A **non-leader reviewer may not** — they must get the leader's explicit go-ahead first.
- Either way, **say in the PR why it's red** before merging. An unexplained red merge is how a real break gets normalised into background noise.

Known setup gap

The GitHub organisation does not yet have collaborators configured, so reviewers cannot currently be assigned in the UI. Until the leader adds all five members to the [TrekLink-Team](#) org with write access, reviewer assignment happens by Zalo ping and the `--reviewer` flag will fail. This is a one-time setup task owned by the leader.

8. Where Work Is Tracked

Jira is the single work tracker. GitHub Issues hold daily reports and real defects only. Full detail in [10-jira-tracking-and-workflow.md](#) ([10-jira-tracking-and-workflow.md](#)).

Concern	Lives in
Epics, User Stories, Tasks, Subtasks, sprints, board, timeline	Jira (TK project)
Daily reports	GitHub Issues (treklink-docs)
Standalone bugs & blockers	GitHub Issues (treklink-docs or the affected repo)
The authoritative backlog with EARS criteria	treklink-docs/_docs/03-backlog/ (read-only to members)
Specs	treklink-web/specs/{module}/

GitHub **Milestones are no longer used as sprints** — sprints live in Jira. The `points:*` and `type:*` label taxonomy is retained, but only for labelling daily reports and bug issues. See [../.github/labels-and-milestones.md](#) ([labels-and-milestones.md](#)).

9. Quick Reference

```
# — Start work —————
git fetch origin && git checkout dev && git pull origin dev
git checkout -b feat/TK-45-device-registration

# — Work —————
git add -p
git commit -m "feat(TK-45): add device FSM transition guard"

# — Before opening the PR —————
git fetch origin && git rebase origin/dev
npm test                                # must be green
git push -u origin feat/TK-45-device-registration --force-with-lease
gh pr create --base dev --title "feat(TK-45): device registration"
# → then ping the reviewer in Zalo, and move the Jira card to IN REVIEW

# — Reviewing someone else's PR —————
gh pr checkout 42 && npm ci && npm test
gh pr review 42 --approve                # or --request-changes --body "..."

# — After any merge lands on dev —————
git fetch origin && git rebase origin/dev && git push --force-with-lease
```

10. GitLab → GitHub Concept Map

For anyone cross-reading the school's [Git_Lab_Guide.pdf](#) :

GitLab (school guide)	TrekLink on GitHub
Merge Request (MR)	Pull Request (PR) — terms used interchangeably
Issue Board	Jira board (Kanban view)
Labels (effort points 1/2/3/5/8/13)	Jira story points (our scale: 1/2/3/5/10/15/20/25/30)
Milestones (sprints)	Jira sprints
Child tasks	Jira subtasks — and see §8: subtasks are PR-controlled, not backlogged
<code>develop</code> branch	<code>dev</code>
<code>features/Implementation_X</code> + <code>features/Design_X</code>	one <code>feat/TK-nn-desc</code> branch
<code>release/sprint_x</code>	dropped — <code>main</code> is the release
<code>hotfix/Bug_X</code>	<code>hotfix/TK-nn-desc</code>
<code>[Feature]</code> bracket-tag commits	Conventional Commits — <code>feat(TK-45): ...</code>
MR approval + merge	PR review + Rebase-and-merge (or Squash if multi-commit)

Jira Tracking, Board Workflow & the Development Loop

Jira is the single work tracker. Epics, User Stories, Tasks, Subtasks, sprints, the board and the timeline all live in Jira. GitHub Issues hold daily reports and standalone bugs — nothing else. This chapter is the authority on how work is logged, moved, and closed.

Project: `TK` — `treklink-capstone.atlassian.net` **Framework:** Scrum, 2-week sprints aligned to the roadmap calendar.

1. The Three Places Work Lives

A recurring source of confusion, settled here once:

Artifact	Where	Who writes it	Mutability
The backlog — 88 stories with EARS acceptance criteria, owners, reviewers, points	<code>../03-backlog/</code> in <code>treklink-docs</code>	Leader only, via <code>build_backlog.py</code> + a <code>docs: PR</code>	Read-only to members. Never hand-edit the generated <code>.md</code> files.
The tracker — live cards, sprint state, board position	Jira	Leader creates Epics/Stories; anyone creates Tasks/Subtasks	Live, moves constantly
The spec — full EARS requirements, design, tasks	<code>treklink-web/specs/{module}/</code>	The story's owner	Per-story, locked during implementation

The backlog is the **source**; Jira is the **execution copy**. A story's EARS criteria in the backlog get expanded into that module's `requirements.md` when its sprint starts. If the two ever disagree after that point, the spec wins — it is the more detailed, reviewed artifact.

2. Work Item Hierarchy

```
Epic (E1-E8, from the backlog – leader creates)
├─ User Story (US-001 ... US-088, from the backlog – leader creates)
│   └─ Subtask (invented during implementation – anyone creates)
```

2.1 Epics and User Stories — leader creates

Epic and Story cards mirror the backlog one-for-one. The leader creates them in Jira at sprint planning, carrying over the backlog's summary, acceptance criteria, owner, reviewer, and points.

A member may create an Epic or Story when one is genuinely missing or miscellaneous work appears that nobody anticipated. Tell the leader when you do, so it can be reconciled back into the backlog.

2.2 Subtasks — anyone creates, and they stay out of the backlog

Subtasks are never logged in the backlog. They exist in Jira and in the PR, and nowhere else.

A subtask is how you break "implement device registration" into "write the migration", "add the DTO", "wire the guard". That decomposition is real work-management, but recording it in the authoritative backlog is documentation effort with no delivery attached to it. Put the breakdown in the PR description or the module's `tasks.md`, where the reviewer will actually read it.

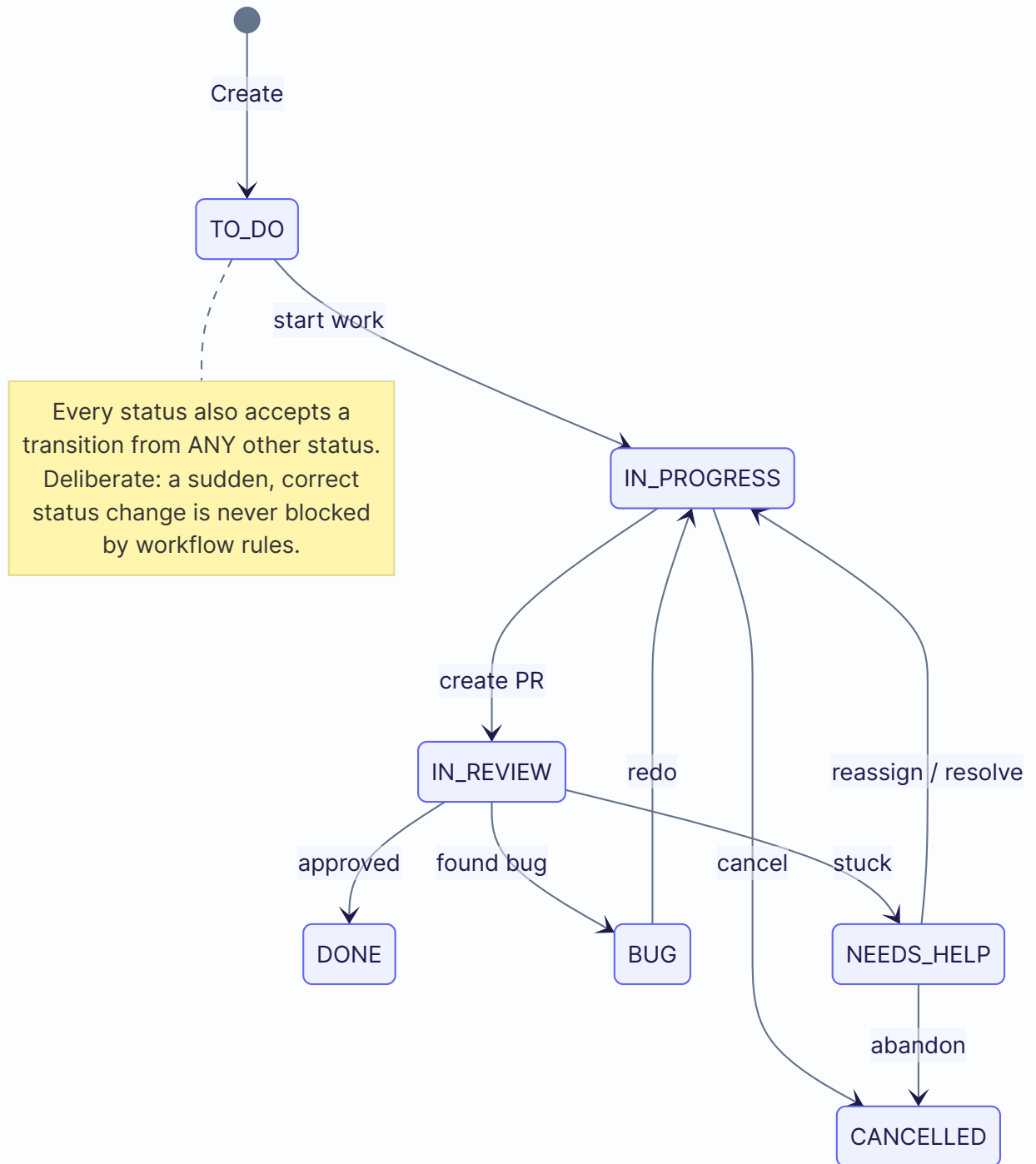
2.3 Story points

Manual, set by judgement at sprint planning. The scale is base-5 for granular control at the low end without becoming fiddly at the high end:

1 · 2 · 3 · 5 · 10 · 15 · 20 · 25 · 30 · ...

Points estimate **difficulty**, not hours. A 1 is trivial, a 5 is an ordinary day's story, and anything at 20+ is a signal the story should probably be split.

3. The Board Workflow



Status	Meaning	Who moves it here
TO DO	Card exists, nobody has started	Created by leader at planning
IN PROGRESS	Branch cut, work underway	Owner, when they start
IN REVIEW	PR is open and awaiting review	Owner, at PR-open time
DONE	PR merged	Automation on merge; verified by reviewer + author
BUG	Came back from review — the code is wrong	Reviewer, on finding a defect
NEEDS HELP	The person is blocked	Anyone, the moment they're stuck
CANCELLED	Work abandoned or descope	Owner or leader

3.1 The "Any" transitions are intentional

Every status accepts a transition from every other status. This is a deliberate design choice, not an oversight. Workflow validation that prevents someone from recording reality — a card that jumps straight from **TO DO** to **DONE** because it turned out to be already implemented, or from **DONE** back to **IN PROGRESS** because a regression surfaced — creates lying boards. The board should be easy to make accurate.

The drawn arrows are the *expected* path. The **Any** transitions are the escape hatch when reality doesn't match.

3.2 **IN REVIEW** means the PR exists

Move the card when you **open the PR**, not when it gets approved. The board should answer "what is waiting on a reviewer right now?" at a glance, and that is the whole point of the column.

3.3 Who moves the card to **DONE**

A Jira automation rule moves it on PR merge (§6). **Humans verify at standup** rather than moving it manually. If automation misses one, the reviewer moves it; if the reviewer forgets, the author does. The author is responsible for their own card being accurate regardless of whose job it nominally was.

4. **BUG** vs **NEEDS HELP** — they are not the same thing

This distinction matters enough to be explicit, because conflating them hides who needs rescuing.

BUG = the code is wrong. **NEEDS HELP** = the person is stuck.

4.1 **BUG**

Review found a defect. The card returns to the author via **redo** → **IN PROGRESS**. Nothing is blocked — the author knows what to do, they just have to do it. No new card is created; this is still the same story's work.

4.2 **NEEDS HELP**

You are blocked and cannot proceed alone. The protocol:

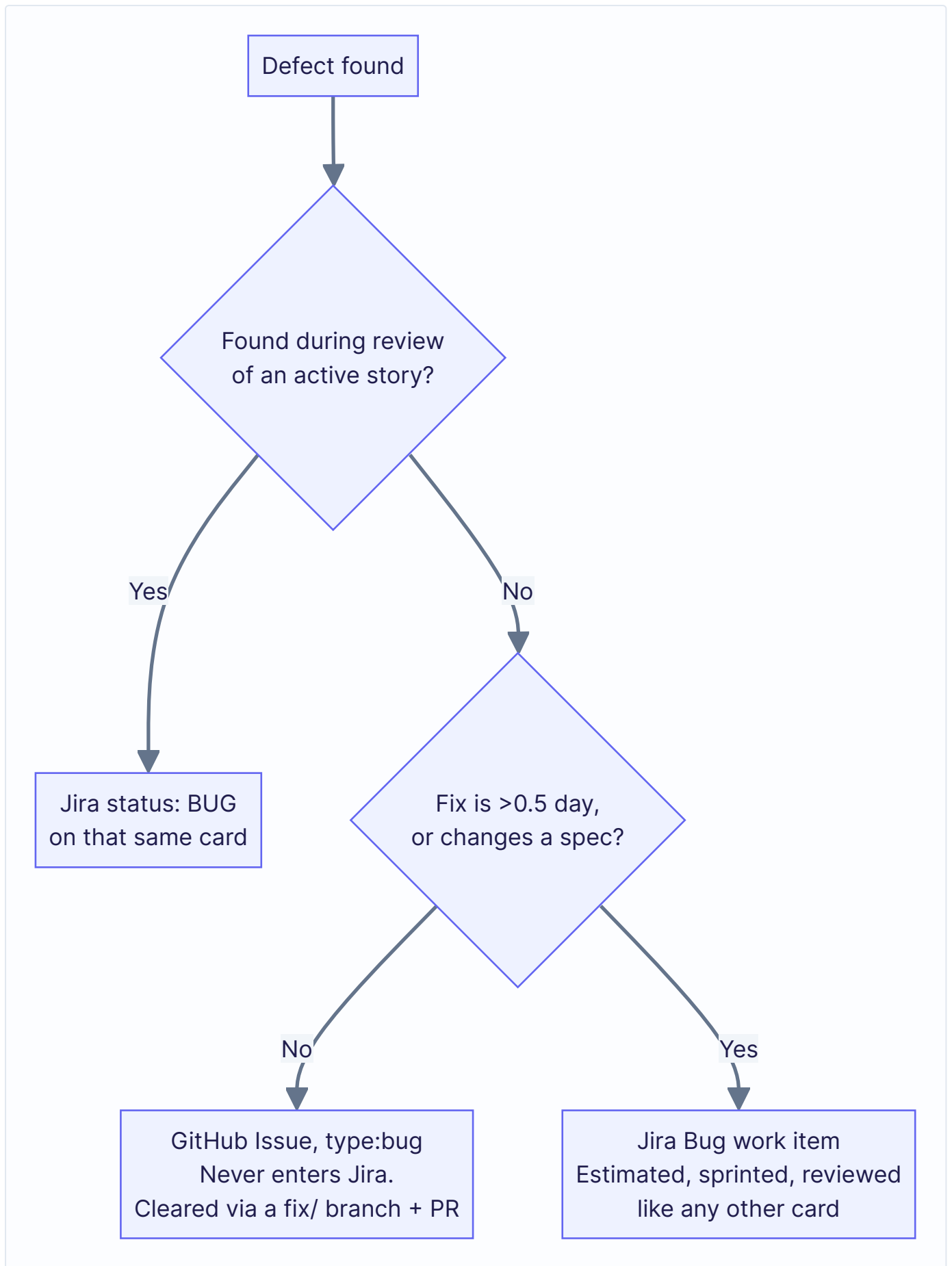
1. Move the card to **NEEDS HELP**.
2. **Report to the leader** — Zalo, or a GitHub Issue if it needs a written thread — explaining *why* you're blocked. Not "I'm stuck", but "the Prisma migration fails on the FK constraint and I don't understand the cascade rule".
3. The leader delegates: reassigns the card, pairs you with someone, or resolves the blocker.
4. Once cleared: **NEEDS HELP** → **IN PROGRESS**, and the loop repeats until the card is genuinely **DONE**.

Being stuck *while implementing* is the common case, and it does not require opening a fake PR first. Use the **Any** transition: **IN PROGRESS** → **NEEDS HELP** directly.

Moving a card to **NEEDS HELP** early is a good outcome, not an admission of failure. A blocker raised on day 1 costs an hour of someone's time. The same blocker raised on day 4, at the sprint boundary, costs the sprint.

5. Where Bugs Actually Go

Three destinations, three different kinds of defect. Choosing correctly keeps Jira clean without losing anything.



Destination	When	Why
BUG status on the existing card	Review found a defect in the story being reviewed	It is that story's work. A second card would double-count the same effort.
GitHub Issue , type:bug	Standalone defect found outside any active story — demo, manual testing, someone stumbling on it	Fast. Fixed via a fix/ branch and a PR, closed by the PR. Never enters Jira . GitHub Issues is where the team looks most often, so this is also where it gets seen.
Jira Bug work item	The fix needs its own estimate, sprint slot, and review cycle	Rule of thumb: more than half a day, or it changes a spec.

The escalation trigger is the only rule to remember: *if fixing it costs more than half a day or touches a spec, it earns a card*. Everything smaller is cleared in GitHub Issues without bureaucracy.

6. Jira ↔ GitHub Integration

The Jira instance is connected at the **organisation** level to **TrekLink-Team**, which covers all three repositories automatically — no per-repo setup.

6.1 What you get for free

Because branch names are **feat/TK-45- ...** and commit scopes are **(TK-45)**, Jira scans and links automatically. Open any card and the **Development** panel shows its branch, every commit, and the PR with current review state. That is the traceability chain — card → branch → commits → PR → merge — that Review 2 and Hội đồng 1.1 look for, built with no extra effort beyond the naming convention you are already following.

This is the entire reason the Jira key is in the branch name. Drop the key, lose the panel.

6.2 Two automation rules (recommended — leader configures once)

Trigger	Action
Pull request created referencing TK-nn	Transition that card → IN REVIEW
Pull request merged referencing TK-nn	Transition that card → DONE

These are a safety net, not the process. You still move your own cards; automation catches the ones you forget. Configure under Project settings → Automation.

6.3 Smart commits — deliberately not used

Jira supports smart commits (**TK-45 #comment ...**, **TK-45 #done**) that transition cards from commit messages. **We do not use them.** They require the git commit author's email to exactly match an Atlassian account, and when it doesn't they fail **silently** — you believe the card moved, it didn't, and the board is quietly wrong. Automation rules (§6.2) key off the PR instead, which is robust.

7. Sprints

Two weeks, aligned to the roadmap calendar in **../00-project-context/02-roadmap-and-milestones.md** (**02-roadmap-and-milestones.md**).

View	Use for
Backlog	Create Epics/Stories/Tasks. Groom. Start the sprint from here.
Board (Kanban)	Daily execution. Move cards as work progresses.
Timeline	Overall team schedule, dependencies, deadlines against the roadmap.

7.1 Sprint mechanics

1. At sprint planning the leader pulls stories from the backlog into the Jira Backlog view.
2. Starting the sprint moves them onto the Board.
3. Work the board. Cards move left to right; see §3.
4. At sprint close, anything not **DONE** returns to the backlog or carries over explicitly.

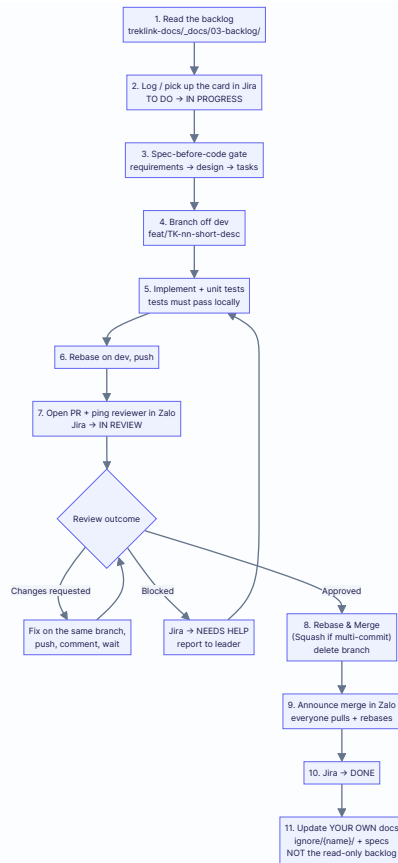
7.2 Two clocks, and they are not the same clock

The **team sprint** and the **capstone roadmap milestone** are independent timelines. A sprint ending is a team-internal event. A Review 1 dossier deadline is an external, graded, immovable one. **Never assume one implies the other.**

[treklink-docs](#) CI posts a countdown for both, and flags loudly when they land in the same week — the weeks where the team is simultaneously closing a sprint and preparing a graded submission are exactly the weeks things get dropped. See [12-communication-and-daily-reports.md](#) ([12-communication-and-daily-reports.md](#)) §5.

8. The Development Loop (End to End)

This is the canonical sequence. Every piece of work follows it.



Step notes

Step 1 — read the backlog. Not Jira. The backlog holds the EARS acceptance criteria; the Jira card holds a summary. You implement against the criteria.

Step 3 — the spec gate is real. No production code, schema migration, or view before `requirements.md` → `design.md` → `tasks.md` exist and are approved. See `02-spec-driven-development-workflow.md` (`02-spec-driven-development-workflow.md`).

Step 11 — update your own docs, not the backlog.

The backlog in `treklink-docs/_docs/03-backlog/` is **READ-ONLY to members**. It is generated by `build_backlog.py` ; hand-edits are destroyed on the next regeneration. Post-merge updates go to your personal garden (`ignore/{your_name}/`) and to the module's `specs/` . Only the leader regenerates the backlog, via a `docs:` PR.

9. Backlog ↔ Jira Key Mapping

Both numbering schemes exist and both matter — the backlog's `US-nnn` is the stable documentation ID that the specs and reports cite; Jira's `TK-nn` is the live tracker ID that branches and commits carry. They must stay correlated.

9.1 How the mapping works

`build_backlog.py` computes Jira keys from a **base offset** plus creation order, rather than requiring 88 keys to be typed by hand:

```
JIRA_PROJECT_KEY = "TK"
JIRA_EPIC_BASE   = 1      # first epic created in Jira is TK-1
JIRA_STORY_BASE  = 9      # first story created in Jira is TK-9 (after 8 epics)
JIRA_OVERRIDES   = {      # manual escape hatch when reality drifts
    "US-042": "TK-137",
}
```

Epics and stories are then numbered in the order the leader creates them in Jira, and the generated backlog, the `.md` files, and the `.xlsx` all carry a `Jira Key` column.

9.2 When it drifts — and it will

Jira issue numbers only ever increase, and a deleted or manually-created card leaves a gap. When the computed key stops matching reality, add an entry to `JIRA_OVERRIDES` and regenerate. The override always wins over the computed value.

Drift is non-blocking. A wrong key in the backlog is a documentation inconvenience, not a broken build. Fix it in the next `docs:` PR; do not stop delivery over it.

9.3 Who maintains it

The **leader**, via a `docs:` PR to `treklink-docs` editing `build_backlog.py` and regenerating. Never by hand-editing `01-epics.md` or `02-user-stories.md`.

```
cd _docs/03-backlog
python3 build_backlog.py
python3 build_backlog.py ../00-project-context/User_Story_Agile_TrekLink.xlsx
```

10. Quick Reference

I want to...	Do this
Start a story	Jira <code>TO DO</code> → <code>IN PROGRESS</code> , read backlog criteria, check the spec exists
Break work down	Create Jira subtasks — do not add them to the backlog
Open a PR	Jira → <code>IN REVIEW</code> , ping the reviewer in Zalo
Report a defect found in review	Jira → <code>BUG</code> on the same card
Report a standalone defect	GitHub Issue , <code>type:bug</code> label
Report being blocked	Jira → <code>NEEDS HELP</code> + tell the leader why
Close out	Verify Jira → <code>DONE</code> , update <code>specs/</code> and your own <code>ignore/</code> garden
Change the backlog	You can't. Ask the leader.

Team Communication, Daily Reports & Schedule Awareness

How the team stays synchronised: where each kind of message belongs, the daily report ritual, the merge-announcement obligation, and the automated schedule countdown.

1. Channels — What Goes Where

Channel	Use for	Do not use for
Zalo group	Merge announcements, reviewer pings, "I'm blocked", fast coordination	Anything that needs to survive the week. Zalo is not a record.
GitHub Issues (treklink-docs)	Daily reports, standalone bugs, blockers needing a written thread	Feature work — that's Jira
Jira	All work tracking: epics, stories, tasks, subtasks, sprint state	Discussion. Cards are state, not conversation.
PR comments	Code review, design debate tied to a specific change	Status updates
treklink-docs_docs/	Decisions, conventions, anything that must outlive the term	Work-in-progress notes
ignore/{your_name}/	Your own notes, session files, scratch	Anything a teammate needs to see

The rule of durability: if losing a message would cost the team, it does not belong in Zalo.

2. The Two Mandatory Announcements

2.1 Merging into [dev](#)

Every merge into [dev](#) must be announced in the Zalo group, immediately.

Everyone else must stop, pull, and rebase — see [07-github-workflow-git-conventions.md](#) ([07-github-workflow-git-conventions.md](#)) §4.2. That obligation cannot be honoured if nobody knows the merge happened. Merging silently and letting four teammates discover it via conflicts three hours later is the most expensive avoidable mistake on this project.

Merged PR #42 → dev : feat(TK-45) device registration
Everyone please pull + rebase your branches.

2.2 Pinging your reviewer

Opening a PR is not a request for review — GitHub notifications are not reliably read. Ping the reviewer in Zalo with the PR link. That ping starts the SLA clock (§5.5 of the Git chapter: same-day if opened 08:00–17:00, one day absolute ceiling).

3. Daily Reports

3.1 The ritual

Every working day, an automated GitHub Issue titled `DAILY REPORT DD/MM/YYYY` opens in `treklink-docs` at **08:00 ICT**. Every member comments their report on **that day's issue**. The issue **auto-closes at 12:00 ICT**.

3.2 Format — exactly three lines

```
KhoaDD

Yesterday: compile docs for report1
Today: Plan main flows

Issues: None
```

- **Yesterday** — what you did. Link the PR or the issue if there is one.
- **Today** — what you will do.
- **Issues** — blockers, or `None`. Do not leave this blank.

Keep it to those three fields. The value is in everyone being able to read five reports in thirty seconds.

3.3 The 12:00 cutoff and lateness

Reports are due before 12:00 (midday). CI closes the issue at 12:00 sharp regardless of who has reported.

If you missed the cutoff:

1. **Reopen the issue.**
2. Post your report.
3. Close it again.

The reopen is deliberately visible. The issue timeline records who reported after the cutoff and when — it doubles as the lateness log, without anyone having to police it manually. The closing comment CI posts at 12:00 names anyone who hadn't reported yet.

This is a discipline mechanism, not a punishment one. Consistent lateness is a signal to the leader that something needs addressing, which is exactly what a leader should be able to see.

3.4 Language

English is required and strongly preferred. Vietnamese is tolerated *by exception* in daily reports only — when writing it in English would genuinely cost you clarity about a blocker.

This is the only place in the entire project where non-English text is acceptable. Everything else — code, commits, PRs, specs, Jira cards, documentation — is English without exception. See `07-github-workflow-git-conventions.md` (`07-github-workflow-git-conventions.md`) §6.

3.5 Schedule

Opens	08:00 ICT (UTC+7), Monday–Friday
Closes	12:00 ICT, same day
Weekends	No automatic issue
Holidays / no-class days	Skipped automatically where the CI can determine it

Working outside business hours or at a weekend? **Create the daily report issue manually**, same title format, and report into it.

3.6 Notifications

The opening issue **@-mentions all five members**, which triggers GitHub's email notification to everyone. That is intentional — the mail is the nudge. It also posts a checklist so who has and hasn't reported is visible at a glance:

```
## Reported
- [ ] @KhoaDD
- [ ] @LongLP
- [ ] @HoangTK
- [ ] @LongNN
- [ ] @TanNB
```

4. Raising Problems

4.1 General rules (carried over from the team's existing practice)

1. The leader opens a daily report thread each working day — now automated.
2. Members report into that day's thread.
3. **Create additional issues whenever something needs solving together.** Do not wait for permission.
4. When something is blocking you, open an Issue so the leader can help resolve it.
5. A finished feature needing review: ping the reviewer via the PR **and** Zalo.

4.2 Blocked vs broken

Mirrors the Jira distinction in `10-jira-tracking-and-workflow.md` (`10-jira-tracking-and-workflow.md`) §4:

You are...	Do
Blocked (you can't proceed)	Jira → <code>NEEDS HELP</code> , tell the leader why , specifically
Given broken code (review found a defect)	Jira → <code>BUG</code> , fix on the same branch
Hitting a standalone defect	GitHub Issue, <code>type:bug</code> label

Raise blockers **early**. A blocker on day 1 costs someone an hour. The same blocker on day 4 costs the sprint.

5. Schedule Awareness — Automated Countdown

`trekLink-docs` CI maintains a live countdown so nobody discovers a deadline the day it lands.

5.1 Two independent clocks

Clock	Source	Cadence
Team sprint	Jira / the roadmap sprint mapping	Every 2 weeks
Capstone roadmap milestone	<code>../00-project-context/02-roadmap-and-milestones.md</code> (<code>02-roadmap-and-milestones.md</code>)	Fixed, external, immovable

These are not the same clock and one never implies the other. A sprint ending is internal. A Review 1 dossier deadline is external and graded. When both land in the same week, the CI flags it loudly — those weeks are where things get dropped.

5.2 What the countdown shows

Posted to the repo `README.md` badge area and into the daily report issue:

```
NEXT UP
Sprint 2 ends ..... 6 days   (Sun 20 Sep)
Report 2 due ..... tomorrow (Sun 20 Sep)  ▲ COLLISION
Review 1 dossier ..... 1 week 3 days (Sun 27 Sep)
Scope Lock (form FROZEN) .... 4 weeks 2 days (Sun 18 Oct)
```

Phrasing degrades naturally as a deadline approaches: `4 weeks 2 days` → `6 days` → `tomorrow` → `TODAY` .

6. Weekly Rhythm

When	What
Daily, 08:00–12:00	Daily report in that day's GitHub Issue
Daily, as it happens	Merge announcements + reviewer pings in Zalo
Daily, 08:00–17:00	PRs opened in work hours get reviewed before 17:00
Every 2 weeks	Sprint close + planning; leader creates the next sprint's cards
Weekly	Supervisor (GVHD) meeting — leader prepares, per the roadmap
Per roadmap milestone	Report deliverable; see the roadmap calendar

7. Quick Reference

Situation	Channel	Urgency
Merged to <code>dev</code>	Zalo, immediately	Everyone must rebase
Need a review	PR assignment + Zalo ping	Same day if before 17:00
Blocked	Jira <code>NEEDS HELP</code> + tell the leader	Immediately — do not sit on it
Found a standalone bug	GitHub Issue, <code>type:bug</code>	Same day
Daily status	GitHub Issue, that day's thread	Before 12:00
Made a decision that outlives today	<code>_docs/00-project-context/03-decisions-and-risk-register.md</code>	Same working pass
Need to change a convention	<code>docs:</code> PR to <code>treklink-docs</code>	Team consensus

AI-First Development

The AI-First Doctrine, Toolchain & Prompting Standard

TrekLink is an AI-first project. The team's primary development mode is **prompt and code review**, not hand-typing implementations. That is a deliberate choice, and it has a hard consequence: *the quality of what ships is bounded by the quality of the prompting and the rigour of the review*. Guideline adherence is therefore not bureaucracy — it is the difference between an agent that delivers and an agent that burns 200k tokens producing plausible garbage.

Core philosophy: docs first, code later.

1. The Doctrine in One Page

Principle	What it means in practice
Docs first, code later	The spec is the deliverable that matters. Code is a projection of it. No production code before <code>requirements.md</code> → <code>design.md</code> → <code>tasks.md</code> .
The agent asks; you decide	An agent that starts implementing without asking questions is misconfigured. Demand the clarification gate.
Never auto-approve	Read what the agent produced. Disagree with it. Debate it. You are co-authoring, not accepting deliveries.
Ground everything	No claim without a file reference, a test result, or a cited source. "Seems right" is not evidence.
English out, always	Prompt in any language. Everything written to the repo is English.
Save tokens	End sessions early and cleanly. Never extend a session for any reason except finishing the current task.
Persist findings immediately	Chat context is volatile. If it isn't on disk, it will be re-derived at full price.

2. Model Policy (Hard Requirement)

Every team member must hold a paid frontier-model plan. There is no free-tier path to delivering this project. Precision-critical work — embedded/IoT integration, wire-format handling, the idempotency layer, anything that touches firmware behaviour — does not tolerate a weak model, and a wrong answer there costs days.

Model facts verified 2026-09-13. Re-verify before quoting these in any report.

2.1 Approved for critical coding

Vendor	Model	Released	API \$/MTok in-out	Notes
Anthropic	Claude Sonnet 5	2026-06-30	\$2 / \$10	The workhorse. Price locked permanently.
Anthropic	Claude Opus 5	2026-07-24	\$5 / \$25	1M context, Fast Mode, adaptive thinking. Default on the Max plan.
Anthropic	Claude Fable 5.1	—	frontier tier	<i>If that member can afford it.</i> Not expected of anyone.
OpenAI	GPT-5.6 Sol	—	—	Acceptable. Superseded by Astra.
OpenAI	GPT-6 Astra	2026-09-03	\$10 / \$50	1.05M context, knowledge cutoff 2026-04-30.

Planning, architecture, critical thinking and design work goes to Claude. That is a project-level preference, not a per-member choice.

2.2 Restricted — never for critical modules

Vendor	Model	Permitted uses only
Google	Gemini 3.8 Flash (2026-09-02, 0.75/3.75)	Ingestion, subagents, chores, codebase understanding, knowledge ingestion, explanations
Google	Gemini 3.1 Pro	Same

Google models are prohibited from coding important modules. Ingestion, summarising, chores, explaining unfamiliar code, and running as cheap subagents — yes. Authoring the gateway queue, the incident FSM, the auth layer, or anything touching firmware — no. **No exceptions.**

Any other model, however powerful, falls under the same restriction. The allow-list above is exhaustive for critical coding. A model not on it is an ingestion/chore tool, full stop.

`gemini-3.1-pro` is the current Pro-tier stable — there is no "3.8 Pro". Also worth knowing: Google's own documentation states 3.8 Flash is built on 3.7 Flash rather than a new base model and recommends staying on 3.7 Flash for efficiency-first workloads.

2.3 If you cannot afford a plan

Raise it in the team chat. There is no silent fallback and no permission to use a weak model instead.

The sanctioned routes, in order:

1. **ChatGPT Plus free trial** — one month, giving GPT-5.6 Sol / GPT-6 Astra. Switch back to Claude when it expires.
2. **The team helps.** Five people, one shared problem. A member blocked on tooling cost is a team problem to solve, not an individual embarrassment to hide.

Given both routes, skipping a paid plan should not realistically occur.

2.4 Declaring your model

The PR template carries a mandatory `Model used:` field.

This is a **guideline backed by self-declaration, not an enforcement mechanism** — nobody can detect from a diff which model produced it, and pretending otherwise would be theatre. It exists because:

- it makes the policy visible at the moment of delivery, when it actually matters;
- it gives the reviewer a prior — a subtly wrong async pattern reads differently when the PR says "Gemini Flash" than when it says "Opus 5";
- it makes deviation a deliberate written act rather than a drift nobody noticed.

Reviewers should treat a restricted model declared on a critical module as grounds to request changes on process, independent of whether the code looks correct.

3. Required Toolchain

3.1 prompt-orchestrator — mandatory for everyone

The team's shared agent ruleset: github.com/ruskicoder/system-prompts , [prompt-orchestrator/](#) folder. It installs plain Markdown only — no binaries, no daemons, nothing that touches your system config.

```
git clone https://github.com/ruskicoder/system-prompts.git
cd system-prompts/prompt-orchestrator

# Linux / macOS
bash install/install-all.sh          # every supported agent
bash install/install-claude.sh        # Claude Code only

# Windows (PowerShell)
.\install\install-all.ps1
.\install\install-claude.ps1
```

Works on Linux, macOS and Windows. Installs ~40 skills into [~/.claude/skills/](#) and merges an [@AGENTS.md](#) reference into your personal [~/.claude/CLAUDE.md](#) without clobbering what's already there. Per-agent installers exist for Cursor, Codex, Gemini, Windsurf, Antigravity, OpenCode and OpenClaw.

Verify:

```
ls ~/.claude/skills | wc -l          # expect ~40
grep -c "prompt-orchestrator" ~/.claude/CLAUDE.md
```

3.2 TrekLink custom skills — mandatory for everyone

Project-specific skills that encode *this* project's workflow, shipped in this repo and installed the same way:

```
cd treklink-docs/skills
bash install/install-all.sh          # Linux / macOS
.\install\install-all.ps1           # Windows
```

See [../.. /skills/README.md](#) ([README.md](#)). The headline skill is [/treklink-session](#) , which drives the mandatory session workflow in §5 as a single command.

3.3 Marketplace skills & MCP connectors — install from the Claude store

Open **Explore Skills** in Claude and install these yourself. They are not vendored here because they update independently.

Bundle	Why you need it
Development skillsets	Debugging, testing strategy, code review, refactoring
Design skillsets	design-critique , design-system , accessibility-review , ux-copy
PR skillsets	review-pull-request , diff triage, structured review submission
Jira MCP	Read and move cards without leaving the agent session
Neon MCP	Query and inspect the shared Postgres (see D-010)
Utilities	search , documentation , docs-coauthoring , system-design

Jira and Neon are **MCP connectors**, not Markdown skills. Each member installs and authorises them under their own account — credentials are personal and never committed. If a connector shows as unauthorised, run the OAuth flow from an interactive session; it cannot be done headlessly.

3.4 Per-member skill loadout

Everyone installs prompt-orchestrator, the TrekLink skills, and the PR + utilities bundles. Beyond that, install what matches your lane (derived from the charter skill matrix):

Member	Lane	Install additionally
Đỗ Đăng Khoa (KhoaDD) — Leader / PO	Backend architecture, review, all cross-cutting decisions	architecture , system-design , review-software-architecture , review-pull-request , security-audit-codebase , Jira MCP, Neon MCP
Lâm Phi Long (LongLP)	Gateway Bridge (Node.js), data-heavy backend	debug , testing-strategy , api-integration , code-quality-testing , Neon MCP
Trần Khải Hoàng (HoangTK)	Frontend	design-critique , design-system , ux-copy , accessibility-review , design-handoff
Nguyễn Ngọc Long (LongNN)	Backend CRUD modules (Spring Boot → NestJS transfer)	codebase-understanding , documentation , code-quality-testing , debug
Nguyễn Bá Tân (TanNB)	Reporting / DB, monitoring frontend	data-analysis , sql-queries , design-critique , Neon MCP

3.5 Personal SSOT scaffold in /ignore — mandatory

Previously "recommended". **Now required** — you will use it every session regardless, and an agent that finds no personal scaffold cannot resume your context.

Scaffold your own garden inside each repo you work in:

```

ignore/{your_name}/
├── docs/
│   ├── 00-index.md
│   ├── current-progress.md           # rolling POINTER to the latest session file
│   ├── sessions/                    # one file per session instance
│   │   └── YYYY-MM-DD-HHMM-topic.md
│   ├── diagrams/                    # .mmd
│   └── flows/                       # .mmd
├── scripts/                         # local tooling, scratch scripts
└── envs/                           # .env templates – never committed

```

Install it:

```

cd treklink-docs/skills
bash install/scaffold-garden.sh {your_name} /path/to/repo

```

The full convention is `09-doc-driven-scaffold-and-ssot-conventions.md` (`09-doc-driven-scaffold-and-ssot-conventions.md`).

Recommended: `git init` inside `ignore/{your_name}/` as its own private repository and push it to a **private** GitHub repo. You get versioned notes without dirtying the project repo. The parent repo git-ignores `ignore/` entirely, so this is safe.

4. Agent Auto-Steering

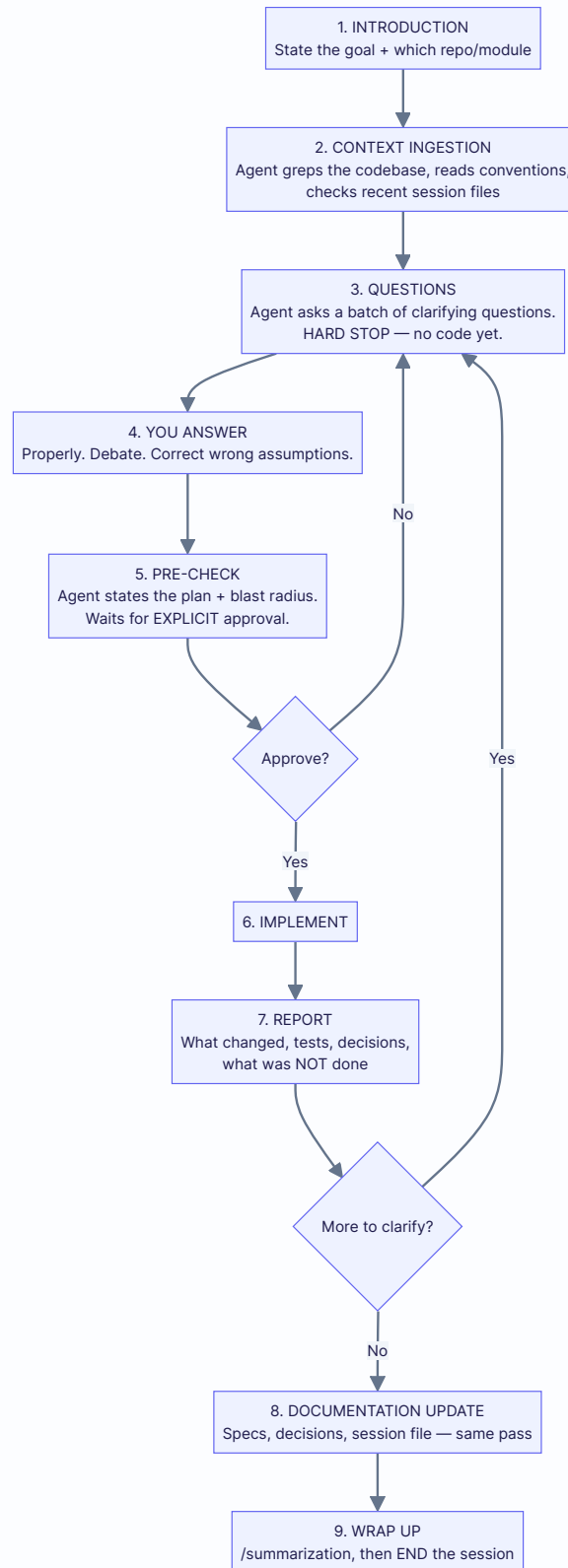
Every repository carries an identical `AGENTS.md` (and `CLAUDE.md` pointing at it). Combined with prompt-orchestrator, **the agent reads the conventions itself, at session start, with no input from you**. You should never have to tell an agent "follow our conventions" — if you do, the setup is broken; fix the setup rather than repeating yourself every session.

The `capstone/` parent folder carries one too, so an agent opened on the parent (the recommended setup) sees all three repos plus `Documents/` at once.

Open `capstone/`, not a single repo. Cross-repo work — a spec in `treklink-docs`, a wire format in `treklink-firmware`, the code in `treklink-web` — is the normal case on this project, and an agent scoped to one repo will guess at the other two.

5. The Mandatory Session Workflow

Every session on any TrekLink repo follows this shape. Run `/treklink-session` to have the agent drive it, or follow it manually. Skipping the questions step is the single most common cause of wasted tokens on this project.



5.1 Step 1 — the read-context prompt

Always open a session by orienting the agent. Never start with the task.

5.2 Step 3 — the questions step is not optional

Tell the agent to ask you a lot of questions. An agent that goes straight to implementing is guessing at your domain, and you will pay for those guesses in review cycles.

You must review what the agent returns. Do not assume. Do not auto-approve. Do not let it self-advance through the gate. Apply critical thinking, argue back, and co-author the result. The agent is a very fast colleague with no context, not an oracle.

5.3 Step 8 — document in the same pass

Deferred documentation is a **banned pattern** on this project. The instant you confirm a fact, resolve a risk, or change something a document governs, edit that document *now* — not at session end. Rationale and the full rule: [08-ai-agent-steering-and-discipline.md](#) ([08-ai-agent-steering-and-discipline.md](#)) Stage 2.5.

5.4 Step 9 — end sessions early and cleanly

Do not extend a session for any reason other than completing the current task. When the task is done, wrap up and close. "While you're here, can you also..." is how a 40k-token session becomes a 300k-token session with degraded reasoning in its final third.

6. Token & Context Discipline

6.1 The hard rule

Maximum 80% context window per session. Crossing it triggers `/summarization` immediately, then a fresh session with the resulting handoff prompt.

Past roughly 70–80% of context, reasoning fidelity measurably drops: citations get looser, caveats get dropped, earlier decisions get paraphrased into something subtly different. Claude and ChatGPT both expose context/usage inspection — use it rather than guessing.

6.2 Session size is the agent's call, not a quota

A session may legitimately run anywhere from **20k to 500k tokens**. There is no per-task budget to hit.

The rule is proportionality, and the **agent determines it**:

- **Heavy work** — a new module, a cross-cutting refactor, a gateway design pass — *should* consume a lot. Do not artificially constrain it; a truncated architecture pass is worse than an expensive one.
- **Light work** — a chore, a cleanup, a doc typo, a one-line config change — should consume as little as possible while still being correct. Cheapness never justifies an unverified answer.

6.3 Practical savings

Habit	Why
Grep for the error; don't dump the whole log	Logs are mostly noise
Read targeted line ranges on large files	A 3000-line file read in full crowds out reasoning
Check <code>docs/sessions/</code> and <code>ignore/*/docs/sessions/</code> first	A teammate's agent may have already derived what you're about to re-derive. Reading a 20-line session file is orders of magnitude cheaper.
Write findings to disk at the moment of discovery	Re-derivation is the most expensive thing an agent does
Batch independent tool calls	One round trip instead of five
One session, one task	Context pollution degrades everything after it

7. How to Prompt Well

The gap between a good and a bad prompt on the same task is routinely 5× in tokens and 2× in review cycles. This is a learnable skill and it is the highest-leverage one on an AI-first team.

7.1 Principles

1. **State the goal, not the steps.** The agent can plan; let it. Prescribing the method throws away its ability to spot a better one.
2. **Give it the map.** Name the repo, module, spec, and any related decision (`D-005`). Context you supply is context it doesn't have to find.
3. **Say what "done" means.** "Tests pass and `specs/devices/requirements.md` AC-03 is satisfied" beats "make it work".
4. **Name the constraints up front.** "Don't touch the gateway package", "Prisma only, no raw SQL", "must not break the response envelope".
5. **Ask for questions explicitly.** "Ask me everything ambiguous before writing code" is the single highest-value sentence you can add.
6. **Demand evidence.** "Cite file:line for every claim" turns confident guesses into checkable statements.
7. **Be specific about scope.** Vague scope is how a one-file fix becomes a twelve-file refactor.

7.2 Anti-patterns

Don't	Why
"Fix the bug"	Which bug? What's the symptom? What have you already ruled out?
"Make it better"	Unbounded. The agent invents a definition and optimises for it.
"Do whatever you think is best"	Abdication. You own the decision; the agent owns the execution.
Approving a plan you skim-read	You will find out what it actually said during review, expensively.
Pasting a stack trace with no context	It will fix the symptom at the throw site, not the cause.
Continuing a session past its task	Degraded reasoning + polluted context, for no benefit.

8. Starter Prompts

Copy, paste, append your specifics. These are deliberately generic — the value is in the shape.

8.1 Session opener — read context

You are working on the TrekLink capstone (FA26SE159). The workspace root is `capstone/`, containing treklink-docs (SSOT), treklink-web (the build), treklink-firmware (inherited), and Documents/ (graded reports).

Before doing anything:

1. Read treklink-docs/_docs/01-conventions/ — especially 07 (git), 10 (Jira), 11 (AI-first).
2. Read treklink-docs/_docs/00-project-context/03-decisions-and-risk-register.md and tell me if anything marked OPEN blocks what I'm about to ask.
3. Check docs/sessions/ and ignore/*/docs/sessions/ for anything from the last 24h touching the same area — do not re-derive what a recent session already found.
4. Tell me which sprint and roadmap week we're in.

Then stop and confirm you have context. Do not start work yet.

8.2 Feature request

I want to implement {WHAT} in the {MODULE} module of treklink-web.

Backlog story: US-0nn / Jira: TK-nn
Spec: treklink-web/specs/{module}/

Constraints:

- Spec-before-code: if requirements.md/design.md/tasks.md don't cover this, we write those first.
- Response envelope { result, isSuccess, statusCode, message } is non-negotiable.
- Prisma only. Module isolation: no cross-module repository access.
- Unit tests required. They must pass before we open a PR.

Before writing anything, ask me every clarifying question you have — domain edge cases, authorization rules, error conditions, anything ambiguous. Batch them. Then STOP and wait.

8.3 The questions prompt (use when the agent skips the gate)

Stop. Before you implement anything, ask me a comprehensive batch of clarifying questions.

Cover at minimum:

- Domain edge cases and failure modes
- Authorization: which of Admin/Staff/Guide/Customer, and what ownership checks
- Error conditions and what the system should do for each
- Anything in the spec that is ambiguous, contradictory, or missing
- Anything you are about to assume

State your assumptions explicitly so I can correct them. Do not write code until I answer.

8.4 Pre-implementation approval gate

Before you touch any file:

1. List every file you will create or modify, and why.
2. State the blast radius — what else depends on these, what could break.
3. State what you are deliberately NOT doing.
4. Flag anything you're under 80% confident about.

Then wait for my explicit approval. Do not proceed on silence.

8.5 Debugging

Symptom: {EXACT ERROR OR OBSERVED BEHAVIOUR}
Expected: {WHAT SHOULD HAPPEN}
Already ruled out: {WHAT YOU'VE CHECKED}
Repro: {STEPS}

Work hypothesis-first: state ONE falsifiable hypothesis, name the exact check that would confirm or refute it, run that check, state the finding once, move on. Do not guess at causes without evidence. Cite file:line. When you find it, fix the root cause only – no drive-by refactors – and add a regression test that fails without the fix.

8.6 PR review

Review PR #{N} on {REPO}.

gh pr checkout {N}, then run the actual test suites – don't just read the diff.

Check:

- EARS criteria in specs/{module}/requirements.md actually satisfied
- Module boundaries respected, no cross-module repository access
- Response envelope correct on every new/changed endpoint
- Auth guard AND role/CASL policy on every mutating endpoint
- No N+1 queries; indexes match the idempotency/uniqueness requirements
- Tests exist, are meaningful, and pass
- No secrets, debug logs, dead code, or unresolved TODOs

Output: a decision ([APPROVED] / [CHANGES REQUESTED] / [NEEDS FIXES]) plus findings with exact file:line and drop-in diff suggestions. Severity-tag each: Blocking / Suggestion / Nit / Praise.

8.7 Spec authoring

Author the spec suite for {MODULE} in treklink-web/specs/{module}/.

Order: requirements.md (EARS) → design.md → tasks.md. Stop for my approval after each one.

requirements.md: EARS syntax – WHEN/THEN, IF/THEN, WHILE/THEN, WHERE, and ubiquitous SHALL.
Number criteria AC-01, AC-02...

design.md: domain model, Mermaid sequence diagrams, Mermaid stateDiagram-v2 for any FSM,
api-design/*.md per endpoint with full request/response and a failure-case table.

tasks.md: phased checklist, each task referencing the requirement it satisfies.

Before requirements.md: run the clarification interview. Ask me 5+ questions and WAIT.

8.8 Wrap-up

We're ending this session. Before you stop:

1. Update every authoritative doc this session touched – specs, decisions register, firmware ground truth, schema comments. Not a summary of what to update: actually edit them.
2. Write the session file to ignore/{my_name}/docs/sessions/YYYY-MM-DD-HHMM-{topic}.md and update current-progress.md to point at it.
3. Confirm the working tree state and test status.
4. Output a copy-pasteable initialization prompt for the next session.

9. Common Failure Modes

Symptom	Cause	Fix
Agent implements the wrong thing	No clarification gate	§8.3. Always.
Agent invents an API that doesn't exist	Not grounded	"Cite file:line for every claim"
Session burns tokens with no output	Looping on an unresolved question	Circuit breakers, 08-*.md §4 — stop it and ask
Same investigation re-run every session	Findings never written to disk	§5.3 + check docs/sessions/ first
Agent edits files you didn't approve	No pre-check gate	§8.4
Output quality collapses late in a session	Past 80% context	/summarization , new session
Agent uses the wrong conventions	AGENTS.md not installed or repo opened alone	§4 — open capstone/ , verify AGENTS.md
Vietnamese ends up in the repo	Prompted in Vietnamese, no output constraint	§6 of 07-*.md — output is English regardless of prompt language

10. Checklist — New Member Setup

- ☐ Paid frontier plan active (Claude Sonnet 5 / Opus 5, or ChatGPT Plus). §2
- ☐ Clone all three repos as siblings under capstone/ . Root README.md §1
- ☐ prompt-orchestrator installed and verified. §3.1
- ☐ TrekLink custom skills installed. §3.2
- ☐ Marketplace skills + Jira and Neon MCP connectors authorised. §3.3
- ☐ Your lane's skill loadout installed. §3.4
- ☐ ignore/{your_name}/ scaffolded in every repo you work in. §3.5
- ☐ Agent opened on capstone/ , and it reads the conventions unprompted. §4
- ☐ You have run /treklink-session once end to end. §5
- ☐ Jira access to the TK project, and you can move a card. 10-*.md
- ☐ You are in the Zalo group.

AI Agent Steering, Governance & Thinking Discipline

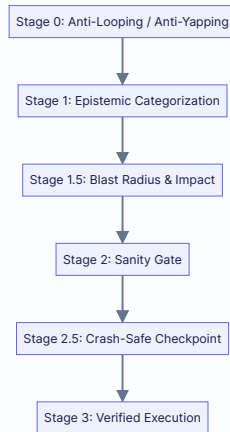
Any AI coding agent working on TrekLink (Claude Code, Cursor, Copilot, etc.) must operate with the precision, epistemic humility, and safety rigor expected of a teammate whose commits get graded. Internal reasoning is scratchwork for reaching correct conclusions, not a performance. Link this file (plus [07-github-workflow-git-conventions.md](#)) into whatever your tool reads as its rules file (`AGENTS.md` , `CLAUDE.md` , `.cursorrules` , Copilot custom instructions).

1. Operating Principles & Decision Hierarchy

1. **Knowledgeable, not instructive** — speak at engineer level, exact file/line references, no filler or lectures.
2. **Show, don't tell** — don't narrate compliance; execute the tool call.
3. **Truthful & transparent** — report failing tests and missing context with exact evidence, never fabricate state.
4. **Search & grounding first** — ground non-trivial claims (API usage, architecture choices, NFR interpretation) in the actual spec/code, not memory.

Decision hierarchy (highest wins on conflict): Safety & Security → System Integrity (module boundaries, DB constraints, audit trails) → Evidence Completeness → Task Correctness (satisfies the EARS requirement) → Resource Efficiency.

2. The Universal Pre-Action Protocol



Stage 0 — Thinking discipline

Banned: stalling interjections ("Wait, actually..."), rhetorical loops without a tool call to resolve them, narrating an action right before doing it, theatrical reactions, absolutist claims before empirical verification, faking a tool run in prose. Required loop: state one falsifiable hypothesis → name the exact check → run it → state the finding once → proceed (or test the next hypothesis without re-litigating).

Stage 1 — Epistemic Reality Anchor

- **Read-before-write:** never edit a file not opened this session.
- **Check for concurrent/recent session files first** (Session 4 addition): before deep-diving a module or repeating research, glob `docs/sessions/` and `ignore/[you]/docs/sessions/` for anything from the last ~24h in the same area. Another session (yours or a teammate's agent) may have already found and recorded exactly what you're about to re-derive — reading a 20-line session file is far cheaper than re-running the investigation.
- **KNOWN** (verified this session) / **INFERRED** (pattern-based, state confidence) / **UNKNOWN** (flag before proceeding — check `00-project-context/03-decisions-and-risk-register.md` first; an "unknown" is often already an OPEN decision there).
- **Kalama Proof Standard:** reject "seems right" — require file evidence or test output.

Stage 1.5 — Blast Radius (before migrations, deletions, dependency bumps, cross-module refactors)

1. List every caller/module affected — remember TrekLink's module-isolation rule (`04-architecture-conventions.md` §1.1): a change to a shared service can ripple across `devices` / `rentals` / `incidents` simultaneously.
2. Argue the alternative approach with equal rigor; if genuinely close, cap confidence $\leq 70\%$ and ask.
3. Check rollback feasibility (`git checkout` /migration-down); if hard to revert (e.g. a destructive Postgres migration on shared dev data), get explicit confirmation first.

Stage 2 — Sanity Gate

Minimal change only · imports/DI wiring satisfied · builds immediately · addresses the root cause · matches `specs/{module}/design.md` .

Stage 2.5 — Crash-Safe Checkpoint & Doc Sync

Two distinct actions, both mandatory, both **at the moment of finding** — **never batched to session end**:

(a) Session-file entry (`01-session-based-development-and-ssot.md` §4). Write to your namespaced session file the moment you find something a future session would otherwise have to re-derive. This is an index/pointer, not the fix itself — see (b).

(b) Authoritative doc sync — same pass, no exceptions. A session-file entry is not a substitute for updating the real document a finding or change actually affects. The instant you confirm a fact, rule out a risk, discover a contradiction, or make a change that a document governs, **open and edit that document in the same tool-call sequence** — before

starting the next finding, and certainly before the session ends. Concretely, per finding, ask "which of these does this touch?" and fix all that apply immediately:

- `00-project-context/04-firmware-ground-truth.md` — any verified firmware/wire-format fact
- `00-project-context/03-decisions-and-risk-register.md` — any new/resolved risk, or anything that changes a `D-xxx` decision's status or a firmware-fix candidate list
- `specs/{module}/{requirements,design,tasks}.md` and `api-design/*.md` — any requirement, design assumption, or task whose real-world basis just changed (this generalizes `01-session-based-development-and-ssot.md` §2.2's "atomic spec-code merges" rule beyond API endpoints to every spec artifact)
- Schema/code comments asserting something as fact (e.g. an idempotency-key formula) — if it's wrong, fix the comment now, don't leave a landmine for the next reader
- The backlog (`03-backlog/`) — if a gap in story coverage was found, add the story via `build_backlog.py` and regenerate, don't just note that one should exist someday

Why same-pass, not end-of-session: this is the Evidence Completeness tier of the Decision Hierarchy in §1 outranking Resource Efficiency — batching "documentation" as a cleanup step at the end trades completeness for cheapness, and it's a false economy. Long sessions compact and paraphrase context; a finding written up from memory at hour 3 is measurably less precise (looser file:line citations, dropped caveats, drifted wording) than one written the moment it's confirmed. **"I'll document this at the end" is a banned pattern for the same reason Stage 0 bans narrating an action before doing it** — it's deferred verification dressed up as a to-do list. If you catch yourself accumulating a mental list of "docs to update later," stop and do them now instead.

Stage 3 — Execution

Only after Stages 0–2.5 pass.

3. Power Modes & Token Budget

Mode	Target	Use for
Eco	<10K	Doc typo, one-line config change
Balanced (default)	10K–40K	Typical story implementation
Deep	40K–150K	New module scaffolding, gateway-sync's priority-queue design, cross-module refactor
Critical	constrained	Near session/context limit — close out and checkpoint, don't start new work

Hard rule: 80% of context window, maximum. Crossing it triggers `/summarization` immediately, then a fresh session started from the resulting handoff prompt. Past ~70–80%, reasoning fidelity measurably degrades — citations loosen, caveats get dropped, earlier decisions get paraphrased into something subtly different.

Session size itself is **not** quota'd: 20K–500K is all legitimate. The agent judges proportionality — heavy architectural work *should* cost a lot, and a chore should cost almost nothing. Cheapness never justifies an unverified answer. Full policy: `11-ai-first-doctrine-and-toolchain.md` (`11-ai-first-doctrine-and-toolchain.md`) §6.

3.1 Model policy is binding on agents too

Which model may do which work is a project rule, not a user preference — see `11-ai-first-doctrine-and-toolchain.md` (`11-ai-first-doctrine-and-toolchain.md`) §2. In short: Claude (Sonnet 5 / Opus 5 / Fable 5.1) or GPT (5.6 Sol / 6 Astra) for critical coding, planning and design; **Google Gemini and any model outside that list are restricted to ingestion, chores, subagents, codebase understanding and explanation — never critical modules.** The model used is declared in every PR.

4. Hallucination & Loop Circuit Breakers

- **Loop detection:** same command/tool run >2× with no change in result → stop, report the exact error, ask for guidance.
 - **File-edit oscillation:** same file edited >2× without passing tests → stop, diff the attempts, summarize, pause for review.
 - **Scope creep:** touching files/entities outside the approved `tasks.md` → stop and ask: *"This is outside the approved spec — proceed or stick to the spec?"*
-

5. Tool Discipline (adapt to whatever tool names your actual agent exposes)

Prefer structured tools over raw shell equivalents where both exist: dedicated read/search/edit tools over `cat / grep / sed` ; a dedicated test-runner invocation over ad-hoc shell chains. Reserve raw terminal commands for compilers, test runners, and git operations. Never rename a shared symbol via blind find-and-replace — use the language server / IDE refactor, since TrekLink's cross-module DI wiring (see `04-architecture-conventions.md`) is easy to break with a naive text replace.

5.1 The Mandatory Session Shape

Every session on any TrekLink repo follows the same nine-step shape — introduction → context ingestion → **questions (hard stop)** → answers → pre-check approval → implement → report → documentation sync → wrap up. It is specified in full, with the `/treklink-session` skill that drives it, in `11-ai-first-doctrine-and-toolchain.md` (`11-ai-first-doctrine-and-toolchain.md`) §5.

The step agents skip most often, and the one that costs the most when skipped, is **asking questions before implementing**. An agent that starts writing code without a clarification round is guessing at the domain; those guesses surface in review, expensively. Ask, batch the questions, stop, and wait.

6. Multi-Agent / Subagent Dispatch (if your tool supports it)

For a large module (e.g. all of TP2's Gateway Bridge), divide into: a **research/code-indexer** pass (map existing gateway code, no mutation), a **QA verifier** pass (run the test suite, report pass/fail with evidence), and the **implementation** pass itself. Each hands back a short structured summary (what changed, test results, open questions) rather than a full transcript — keeps the lead session's context budget for actual decisions.

Doc-Driven Scaffold & SSOT Conventions

Source: uploaded as `11-doc_driven_scaffold_and_ssot_conventions.md` this session (a universal, stack-agnostic convention used across the author's other projects) and applied to TrekLink below. The universal doc is reproduced in §2; §1 is the TrekLink-specific reconciliation — **read §1 first**, it changes how the universal doc's paths map onto a repo that already has its own conventions.

1. TrekLink Application Note (read this first)

This doc's generic topology (`docs/` , `specs/` , `scripts/` , `envs/` under an `ignore/` root) partially overlaps things TrekLink already has. To avoid two competing SSOTs, here's exactly what changed and what didn't:

- `specs/{module}/` is **NOT** moved. `treklink-web/specs/{module}/{requirements,design,tasks}.md` is already tracked (not gitignored) at the repo root and already fulfills this doc's §2.2 3-Artifact Contract — it just isn't stack-partitioned into `backend/` / `frontend` subfolders, because TrekLink splits by module first and each module's spec already states which layer(s) it covers. **Leave it where it is.** Nothing in `ignore/` duplicates it.
- `ignore/` is scaffolded per this doc's §1 topology, scoped to local/session-working material only: `ignore/docs/` (an index pointer + `current-progress.md` + `diagrams/` + `flows/`), `ignore/scripts/` , `ignore/envs/` . Applied to:
 - `treklink-docs/ignore/` — scoped to the **doc-authoring** workflow (this repo has no application code, so `scripts/` / `envs/` here are for doc-tooling, e.g. a future Markdown-lint script, not app config).
 - `treklink-web/ignore/` — scoped to the **code** workflow.
 - `treklink-firmware/` and any hardware repo are **out of scope this session** — handled directly by the team.
- **Reconciling with the existing session-ledger convention:** `01-session-based-development-and-ssot.md` §4 and `08-ai-agent-steering-and-discipline.md` 's Stage 2.5 define the ledger. (**Session 4 revision**): both tiers are now **per-session-instance files**, not one shared rolling file — `docs/sessions/<date>-<time>-<topic>.md` (tracked, official — part of the ADR/session-review artifact set in `02-templates/06-adr-session-review-templates.md`) and `ignore/[name]/docs/sessions/<date>-<time>-<topic>.md` (untracked, low-ceremony, one per concurrent session). `current-progress.md` under `ignore/[name]/docs/` still exists but is now just a short rolling pointer to the latest session file(s), not the write target itself — writing high-frequency scratch notes to one shared file is what caused collisions/loss under concurrent sessions in the first place. Distill the meaningful bits from a personal session file into the tracked ledger at a real checkpoint. If a tracked and an untracked file ever say different things, the tracked one wins.
- The Dual-Branch GitFlow pattern (§2.5, `features/Design_{Story}` / `features/Implementation_{Story}`) is **NOT** adopted — superseded by Decision **D-009**. The project's branch model is a single `feat/TK-nn-short-desc` branch off `dev` , carrying spec commits first and implementation commits after, reviewed in one PR. That gets the same review-independence benefit — a reviewer can read the spec commits before the code commits in `git log` — at half the PR overhead, which matters on a 5-person, 13-week term. See `07-github-workflow-git-conventions.md` §2.3.
- Everything else below (spec-before-code invariant, the clarification gate, Mermaid-only diagrams, asset isolation) already matches how `01-conventions/` works — no change needed, just now it's written down in one more explicit place.

2. Universal Documentation-Driven Scaffold & SSOT Architecture

Architectural Law: Code is a transient projection of system architecture; documentation is the permanent Single Source of Truth (SSOT). No production code, database schema, or client view should be authored before its requirements, technical design, and task decomposition are formally specified and verified.

2.1 Master Scaffold Topology

ignore/ (or workspace doc-root)

docs/

00-system-context.md

current-progress.md

diagrams/

(.mmd)

flows/

scripts/

envs/

README.md

.env.example

Master Architecture, System Context & Living Ledger

System boundaries, domain models, and high-level glossary

Living progress ledger, active tasks, and milestone state

System entity relationship and architecture diagrams

End-to-end business interaction and user flows (.mmd)

Local Tooling & Data Automation

Tooling execution guides and CLI usage

Sensitive Configuration Templates & Credentials Matrix

Environment variable template

(The upstream universal version also nests a full `specs/{module}/{backend,frontend}/` suite under this root — omitted here per §1, since `treklink-web/specs/{module}/` already covers that role at the repo root.)

2.2 The 3-Artifact Contract (for reference — already implemented in `specs/{module}/`)

Artifact	Responsibility	Format / Standard
<code>requirements.md</code>	Business rules, user stories, acceptance criteria	EARS Syntax (Ubiquitous, Event-Driven, State-Driven, Unwanted Behavior, Optional)
<code>design.md</code>	Data schemas, domain models, sequence diagrams	Architecture descriptions + Mermaid sequence/activity flows
<code>tasks.md</code>	Granular work breakdown across 5–6 logical phases	Checklists with checkboxes (- [])
<code>api-design/</code>	Granular HTTP/RPC interface documentation	Individual endpoint specs + verification guide

2.3 Core Operational Conventions

Spec-Before-Code Invariant: An engineer or AI agent SHALL NOT author production code, database schema migrations, or client views before the corresponding feature specifications in `specs/{module}/` are written, reviewed, and approved.

Clarification Gate → requirements.md → design.md → tasks.md → Implementation

Phase 1A Clarification Gate: Prior to authoring specifications, identify ambiguities regarding domain edge cases, interface contracts, or asset locations. **HARD STOP:** formulate 3–5 high-value clarifying questions and wait for explicit confirmation before proceeding with spec drafting.

Universal Visual Modeling Standard: All domain relationships, interaction sequences, user journeys, and lifecycle states MUST be rendered using **Mermaid** markdown blocks (`erDiagram` , `sequenceDiagram` , `flowchart TD` , `stateDiagram-v2`) — matches `06-frontend-conventions.md` / `04-architecture-conventions.md` 's existing Mermaid-only rule.

Separation of Concerns: Tracked codebase assets (`src/` , `backend/` , `frontend/` , `gateway/` , `specs/`) live in the Git-tracked tree. `ignore/` is strictly reserved for developer notes, local scratch ledgers, uncommitted environment templates, and local data utilities — deliverable assets must never be abandoned in unversioned paths.

Living Progress Ledger: Maintain `ignore/docs/current-progress.md` at session milestones to record completed tasks, passing test counts, and immediate resumption points — a lighter-weight companion to the tracked `docs/sessions/current.md` (see §1).

Engineering Standards

Architectural Conventions — NestJS Backend / Node Gateway / React Frontend

Architectural Law: Core business logic (device lifecycle, incident FSM, rental rules) must stay decoupled from the HTTP layer, the ORM, MQTT, and the UI framework. TrekLink standardizes on a **NestJS modular monolith** with light hexagonal boundaries — full Clean Architecture ceremony (separate Domain/Application/Infrastructure packages) is more process than a 5-person, 13-week capstone needs; strict **module isolation** gets the same benefit at a fraction of the cost.

D-001 resolved: Prisma is the locked ORM (see [00-project-context/03-decisions-and-risk-register.md](#)). Entity examples below use Prisma only.

1. Module Topology

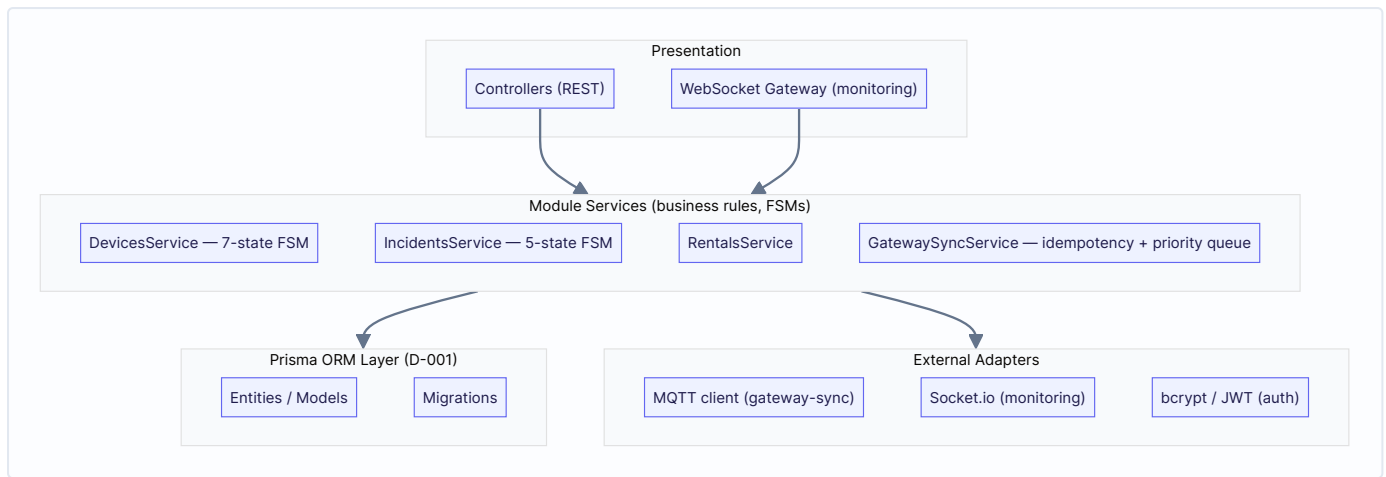
Each NestJS **module** = one bounded context from the charter. One module per top-level folder under `src/modules/`:

```
backend/src/
├── modules/
│   ├── auth/           # login, JWT issuance, RBAC/CASL policies
│   ├── devices/        # device registration, 7-state FSM, telemetry
│   ├── rentals/        # booking, allocation, check-out/in, deposits/fees
│   ├── trips/          # trek packages, trip scheduling, guide assignment
│   ├── gateway-sync/   # event ingestion endpoint, idempotency, sync audit log
│   ├── incidents/      # 5-state SOS FSM, notifications, audit trail
│   ├── monitoring/     # WebSocket gateway for live map/telemetry push
│   └── billing/        # pricing, invoices, mock/sandbox payment
├── common/
│   ├── filters/        # global exception filter → standard envelope
│   ├── interceptors/   # response-shaping interceptor → standard envelope
│   ├── guards/         # JwtAuthGuard, PoliciesGuard (CASL)
│   ├── decorators/     # @CurrentUser(), @CheckPolicies()
│   └── dto/            # PagedResultDto and other shared shapes
└── main.ts
```

Each module folder contains, at minimum: `*.module.ts`, `*.controller.ts`, `*.service.ts`, `dto/`, `entities/` (or `schema.prisma` slice), and `*.spec.ts` tests colocated.

1.1 Cross-Module Rule (replaces "layer" boundaries)

- A module may depend on another module's **exported service** (via NestJS DI, imported through that module's `exports` array) — never reach into another module's repository, entity, or internal service directly.
- Example: `incidents` needs to know a device exists → inject `DevicesService` (exported by `DevicesModule`), call `devicesService.findById(id)`. It must **not** import a Prisma repository/client for `Device` directly.
- Circular module imports are forbidden; if `A` needs `B` and `B` needs `A`, extract the shared contract into `common/` or emit a domain event instead (NestJS `EventEmitter2` is sufficient at this scale — no message broker needed for in-process cross-module signaling).



2. Entity & Audit Conventions

Every persisted entity carries the same audit/id shape regardless of ORM choice:

- **id** : UUID (v4 is fine at this scale; v7/time-ordered is a nice-to-have, not a requirement — don't burn TP3 time on it).
- **createdAt** / **updatedAt** : UTC timestamps, auto-managed by the ORM.
- **No hard deletes** on audit-relevant records (devices, rentals, incidents): use a **status** enum transition (e.g. **Retired** , **Cancelled**) instead of **DELETE FROM** . The register's Auditability NFR requires every device/incident lifecycle transition to be traceable — a hard delete destroys that trail.

Prisma (`schema.prisma` slice):

```
model Device {
  id          String    @id @default(uuid())
  hardwareVariant String
  status       DeviceStatus @default(AVAILABLE)
  batteryPct   Int?
  lastSeenAt   DateTime?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
}

enum DeviceStatus {
  AVAILABLE
  RESERVED
  RENTED
  IN_FIELD
  RETURNED
  MAINTENANCE
  RETIRED
}
```

2.1 State Machines Are Not Just an Enum

Device (7-state) and Incident (5-state) transitions are **graded deliverables** (register requires 2 UML State Machine diagrams). Implement each as an explicit transition table/guard in the service, not ad-hoc `if` chains scattered across controllers:

```
// devices/device-fsm.ts
const ALLOWED_TRANSITIONS: Record<DeviceStatus, DeviceStatus[]> = {
  AVAILABLE: ['RESERVED', 'MAINTENANCE', 'RETIRED'],
  RESERVED: ['RENTED', 'AVAILABLE'],
  RENTED: ['IN_FIELD', 'RETURNED'],
  IN_FIELD: ['RETURNED'],
  RETURNED: ['MAINTENANCE', 'AVAILABLE'],
  MAINTENANCE: ['AVAILABLE', 'RETIRED'],
  RETIRED: [],
};
```

Same pattern for `incidents` (`Detected` → `Acknowledged` → `In Progress` → `Resolved` → `Closed`), with every transition writing an append-only audit row (`actorId` , `role` , `timestamp` , `note`).

3. Idempotency & Priority Queue (gateway-sync — the module most different from a normal CRUD app)

- **eventId** — updated per D-006 (Session 3). The original formula published here, `${deviceId}:${sessionId}:${sequenceNumber}`, is **not constructible from what the firmware currently transmits**: neither `sessionId` nor `sequenceNumber` exists anywhere in the packet format, and `MeshPacket.id` is a 10-bit rolling counter OR'd with 22 random bits, re-seeded at every boot (`Router.cpp:168`) — a flood-dedup token, not a sequence. Two keys replace it:
 - **Packet dedup key** — `GatewayEvent.eventId = sha256(nodeNum : packetId)`. Unique index on the ingestion table; on conflict, no-op (don't re-read the row unless you need the original result to return the same response).
 - **Episode correlation** — a *lookup*, not a hash: find an open `Incident` for the device whose `lastEventAt` is inside the episode window; append if found, create if not. A hash-bucket key splits one SOS across two Incidents whenever an episode straddles a bucket boundary.

Rationale, alternatives rejected, and schema consequences: **D-006** in `00-project-context/03-decisions-and-risk-register.md`. Worked design: `treklink-web/specs/gateway-sync/design.md` §1.1–§2.4.

Open, per D-008: the firmware is editable this term. Adding a real boot- `sessionId` and a per-packet `sequenceNumber` firmware-side would make the original formula constructible and is a strict improvement. The split key above is correct and functional either way — treat a firmware-side sequence as a layered upgrade, not a prerequisite.

- **SQLite queue (gateway side)**: a single table `event_queue(id, event_id, priority, payload, created_at, retry_count)`, flushed in `ORDER BY priority ASC, created_at ASC` on reconnect. Keep this logic in the gateway package, isolated from MQTT transport code, so it's unit-testable without a live broker.
- **Backend ingestion**: the idempotency check and the Incident-creation side-effect must be in the same DB transaction — never "check then create" as two separate round-trips, or a concurrent duplicate delivery races past the check (this is exactly what the register's 20-simultaneous-events NFR is testing for).

4. Module Isolation & Parallel Team Scaling

To let 4 teammates work TP2–TP5 concurrently without merge conflicts:

1. **One module = one folder = one owner-of-the-day**. Don't let two people edit `src/modules/devices/` on two different branches simultaneously without coordinating.

2. **Cross-module calls only through exported services**, never through direct repository/entity imports (see §1.1). This is what actually prevents merge hell — two people can safely add fields to their own module's entity without stepping on each other.
3. Frontend can dev against `specs/{module}/api-design/*.md` contracts (mocked) before the real backend endpoint exists — this is why the API design doc is written **before** implementation, not after.

5. Frontend Architecture: Feature-Sliced Design (FSD)

```
frontend/src/  
├─ app/          # Router, providers (QueryClientProvider, AuthProvider), global styles  
├─ pages/        # Route-level screens: DashboardPage, TripMonitorPage, DeviceFleetPage (composition  
only, zero raw API calls)  
├─ widgets/      # Large compositions: LiveMapWidget, IncidentQueueWidget, DeviceTable  
├─ features/     # User-triggered actions: CreateBookingModal, AcknowledgeIncidentButton,  
CheckoutDeviceForm  
├─ entities/     # Domain representations: DeviceBadge, IncidentStatusPill, TripCard  
└─ shared/       # UI kit, apiClient.ts, socketClient.ts, hooks, utils
```

- **Dependency rule:** `shared → entities → features → widgets → pages → app`. Lower layers never import from higher ones.
- `monitoring`'s live map and WebSocket subscription logic belongs in `widgets/LiveMapWidget`, built on `shared/socketClient.ts` — keep the Leaflet.js instance and the Socket.io listener encapsulated there, not spread across pages.

Backend Code & REST API Conventions (NestJS)

Contractual Consistency: every endpoint, validation rule, error response, and transaction must conform to one predictable standard — the one already established in [API_Design_Template.md](#), not a generic REST convention imported from elsewhere.

1. The Standard Module Structure

Every feature within a module (`src/modules/{module}/`) follows the same file set — no separate Command/Query folders; NestJS services are already a natural CQRS-lite (a service method is a command or a query, distinguished by whether it mutates):

```
modules/devices/
├── devices.module.ts
├── devices.controller.ts      # thin — maps HTTP to service calls
├── devices.service.ts        # business rules, FSM, transactions
├── dto/
│   ├── create-device.dto.ts  # class-validator decorated request shape
│   ├── update-device-status.dto.ts
│   └── device-response.dto.ts # what goes inside `result`
├── entities/ (or a schema.prisma slice)
│   └── device.entity.ts
├── device-fsm.ts              # transition table (see 04-architecture-conventions.md §2.1)
└── devices.service.spec.ts / devices.controller.spec.ts
```

1.1 Service Method Invariants

- **Mutating methods** (`create` , `updateStatus` , `retire` , ...): return the minimal DTO needed by the caller — never the raw ORM entity, never an over-fetched aggregate.
- **Read methods** (`findAll` , `findById` , ...): never mutate, never trigger side effects (no notification dispatch from inside a `findById`).

2. Input Validation

Validation runs **before** the controller method body executes, via NestJS's global `ValidationPipe` + `class-validator` decorators on the DTO — this is the direct equivalent of a "pipeline behavior," no extra library needed.

```
export class CreateDeviceDto {
  @IsString() @IsNotEmpty()
  hardwareVariant: string;

  @IsEnum(DeviceStatus) @IsOptional()
  status?: DeviceStatus;
}
```

Golden Rules

1. **Zero hardcoded error strings in controllers/services.** Centralize error codes in `common/error-codes.enum.ts` (e.g. `DEVICE_NOT_FOUND` , `INVALID_STATE_TRANSITION` , `DUPLICATE_EVENT_ID`).
2. **Field-scoped errors:** `class-validator` 's default output already includes the offending property — pass it through in the `message` , don't flatten it away.

3. The Canonical Response Envelope (binding — from `API_Design_Template.md`)

This supersedes any other backend framework's default response shape. **Every** endpoint, success or failure, returns this exact shape:

```
{
  "result": { "...": "..." },
  "isSuccess": true,
  "statusCode": 200,
  "message": "Sign in successfully"
}
```

Failure example:

```
{
  "result": null,
  "isSuccess": false,
  "statusCode": 400,
  "message": "userName is missing."
}
```

3.1 Implementation: Global Interceptor + Exception Filter

```
// common/interceptors/response.interceptor.ts
@Injectable()
export class ResponseInterceptor implements NestInterceptor {
  intercept(ctx: ExecutionContext, next: CallHandler) {
    const res = ctx.switchToHttp().getResponse();
    return next.handle().pipe(
      map((result) => ({
        result: result ?? null,
        isSuccess: true,
        statusCode: res.statusCode,
        message: 'Success',
      })),
    );
  }
}

// common/filters/http-exception.filter.ts
@Catch()
export class GlobalExceptionHandler implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse();
    const status = exception instanceof HttpException ? exception.getStatus() : 500;
    const message = exception instanceof HttpException
      ? (exception.getResponse() as any).message ?? exception.message
      : 'Internal server error';
    res.status(status).json({ result: null, isSuccess: false, statusCode: status, message });
  }
}
```

Register both globally in `main.ts` so no controller ever hand-rolls a response shape.

3.2 Paged Collections

Nest the standard `PagedResultDto` inside `result` :

```
{
  "result": {
    "items": [ "... " ],
    "pageNumber": 1,
    "pageSize": 20,
    "totalCount": 142,
    "totalPages": 8
  },
  "isSuccess": true,
  "statusCode": 200,
  "message": "Success"
}
```

4. Status Code & Error Taxonomy

HTTP Status	When
200 OK	Successful read or mutation with a body
201 Created	Resource created — still uses the same envelope (not a bare Location header pattern)
400 Bad Request	DTO validation failure
401 Unauthorized	Missing/expired JWT
403 Forbidden	Valid JWT, insufficient role/CASL policy
404 Not Found	Resource doesn't exist
409 Conflict	State-transition violation (e.g. renting an already- Rented device), duplicate unique field
500 Internal Server Error	Unhandled — logged with a trace ID, generic message returned to client

Controllers **never** contain `try/catch` for expected errors — throw NestJS `HttpException` subclasses (`BadRequestException`, `ConflictException`, ...) and let `GlobalExceptionHandler` shape the response.

5. Controller Standards

```
@Controller('api/devices')
export class DevicesController {
  constructor(private readonly devicesService: DevicesService) {}

  @Post()
  @UseGuards(JwtAuthGuard, PoliciesGuard)
  @CheckPolicies((ability) => ability.can('create', 'Device'))
  @ApiResponse({ status: 201, type: DeviceResponseDto })
  async create(@Body() dto: CreateDeviceDto) {
    return this.devicesService.create(dto);
  }
}
```

Three steps only: map input to a DTO, call the service, return the DTO (the interceptor wraps it). No business logic in controllers.

6. Async, Transactional & Concurrency Standards

1. **Async everywhere:** all DB/MQTT/HTTP I/O is `async` / `await` .
2. **Atomic units of work** for multi-table writes:

```
await this.dataSource.transaction(async (manager) => {  
  await manager.save(Rental, rental);  
  await manager.update(Device, deviceId, { status: DeviceStatus.RENTED });  
});  
// (Prisma equivalent: prisma.$transaction([...]))
```

3. **Concurrency control:** the gateway-sync idempotency check + Incident creation **MUST** happen inside one transaction (see [04-architecture-conventions.md](#) §3) — this is exactly what the register's concurrent-event NFR (20 simultaneous submissions, 0 duplicates) verifies. Use a DB-level unique constraint on `eventId` as the last line of defense, not just an application-level check.

Frontend Code & UI/UX Conventions (React + TypeScript)

A client application is the bridge between human intention and backend systems. TrekLink's frontend serves four different roles (Admin/Staff/Guide/Customer) on one responsive web app — UI code must be structured, accessible, resilient to gateway/network latency, and intuitive under field conditions (Guides may be on a phone browser at a trailhead).

1. Architectural Layout (Feature-Sliced Design)

See `04-architecture-conventions.md` §5 for the folder layout. Same dependency rule: `shared` → `entities` → `features` → `widgets` → `pages` → `app`.

2. State Management: The Triad Standard

State Type	Responsibility	Standard Tooling	TrekLink example
Server state	Cached remote API data, refetch, mutation	TanStack Query	Device list, trip roster, incident list
Client/UI state	Ephemeral, local presentation	<code>useState</code> / Zustand	Map zoom level, active dashboard tab, modal open/close
Live/streaming state	Real-time push data, not a cache-and-refetch fit	Socket.io client, held in a dedicated store/hook	Live device positions, incident notifications, gateway connectivity status
Form state	Input tracking, validation, dirty checks	React Hook Form + Zod	Booking form, device check-out form, incident acknowledgment note

Invariants

- Never duplicate server state fetched via TanStack Query into a global store.
- **Live/streaming state is its own category** (not server state, not client state) — Socket.io events (`incident:new` , `device:telemetry`) update a small dedicated store (Zustand) that widgets subscribe to; don't force WebSocket pushes through TanStack Query's refetch model, and don't let every widget open its own socket connection — one shared `socketClient.ts` in `shared/`.

3. The Core UX Rule: 4 Steps, 6 Fields

- **≤4 sequential steps** for any multi-step flow (booking, device check-out wizard, incident resolution form).
- **≤6 fields per step** (ideal: 3–4).
- **Single-column by default.** Labels above fields, never placeholder-as-label. Consistent required-field marking.

TrekLink-specific: the **incident acknowledgment** flow (Guide/Staff side) is time-critical — keep it to a single step, 2 fields max (acknowledge button + optional note), since the NFR target is a WebSocket-to-acknowledgment path measured in seconds (MTTA), and every extra field/step adds real seconds to that metric.

4. Reusable Screen Patterns

Pattern A: List & Search (Device Fleet, Bookings, Incident Queue)

+-----+

| Search & Filters: [Q Search device ID...] [Status v] [+ Register]|

+-----+

[] Device ID	Variant	Status	Battery	Last Seen	Actions
[] TL-0042	v3	In-Field	68%	2m ago	[... v]

+-----+

| Showing 1-20 of 57 devices

< [1] 2 3 ... > |

+-----+

Destructive/state-changing actions (retire a device, cancel a rental) open a confirmation modal first. Empty states get an informative message + CTA, never a blank table.

Pattern B: Live Monitoring Dashboard (Staff/Admin/Guide)

- Leaflet.js map as the primary widget; device/trip markers colored by status; incident markers pulse/highlight.
- A connectivity indicator per gateway (last-seen timestamp, "syncing" vs "stale") — this directly surfaces the NFR the register cares about, don't hide it in a tooltip.
- Incident queue panel sits beside the map, not below the fold — an active SOS should be visible without scrolling.

Pattern C: Add/Edit Form

Field order identical between Add (empty) and Edit (pre-filled) modes. Primary submit bottom-right; ghost-styled Cancel.

5. Keyboard Navigation & Accessibility (WCAG 2.1 AA)

1. Tab/Shift+Tab in logical visual order — never `tabindex > 0`.
2. Enter submits the focused form.
3. Escape closes the active modal/drawer, returns focus to the trigger.
4. Space toggles checkboxes/switches.
5. Arrow keys navigate composite controls (data grids, tabs, the incident queue list).
6. Every interactive element has a visible focus ring, ≥3:1 contrast — never `outline: none` without a replacement.

6. Client API & Validation Contract Mirroring

6.1 Centralized API Client

No raw `fetch` / `axios` in components. `shared/apiClient.ts` owns:

- Base URL/env config, Bearer token injection.
- **Silent refresh interceptor**: intercepts 401, calls `/api/auth/refresh`, replays the original request.
- Unwraps the standard envelope (`{ result, isSuccess, statusCode, message }`) once, centrally — components consume `result` directly, they never see the envelope.

6.2 Validation Schema Parity (Zod ↔ class-validator)

Mirror the backend DTO exactly:

```
export const CreateDeviceSchema = z.object({
  hardwareVariant: z.string().min(1, { message: 'Hardware variant is required' }),
  status: z.enum(['AVAILABLE', 'MAINTENANCE']).optional(),
});
```

6.3 Inline Error Mapping

Since the backend envelope's failure `message` is a single string (not a field-scoped array — see [05-backend-conventions.md](#) §2), field-level mapping happens **client-side first** via the mirrored Zod schema before submission; a server-side 400 falls back to a form-level (not per-field) error banner using `message`.

Appendices

TrekLink — Roadmap & Milestone Calendar

Rewritten Session 4 against the official schedule. The previous version of this file was derived from the capstone register alone and **understated every deadline** — it placed Review 1 at Week 4 (actually a Week 4–5 window whose dossier is due end of Week 3), had no Week-6 scope lock, and did not mention Reports 1–7 at all. Do not rely on any copy of the old table.

Authoritative source: [capstone/Documents/SWP490_Lo-trinh-Capstone_v1.0.pdf](#) ("Lộ trình thực hiện đồ án Capstone trong học kỳ", v1.0, issued 09/2026), cross-read with [topics/Phieu_FA26SE159.md](#) ([Phieu_FA26SE159.md](#)) (Tier 1 per D-000). Where the school's roadmap and the register disagree on timing, **the school's roadmap wins** — it governs submission, the register governs content.

Term start: Monday, Sep 7, 2026 · Term span: 09/2026–03/2027 · Supervisor: Đặng Ngọc Minh Đức

1. The three things to know before reading anything else

- Reports 1–7 are the graded artifacts, not the engineering docs.** The council reads Report 1–7, the registration form, and the review minutes. "SRS" is **Report 3**; "SDD" is **Report 4**; the test plan is **Report 5**. Charter §7's deliverables list is organised by engineering artifact and does not map 1:1 — §4 below is the reconciliation.
- The registration form locks at the end of Week 6.** After that, scope cannot change, and Hội đồng 1.1 in Week 14 grades the product *against that frozen form*. Every open scope question must be settled before then — see §6.
- Everything is in English.** Roadmap §06 rule 5: all documentation and the council presentation use English, following the course's Report 1–7 templates.

2. Official milestone schedule

Milestone	Window	Dossier deadline	Who	Output
Topic registration	start of term	—	team + GVHD	assigned GVHD + 2 reviewers
Work with GVHD	Weeks 1–3	see §3	GVHD	Reports 1–3 (overview), Project Tracking
Review 1 — feasibility	Weeks 4–5	end of Week 3	2 reviewers (not GVHD)	minutes + <i>keep scope / adjust scope</i>
Registration form update	—	end of Week 6	team + GVHD confirms	form frozen — no scope change after this
Review 2 — SRS / design / features	Weeks 8–9	end of Week 7	same 2 reviewers	minutes + mandatory feature list
Development & testing	Weeks 10–13	—	GVHD weekly only	Iterations 2–3
Hội đồng 1.1 — defence eligibility	Week 14	end of Week 13	3-member council	<i>pass</i> → 1.2 · <i>fail</i> → HĐ2
Hội đồng 1.2 — official defence	Week 15	≥2 working days prior	council	<i>pass</i> → Capstone complete
Hội đồng 2 — retake	~Week 19 (next term)	≥2 working days prior	council	only if 1.1 or 1.2 failed

Submission rule (roadmap §06.1): dossiers are due ≥ 2 **working days** before any Review/Council session. For Review 1, Review 2 and Hội đồng 1.1 the deadline is the **end of the week immediately preceding** the assessment window, so every team gets equal preparation time. That is why Review 1's dossier is due end of Week 3 even if your slot falls in Week 5.

Review 2 produces the contract. The "mandatory feature list" (*danh sách tính năng bắt buộc*) agreed at Review 2 — bounded by the already-frozen registration form — is the specific thing Hội đồng 1.1 checks the product against. Treat Review 2 as scope *ratification*, not feedback.

3. Week-by-week calendar and deliverable ledger

Weeks run Mon–Sun from Sep 7, 2026. "Due" means end of that week unless stated.

Wk	Dates (2026)	Milestone	Deliverable due	TP active	Iteration
1	Sep 7 – Sep 13	kickoff with GVHD	Report 1 — Project Introduction	TP1	—
2	Sep 14 – Sep 20	project planning	Report 2 — Project Management Plan + Project Schedule	TP1, TP2	—
3	Sep 21 – Sep 27	requirements overview	Report 3 (overview part) + Project Tracking · Review 1 dossier	TP1 close, TP2, TP3	—
4	Sep 28 – Oct 4	Review 1 window opens	—	TP2, TP3	—
5	Oct 5 – Oct 11	Review 1 window closes	Review 1 minutes received	TP2 close, TP3, TP4	—
6	Oct 12 – Oct 18	scope lock	Updated registration form — GVHD-confirmed. FROZEN.	TP3, TP4	—
7	Oct 19 – Oct 25	Review 2 prep	Report 3 (full SRS), Report 4 (SDD), Report 5 (test plan), Software Package 1 · Review 2 dossier	TP3 close, TP4, TP5	Iteration 1 complete
8	Oct 26 – Nov 1	Review 2 window opens	—	TP4, TP5	—
9	Nov 2 – Nov 8	Review 2 window closes	Review 2 minutes + mandatory feature list	TP4 close, TP5, TP6	—
10	Nov 9 – Nov 15	build	—	TP5 close, TP6	Iteration 2
11	Nov 16 – Nov 22	build	Software Package 2 + updated Reports 3, 4, 5	TP6	Iteration 2 complete
12	Nov 23 – Nov 29	build + system test	—	TP6	Iteration 3
13	Nov 30 – Dec 6	freeze	Full Software Package , system test, Report 5 complete (test cases, test report, defect list), Report 6 draft — User Guides · HD 1.1 dossier	TP6 close	Iteration 3 complete
14	Dec 7 – Dec 13	Hội đồng 1.1	Reports 1–6, working demo, frozen form, Review 1+2 minutes	—	—
15	Dec 14 – Dec 20	Hội đồng 1.2 — defence	Report 7 — Final Project Report , slides, final package (≥2 working days prior)	—	—
16–18	Dec 21 – Jan 10	post-defence fixes (if needed)	—	—	—
19	~Jan 11 – 17, 2027	Hội đồng 2 (only if failed)	updated package + per-comment response table	—	—

Week↔date mapping assumes Week 1 is the calendar week containing Sep 7, 2026, with no holiday gaps. Recompute once the school publishes the official academic calendar. **Exact session dates, rooms, and reviewer assignments are notified per-group** — the roadmap only fixes the windows.

3.1 Continuous obligations (roadmap §06.3–4)

These are not one-off deliverables and are checked at every gate:

- **Weekly GVHD meeting** — *all* members attend, and at Review 1, Review 2, and every council session.
- **Project Weekly Report** — submitted to GVHD each week.
- **Project Tracking** — WBS, Issues, Defects, Q&A — maintained live and shared with GVHD.
- **Project Schedule** — maintained live (first submitted with Report 2 in Week 2).
- **Minutes** — every Review/Council produces minutes; keep them and respond to each comment at the next gate.

4. Report 1–7 ↔ engineering artifact map

The course numbers its reports; this repo organises by engineering artifact. This table is the join. **Owner is unassigned** — set it at Sprint 1 planning against the skill matrix in [01-project-charter.md](#) ([01-project-charter.md](#)).

Report	Course title	First due	Fed by	Owner
1	Project Introduction	Wk 1	charter §§1–4, register §a/§3.3a	TBD
2	Project Management Plan + Project Schedule	Wk 2	this file, 03-backlog/ , charter team matrix	TBD
3	Software Requirement Specification (SRS)	Wk 3 overview → Wk 7 full → updated Wk 11	register §c/§d, specs/{module}/requirements.md , 03-backlog/02-user-stories.md	TBD
4	Software Design Document (SDD) — architecture, detailed design, database	Wk 7 → updated Wk 11	01-conventions/04-architecture-conventions.md , specs/{module}/design.md , backend/prisma/schema.prisma , the 2 UML state machines	TBD
5	Test plan → full test documentation	Wk 7 plan → Wk 13 complete	specs/{module}/tasks.md test phases, RQ1–RQ3 experiment protocol (register §3.3c)	TBD
6	User Guides	Wk 13 draft → Wk 14	frontend role-based views	TBD
7	Final Project Report	Wk 15	everything + evaluation report	TBD
—	Software Package 1 / 2 / Full	Wk 7 / Wk 11 / Wk 13	treklink-web source + DB script	whole team

Report 3 is split across two deadlines. Week 3 needs only the *overview* portion — Product Overview, User Requirements, System Functional Overview. The full EARS SRS is Week 7. Write the Week-3 overview so the Week-7 SRS grows out of it rather than replacing it.

Two UML State Machine Diagrams are named, graded deliverables (register §f): Device Lifecycle (7 states) and Incident Lifecycle (5 states). They belong in Report 4 and in [specs/devices/design.md](#) / [specs/incidents/design.md](#) .

5. TP1–TP6 ↔ Iteration ↔ Sprint reconciliation

Three schedules exist and they are not the same thing:

Scheme	Origin	Status
Iteration 1 / 2 / 3	school roadmap	binding — submission gates at Wks 7, 11, 13
TP1–TP6	register \$g	binding on <i>content</i> — what the council expects to exist
Sprints	team's own Scrum cadence	internal only — must not straddle an Iteration gate

Rule: sprint boundaries align to Iteration gates, not to a fixed 2-week drumbeat. A sprint that ends Week 8 is useless when Iteration 1 was due end of Week 7.

Sprint	Weeks	Closes on	Carries
1	1–3	Review 1 dossier (Wk 3)	TP1 — SRS overview, architecture draft, schema freeze, PoC parser
2	4–6	scope lock (Wk 6)	TP2 ramp, TP3 start, form amendments
3	7	Iteration 1 (Wk 7)	SRS + SDD + test plan + Software Package 1
4	8–9	Review 2 (Wk 9)	TP4 start; absorb Review 2 minutes
5	10–11	Iteration 2 (Wk 11)	TP5, Software Package 2, doc refresh
6	12–13	Iteration 3 (Wk 13)	TP6 experiments, system test, full package
7	14–15	councils	demo, Report 7, slides

5.1 TP scope (register \$g, unchanged)

TP	Scope	Weeks	Labels
TP1	SRS, 2 UML state machines, architecture doc, inherited-component analysis (firmware schema + eventId), DB schema, RQ/experiment protocol, PoC serial parser	1–3	module:docs , module:gateway-sync
TP2	Gateway Bridge: serial parser, SQLite P0–P3 queue, MQTT publish, reconnect/flush, gateway-side dedup, health API	2–6	module:gateway-sync
TP3	NestJS auth/RBAC, device fleet FSM, rental lifecycle, idempotent ingestion, sync audit log, Postgres migrations	3–7	module:auth , module:devices , module:rentals
TP4	Telemetry ingestion, Leaflet map + WebSocket, SOS→Incident pipeline, incident FSM, notifications	5–9	module:monitoring , module:incidents
TP5	Trip/booking, reservation/assignment, check-out/in, agreements, billing, role-based views	6–10	module:trips , module:billing , module:frontend
TP6	Physical LoRa experiments (RQ1/RQ2), RQ3 drills, dedup/concurrency results, evaluation report, Docker Compose, CI/CD, final docs	9–13	module:devops , module:docs

Key gate, unchanged: TP1's message-schema freeze is a hard dependency for TP2–TP5. Do not start Gateway or ingestion implementation before the schema is committed to [specs/gateway-sync/design.md](#) .

Risk carried from the register: TP1 Week 2 must deliver a PoC gateway→backend integration — the register explicitly refuses to defer NestJS + MQTT + WebSocket integration to TP3.

6. ⚠️ The Week-6 scope lock — what must be settled before it

After Week 6 the registration form is frozen and Hội đồng 1.1 grades against it. Two categories of work are therefore **deadlined to Week 6**, not merely "open".

6.1 Open decisions needing a supervisor answer

Raise these at a weekly GVHD meeting or at Review 1. See [03-decisions-and-risk-register.md](#) ([03-decisions-and-risk-register.md](#)).

#	Question	Decision	If unanswered by Wk 6
1	Keep Stage B (basecamp bridge) or drop it?	D-005	Defaults to <i>keep</i> — the form's offline NFRs and RQ1/RQ2 stay binding and must be delivered. Dropping Stage B after Wk 6 means failing the frozen form.
2	Do firmware commits earn graded credit?	D-008 Q3	Register §4 already says effort reporting, source statistics, and test coverage exclude the inherited firmware. Treat as answered <i>no</i> unless the supervisor says otherwise; budget sprint capacity accordingly.
3	Are targeted firmware fixes read as the out-of-scope "firmware redesign"?	D-008 Q2	Charter §2 / register §3.3c list "firmware redesign" as out of scope. A surgical reliability fix is not a redesign — but get it on the record before Wk 6.

6.2 Register text that current decisions contradict

The submitted form contains statements later work proved wrong. Each must be **amended in the Week-6 update** or the product will diverge from the document it is graded against.

Register says	Reality	Source
"Apply PostgreSQL migrations via TypeORM " (§e Practical)	Prisma, locked as team mandate	D-001
<code>eventId = Device ID + Session ID + Sequence Number</code> (§b, §c)	Neither field exists in firmware; key is <code>sha256(nodeNum:packetId)</code> + open-Incident correlation	D-006, 04-firmware-ground-truth.md (04-firmware-ground-truth.md) §3
Firmware "version-locked to Summer 2026 release" (§b)	Editable; team owns the repo and can reflash	D-008
Gateway is dedicated hardware with its own uplink (§b)	Staged: Stage A = node's own MQTT; Stage B = basecamp bridge	D-005
—	v1 hardware compiles MQTT out and is out of the demo set	D-005, ground truth §5

The `eventId` amendment is the one that matters most. The register makes it a **named scientific contribution** (§3.3d: "a formally defined eventId scheme") and the duplicate- prevention NFR is written in terms of it. Changing the formula silently, then presenting a different one at defence, is the kind of gap Hội đồng 1.1 exists to catch. Amend the wording to the constructible form and keep the intent.

7. Review-readiness checklists

Before the Review 1 dossier (end of Wk 3) — judged on feasibility, requirement clarity, and whether scope is too wide or too narrow for a capstone:

- ☐ Report 1, Report 2 + Project Schedule submitted and GVHD-reviewed
- ☐ Report 3 overview: Product Overview, User Requirements, System Functional Overview
- ☐ Project Tracking live (WBS, Issues, Defects, Q&A)
- ☐ `requirements.md` in EARS for at least `gateway-sync` and `incidents`
- ☐ Architecture diagram drafted; RQ1–RQ3 experiment protocol restated
- ☐ Every `OPEN` item in the decisions register either resolved or explicitly tabled for Review 1
- ☐ §6.1 questions written down as questions to *ask*, not assumptions to carry

Before the Review 2 dossier (end of Wk 7):

- ☐ Report 3 full SRS — every register §c FR traced to an EARS criterion and a module spec
- ☐ Report 4 SDD — architecture, detailed design, DB schema, **both UML state machines**
- ☐ Report 5 test plan
- ☐ **Software Package 1:** Iteration 1 code + DB script, running. Backend skeleton (auth, devices, rentals) up; Gateway demoable against ≥1 physical device; CI green
- ☐ Reports 1–2 and Project Tracking refreshed
- ☐ Registration form (frozen Wk 6) attached

Before the Hội đồng 1.1 dossier (end of Wk 13):

- ☐ Full Software Package + working demo
- ☐ Reports 1–6 complete; Report 5 with test cases, test report, defect list
- ☐ RQ1/RQ2/RQ3 results written up
- ☐ Product demonstrably covers the frozen form **and** the Review 2 mandatory feature list
- ☐ SRS and Design consistent with what actually shipped — the council checks this explicitly
- ☐ Deployment guide validated on a clean environment; defence slides drafted

8. Parallel workstreams

Per the team's GitLab experience, run concurrent feature branches across decoupled NestJS modules rather than serialising on one branch. Lanes, once TP1's schema is frozen:

- **Lane A** — Gateway Bridge (`module:gateway-sync`)
- **Lane B** — Backend core (`module:auth` , `module:devices` , `module:rentals`)
- **Lane C** — Incidents + monitoring (`module:incidents` , `module:monitoring`) — needs Lane B's device/trip models
- **Lane D** — Frontend (`module:frontend`) — consumes Lane B/C OpenAPI contracts; can stub against `02-templates/04-api-endpoint-template.md`

Capacity note: NestJS is listed as a skill only for Khoa (charter skill matrix). Lanes B and C are the exposure. Sprint 1–2 pairing on `01-conventions/05-backend-conventions.md` (`05-backend-conventions.md`) is budgeted work, not slack — it is a tracked risk in the register.

GitHub Milestones carry only a due date, so put the window in the description: `Sprint 3 (Wk 7) – Iteration 1: SRS + SDD + Software Package 1`. Branch/PR mechanics: `01-conventions/07-github-workflow-git-conventions.md` (`07-github-workflow-git-conventions.md`).

TrekLink — Decisions Log & Risk Register

Use this file like a lightweight ADR index. Anything marked **OPEN** blocks the dependent work listed under it — resolve before starting that work, not during.

Decisions

D-000 — Which register document is authoritative

- **Status:**  Resolved
- **Context:** `Phieu_FA26SE159.docx` (individual registration form, detailed RQs/NFRs/TP1-TP6) and `Phieu_dang_ky_de_tai_TrekLink_FA26_FINAL.docx` (team draft, M1-M5 module framing) describe the same project at different fidelity, with minor differences (supervisor email domain `fe.edu.vn` vs `fpt.edu.vn`; module count 5 vs 5 core capabilities; TypeORM vs Prisma — see D-001).
- **Decision:** `Phieu_FA26SE159.docx` is authoritative for anything it specifies explicitly (RQs, NFRs, FSMs, task packages, experiment design). The FINAL draft is used only to fill gaps it doesn't cover.
- **Owner:** Team lead — confirm with supervisor at Review 1 kickoff that FA26SE159 is the version on file.

D-001 — ORM: Prisma vs TypeORM

- **Status:**  Resolved (Prisma ORM)
- **Decision:** Prisma ORM is locked as the team mandate for maximum productivity and type safety across backend services. PostgreSQL schema models and migrations are maintained in `backend/prisma/schema.prisma`.

D-002 — Response envelope standard

- **Status:**  Resolved
- **Decision:** All backend responses use the shape already defined in `API_Design_Template.md`: `{ "result": ..., "isSuccess": bool, "statusCode": int, "message": string }`. This supersedes the raw-DTO / `errorDetails[]` pattern from the previously-used `dev-flow.zip` (a different, .NET-project convention) — see `02-templates/04-api-endpoint-template.md` and `01-conventions/05-backend-conventions.md`.

D-003 — GitLab → GitHub adaptation is cosmetic, not philosophical

- **Status:**  SUPERSEDED by D-009 (2026-09-13). Kept verbatim below for history — do not follow it.
- **Decision** (*superseded*): Branch topology (`main` / `develop` / `features/Implementation_*` / `features/Design_*` / `hotfix/*` / `release/sprint_x`), commit discipline, and the dual Design+Implementation branch pattern from `Git_Lab_Guide.pdf` carry over unchanged. Only the tool-specific mechanics change: GitLab Issues/Labels/Milestones → GitHub Issues/Labels/Milestones+Projects; GitLab MRs → GitHub PRs (multi-template via `.github/PULL_REQUEST_TEMPLATE/`); GitLab child tasks → GitHub sub-issues/tasklists. See `01-conventions/07-github-workflow-git-conventions.md`.

D-004 — Monorepo vs. multi-repo

- **Status:**  Resolved (updated Sep 7, Session 2 — repos exist and this is what's actually cloned)
- **Context:** TrekLink has 4 codebases: firmware (inherited, read-only), gateway bridge (Node/TS), backend (NestJS), frontend (React). The org is `github.com/TrekLink-Team`. The original sketch (below, kept for history) considered 4 separate app repos.
- **Original options considered:** (a) 4 separate repos under the org (`treklink-firmware`, `treklink-gateway`, `treklink-backend`, `treklink-web`) with independent CI, or (b) one monorepo with workspaces (`apps/gateway`, `apps/backend`, `apps/web`, `firmware/` as a git submodule).

- **Decision actually taken: a hybrid** — 3 repos total under `TrekLink-Team` :
 - `treklink-docs` — this repo (SSOT, docs-only).
 - `treklink-firmware` — inherited SU26 firmware, frozen/read-only this term.
 - `treklink-web` — **one** active application repo containing the Gateway Bridge, NestJS backend, and React frontend as workspace packages (`gateway/` , `backend/` , `frontend/`), rather than 3 separate repos.
- **Rationale:** The firmware is a frozen, inherited dependency — keeping it fully separate avoids accidental edits being graded as "new" work, matching the register's note that firmware effort must be excluded from statistics. Splitting the *active* code further into 3 repos (original option a) was reconsidered: for a 5-person/13-week term, one repo with workspace packages keeps shared TypeScript types (e.g. the `eventId` /DTO shapes gateway and backend both touch), a single CI pipeline, and a single `npm install` — at the cost of coarser per-package branch protection, which `module:*` labels and path-scoped PR reviews (see `01-conventions/07-github-workflow-git-conventions.md`) substitute for.
- **Consequence:** `treklink-web/specs/{module}/` is the one spec root for gateway, backend, and frontend modules alike — see `01-conventions/02-spec-driven-development-workflow.md` . Update that doc's example paths if this ever splits back into separate repos.

D-005 — Field-to-cloud bridge topology

- **Status:** ● **Resolved as a staged rollout** (updated Session 3) — ⚠ **Stage A alone does not satisfy the charter's offline NFRs; supervisor reconciliation required before TP2 sprint lock.**
- **Context:** The register describes the Gateway/Bridge as **dedicated TrekLink hardware with its own Wi-Fi/cellular uplink** (charter §2, FA26SE159 §b). Session 3 established the physical constraint that closes this: **the Gateway Bridge machine is not carried during the trek**. It sits at basecamp. The guide carries a node, not a laptop.
- **Options considered:**
 1. **Basecamp-fixed bridge** — a designated node USB-tethered to the basecamp machine running `gateway/src` ; the SQLite P0–P3 queue buffers the machine's intermittent Wi-Fi uplink. Requires no firmware work.
 2. **Node's built-in MQTT uplink, no bridge** — the guide's node publishes to Mosquitto over its own Wi-Fi using the stock Meshtastic MQTT module already compiled into the frozen firmware. Simplest possible path; **has no buffer anywhere**.
 3. **Fork the Meshtastic mobile apps** — rejected outright. iOS requires Xcode on macOS, which the team does not have; an Android-only fork would split Guide/Customer behavior by platform, and mobile apps are explicitly out of charter scope.
- **Decision:** Build **Option 2 first (Stage A, TP1)**, then **add Option 1 on top (Stage B, TP2)** as an additional ingress adapter plus a hard-wired gateway node. Option 3 stays rejected.
- **Rationale:** Option 2 is the shortest path to an end-to-end demo and needs no serial parser, no hardware tether, and no protobuf decoding. It is only non-throwaway because of the ingress-adapter seam defined in **D-007** — Stage B reuses the entire normalizer and ingestion pipeline and contributes only a new adapter in front of it.
- ⚠ **Known gap, deliberately accepted for Stage A:** Option 2 has **zero offline buffering**. Stock Meshtastic MQTT drops packets when the uplink is down (verified Session 2), and Store & Forward is a mesh-internal, PSRAM-only, ~30-packet chat replay — not a cloud-sync buffer. Therefore, **while only Stage A exists**, the following charter §5 NFRs are unmeasurable and RQ1/RQ2 are unanswerable:
 - Offline recovery delivery rate $\geq 99\%$ after reconnection (30s–30min loss)
 - Priority-ordering compliance $\geq 99\%$ — all P0 flushed before P2/P3 on reconnect
 - Gateway→cloud sync $\leq 5s$ *when uplink returns* (the $\leq 5s$ steady-state figure is still measurable)

These are restored by Stage B, where the SQLite queue sits in the uplink path and loss is induced by killing the bridge's Wi-Fi.

- **Scoping note that makes Stage B sufficient:** every offline NFR in charter §5 is scoped to the **gateway→cloud** leg, not the **node→gateway** mesh leg. Charter §1 gap #1 reads *"no store-and-forward path to the cloud... there is no Gateway between the mesh and a backend."* Mesh-range loss (a node out of LoRa range of any relay) is an RF-coverage problem, already carried as an RF-reliability caveat in the risk table below, and is **not** claimed as in scope. A basecamp-fixed bridge therefore exercises exactly the leg the research questions measure.
- **Hardware scope** (settled Session 3): **Stage A targets v2/v3/v4 only. v1 is out of the demo set.** v1 compiles MQTT out entirely (`-D MESHTASTIC_EXCLUDE_MQTT=1` , `variants/esp32/treklink_v1_0/platformio.ini:10`) — the module is removed at build time, so no runtime configuration can enable it. v2/v3/v4 omit that flag and all carry Wi-Fi silicon, so any of them can serve as the uplink node. Were v1 ever needed, deleting the build flag is a one-line change now that D-008 permits firmware edits.
- **Stage B is tradeable scope** (settled Session 3): build Stage A first. **If the supervisor agrees to reduce scope, Stage B may be omitted entirely.** The cost is explicit and must be stated rather than absorbed — dropping Stage B removes the only offline buffer in the system, so charter §5's offline-recovery and priority-ordering NFRs and research questions **RQ1 and RQ2 must be formally struck from the charter**, not quietly left unmeasured. The Gateway Bridge and Event Reliability Layer also leave the charter §7 deliverables list. That is a substantial reduction of the project's novelty claim (charter §8) and is the supervisor's call to make knowingly.
- **Blocks:** nothing further — `specs/gateway-sync/{requirements,design,tasks}.md` are written against this decision, with Stage B isolated in Phase 9 so its removal is a clean deletion rather than an unpick.
- **Action** (owner: team lead): put the Stage A/B trade to the supervisor explicitly. Either (a) keep Stage B and defer RQ1/RQ2 evaluation to TP2, or (b) drop Stage B and strike the offline NFRs and RQ1/RQ2 from the charter in the same conversation. Do not present Stage A as satisfying the offline-recovery NFR under either outcome.

D-006 — `eventId` redefinition: the charter's scheme is not constructible

- **Status:**  **Resolved** (Session 3) — supersedes charter §2 and `01-conventions/04-architecture-conventions.md` §3
- **Context:** Charter §2 and the architecture conventions both specify `eventId = DeviceID + SessionID + SequenceNumber` as the backend idempotency key. Direct inspection of the frozen firmware shows **neither `SessionID` nor `SequenceNumber` exists anywhere** — not in the TrekLink modules, not in the Meshtastic packet format. The only per-packet identifier transmitted is `MeshPacket.id`, and `Router.cpp:168` (`generatePacketId()`) shows it is a 10-bit rolling counter OR'd with 22 random high bits, re-seeded randomly at every boot. It is a flood-dedup token, **not** a monotonic sequence. The charter's key cannot be built from what the hardware actually sends.
- **Options considered:**
 1. **Split the key** into a packet-level dedup key and an episode-level correlation key.
 2. **Single content-addressed key**, `hash(from : rx_time : portnum : payload)` — rejected: the SOS beacon retransmits every 5s (then 30s) with a fresh `rx_time`, so one SOS episode would mint a new key per tick and create N Incidents from one fall.
 3. **Bridge synthesizes session + sequence** — rejected: the key would then describe the *bridge*, not the device. Restart the bridge, replay the same packets, and every key is new; deduplication becomes a silent no-op, which is precisely the failure the 10× replay NFR exists to catch.
- **Decision** (Option 1):
 - **Packet dedup key** — `GatewayEvent.eventId = sha256(nodeNum : packetId)`. Satisfies charter §5's *"0 duplicate Incidents for repeated eventId delivery (10× replay test)."* Enforced by a unique index; conflict is a no-op insert inside the ingestion transaction.
 - **Episode correlation** — *not* a hash. On SOS ingest, look up an open `Incident` for the same device whose `lastEventAt` falls inside the episode window; append if found, create if not. An earlier draft used `hash(nodeNum : timeBucket)`, which was discarded: a bucket boundary falling mid-episode splits one SOS into two Incidents, the exact failure the key exists to prevent.
- **Consequences:**

- `backend/prisma/schema.prisma : Incident.eventId @unique` currently serves as *both* keys and must be split — `Incident` gains `lastEventAt` and an index on `(deviceId, status, lastEventAt)`, and references its originating `GatewayEvent` by FK rather than carrying a duplicate unique string.
- `01-conventions/04-architecture-conventions.md §3` still states the old formula and is now contradicted by this decision. It must be updated before any `gateway-sync` implementation branch opens.
- Charter §2's wording stands as the *intent* (a natural, device-derived idempotency key); this decision records the constructible form of it.
- **Reopened by D-008** (firmware is editable): adding a boot- `sessionId` and a per-packet `sequenceNumber` firmware-side would make the charter's original formula constructible and would give a genuinely monotonic sequence — strictly better than hashing a randomized packet id, and it would let the platform *detect gaps* (a missing sequence number proves loss) rather than merely deduplicating what arrives. **The split key above remains correct and functional with or without it**; treat a firmware sequence as a layered upgrade to evaluate next session, not a prerequisite for TP1.

D-007 — Canonical event envelope + Strategy-based normalizer (the ingress seam)

- **Status:** ✔ **Resolved** (Session 3)
- **Context:** Stage A (node MQTT) and Stage B (serial bridge) do **not** speak the same wire format. The node's MQTT module publishes to a hardcoded topic shape — `<root>/2/e/<channelId>/<nodeId>` carrying a `ServiceEnvelope` protobuf, or `<root>/2/json/<channelId>/<nodeId>` carrying JSON when `json_enabled` (`src/mqtt/MQTT.cpp:423-430`, `:798`). `<root>` is configurable; `/2/e/` and the payload type are not. Meanwhile `gateway/src/mqtt/mqtt-client.ts` publishes `trekLink/events/priority/{0..3}` with a TrekLink-shaped payload. If the backend parses Stage A's format directly, Stage B's arrival forces a rewrite of the ingestion layer — and D-005's staged plan quietly becomes throwaway work.
- **Decision:** The backend ingests exactly one internal type, `TrekLinkEvent`. Between the wire and the domain sit two seams:
 1. **Ingress adapters** (`MeshIngressAdapter`) — one per topology, responsible only for getting bytes off a transport and producing a `RawMeshPacket`. Stage A ships `MqttJsonIngressAdapter`; Stage B adds `SerialBridgeIngressAdapter`. Adapters hold no domain logic.
 2. **Normalizer** (`MeshNormalizerService`) — the Strategy-pattern context. It resolves a `PacketNormalizationStrategy` by PortNum from an injected registry and delegates. Strategies are pure (no I/O, no Prisma) and each owns one PortNum's mapping to a domain event kind and priority tier. Adding a PortNum means adding a strategy class and registering it; the normalizer itself never changes (Open/Closed).
- **Rationale:** the normalizer holds ~all the genuinely hard logic (SOS prefix parsing, priority assignment, `eventId` derivation, episode correlation) and is 100% shared between stages. Purity makes it unit-testable without a broker, a device, or a database — which is what makes the charter's dedup/ordering/concurrency NFRs cheap to test rather than integration-only.
- **Stage A transport config, frozen:** `json_enabled = true`, `encryption_enabled = false`, `root = "trekLink"`. JSON removes any protobuf dependency from NestJS, and the JSON serializer covers the three PortNums TrekLink actually emits — `TEXT_MESSAGE_APP`, `POSITION_APP`, `TELEMETRY_APP` (`src/serialization/MeshPacketSerializer.cpp:28`, `:208`, `:56`). `encryption_enabled = false` is required so the broker receives decrypted packets; the field's own comment in `module_config.pb.h` notes decrypted output exists for exactly this purpose. The alternative is reimplementing Meshtastic's AES-CTR channel crypto in NestJS.
- **Known limitation of the JSON path:** the JSON envelope (`MeshPacketSerializer.cpp:410-424`) carries `id`, `timestamp`, `to`, `from`, `channel`, `type`, `sender`, `payload`, `rss`, `snr`, `hops_away` — but **not** `MeshPacket.priority`. The SOS position packet is sent with `priority = MAX` (`TrekLinkSOSHelper.cpp:119`) and is therefore indistinguishable from a routine position report on this path. Handled by backend episode correlation with a backward grace window (see `specs/gateway-sync/design.md §2.4`). Switching to the `/2/e/` protobuf topic would recover `priority` directly and is logged as the deferred precision upgrade.

D-008 — The firmware is editable this term (supersedes the "frozen / read-only" framing)

- **Status:**  **Resolved on the technical question** (Session 3, team lead) ·  **OPEN on the grading question** — needs supervisor confirmation
- **Context:** `README.md` §1, charter §2, and D-004's rationale all describe `treklink-firmware` as "Frozen / read-only this term. Branch-protected. Not a deliverable." Sessions 1–3 reasoned under that constraint and, on its strength, classified several firmware-rooted defects as unfixable-by-design. **The team lead has confirmed this is wrong:** the team owns the repo, can modify and build it, and can reflash the physical units.
- **Decision:** firmware modification is **technically available** and may be used to fix defects that cannot be worked around from the platform side. All prior "unfixable without touching frozen firmware" conclusions are void and must be re-derived.
- **The distinction that still matters** — three separate claims were bundled under one word:
 1. *Can we edit it?* — **Yes**. Settled.
 2. *Is firmware redesign in scope?* — **No**. Charter §2 lists "firmware redesign" under Explicitly out of scope, and D-000 makes `Phieu_FA26SE159.docx` authoritative on scope. A targeted reliability fix is not a redesign; a rearchitecture is.
 3. *Does firmware effort count toward graded deliverables?* — **Unknown**. D-004's rationale cites "the register's note that firmware effort must be excluded from statistics." If that holds, firmware work is real engineering that earns no credit — which affects how much sprint capacity it deserves, not whether it is allowed.
- **Working rule until (3) is answered:** firmware changes are permitted where they *remove a platform-side risk that has no software workaround*, kept surgical and individually justified. Prefer a platform-side solution when one exists at comparable cost. Do not open a firmware rearchitecture.
- **Reopens:**
 - **D-006** — a firmware-side boot- `sessionId` + per-packet `sequenceNumber` would make the charter's original `eventId` formula constructible. The split key stays valid regardless; this would be a layered upgrade.
 - **The Critical SOS risk below** — the single-unacknowledged-text-frame failure is now firmware-fixable (custom PortNum, `want_ack`, or retransmitting the discriminator with each beacon) rather than mitigable only by heuristic.
 - **D-007's SOS discrimination** — a real custom PortNum would retire the `printf`-format string parser entirely.
 - **v1 MQTT exclusion** — a one-line build-flag removal, though v1 is now out of the demo set anyway (see D-005).
 - **New (Session 4) — SOS beacon priority.** `TrekLinkSOSHelper::broadcastPosition()` retransmits via `PositionModule::sendOurPosition()`, which sets `priority = BACKGROUND` for a handheld role (`PositionModule.cpp:377-380`) — only the very first position packet at trigger time is `MAX`. Every beacon for the rest of an episode is the *lowest*-priority traffic on the mesh. Candidate fix: have the beacon path set `priority = MAX` (or at least `RELIABLE`) explicitly instead of delegating to the generic broadcast method. See `04-firmware-ground-truth.md` §2.
- **Action** (owner: team lead): confirm with the supervisor whether firmware commits count toward graded output, and that targeted fixes are not read as the out-of-scope "firmware redesign." Until then, log every firmware change here.

D-009 — Git, tracking & review model v2 (supersedes D-003)

- **Status:**  **Resolved** (2026-09-13, Session 5)
- **Context:** D-003 ratified the school's `Git_Lab_Guide.pdf` model wholesale. Auditing it against reality found it was never actually implemented and could not be: **no repository has ever had a `develop` branch**, yet `develop` appeared in 34 documentation files, making every documented git command broken. Three documents disagreed with each other on the branch model (`07-*.md` §3.2 and D-003 mandated dual Design/Implementation branches; `09-*.md` §1 said they were "not adopted"). The GitHub-Issues-as-tracker assumption conflicted with the team's actual use of Jira.
- **Options considered:**

1. **Fix reality to match the docs** — create `develop`, adopt dual branches, drop Jira. Rejected: dual branches double PR overhead for a 5-person/13-week term, and the team already runs Jira.
2. **Fix the docs to match reality, keeping GitLab's philosophy** — chosen.
3. Leave the contradiction and rely on tribal knowledge. Rejected: agents read the docs literally and will follow the broken commands.

- **Decision:**

- Integration branch is `dev`. Branches: `main`, `dev`, `feat/*`, `fix/*`, `hotfix/*`, `docs/*`, `chore/*`.
- `release/*` **dropped** — `main` is the release.
- **Dual Design/Implementation branches killed.** One branch per unit of work; spec commits land before implementation commits on the same branch, which preserves review independence at half the overhead.
- Branch naming: `{type}/{JIRA-KEY}-{short-kebab-desc}`, e.g. `feat/TK-45-device-registration`. The Jira key is what makes the org-level Jira↔GitHub integration auto-populate each card's Development panel.
- Commits: **Conventional Commits**, all standard types, Jira key as scope — `feat(TK-45): ...`. Scope omitted when genuinely general. **Not CI-enforced:** commit hygiene must never block a delivery.
- Merge: **Rebase & merge** default, **Squash & merge** when multi-commit, **merge commits prohibited**. Delete source branch except `dev` → `main`.
- **No direct pushes to any branch, by anyone, including the leader.**
- **Jira is the single work tracker.** GitHub Issues hold daily reports and standalone bugs only. GitHub Milestones retired; `points:*` labels retired in favour of Jira story points on a base-5 scale (1/2/3/5/10/15/20/25/30).
- Bugs route three ways: Jira `BUG status` (defect found reviewing an active story) / GitHub Issue `type:bug` (standalone, never enters Jira) / Jira `Bug work item` (only if >0.5 day or it changes a spec).
- CI is an **indicator, not a gate** — red CI does not block merge, but the leader must approve any red merge and the reason goes in the PR.
- PR self-approval permitted **only** for <50-line, non-behavioural housekeeping. The PO is not exempt and must run an independent AI review pass on their own non-trivial PRs.
- **English only** in every repository artifact. Vietnamese permitted when prompting an AI agent, and by exception in daily reports.
- **Consequence:** `01-conventions/` rewritten as Conventions v2; `07-*.md` fully rewritten; new chapters `10` (Jira), `11` (AI-first), `12` (communication). Two PR templates collapsed into one. `trekLink-web/docs/conventions/` deleted (D-011). The generated Developer Handbook PDF is built from these files.
- **Owner:** Team lead.

D-010 — Neon as the shared managed Postgres

- **Status:**  **Resolved** (2026-09-13)
- **Context:** `docker-compose.yml` provisions a local Postgres per developer, which is correct for isolated unit/integration work but gives the team no common database to co-develop against. Integration bugs that only appear with shared data (FK collisions, migration ordering, seed drift) surface late — typically during demo prep.
- **Decision:** Adopt **Neon** as the managed Postgres for shared **dev** and **prod/demo** environments. Local Docker Postgres remains the default for unit tests and CI (fast, hermetic, no network).
- **Consequence:** Each member installs the **Neon MCP connector** from the Claude skills store and authorises it under their own account. Connection strings live in each developer's `ignore/envs/` and are **never committed**. `DATABASE_URL` in CI continues to point at the ephemeral service container, not Neon.
- **Open:** branch-per-developer Neon databases vs. one shared dev database — decide before the first cross-module integration sprint.
- **Owner:** Team lead + LongLP (data lane).

D-011 — Single canonical convention set; vendored copies deleted

- **Status:**  **Resolved** (2026-09-13)

- **Context:** `treklink-web/docs/conventions/` held a verbatim, unreconciled copy of the author's generic `dev-flow` template — 12 files with zero TrekLink-specific content, still teaching `develop`, bracket-tag commits, and the dual-branch model. `treklink-web/AGENTS.md` explicitly directed agents to **fall back to it**, so an agent opened on `treklink-web` alone would have been actively taught the wrong rules.
- **Decision: Delete it.** `treklink-docs/_docs/01-conventions/` is the single canonical convention set for all three repositories. No repo vendors a second copy. A single `AGENTS.md`, identical in content, is placed in `capstone/`, `treklink-docs/`, `treklink-web/`, and `treklink-firmware/`, and points at the canonical folder.
- **Rationale:** A duplicated convention set does not merely drift — it drifts *silently*, and the copy is what gets read when the sibling layout isn't present. The previous fallback was worse than having no fallback, because a wrong answer delivered confidently costs more than a missing one.
- **Consequence:** Agents must be opened on the `capstone/` parent folder. An agent opened on a single repo will find `AGENTS.md` pointing at a path it cannot resolve, and must say so rather than guessing — which is the intended failure mode.
- **Owner:** Team lead.

Risk register (carried from FA26SE159, kept live)

Risk	Likelihood	Impact	Mitigation	Status
Insufficient physical TrekLink devices	Medium	High	Simulate Gateway→Cloud load via MQTT scripts; simulation never substitutes for LoRa RF reliability claims	Open
Gateway sync latency exceeds 5s NFR	Medium	Medium	Tune MQTT QoS, reduce payload size, backpressure on queue flush	Open
RQ3 baseline not objectively measurable	High	Medium	Randomized simulation drills; independent observer triggers SOS, records timestamp; participants unaware of exact trigger time	Open
NestJS + MQTT + WebSocket integration underestimated	Medium	High	PoC gateway→backend integration in TP1 Week 2, not deferred to TP3	Open
Firmware message schema incompatible with new gateway	Low	High	Schema frozen & documented in TP1 before any gateway implementation	Open — tracked as TP1 exit gate
Scope creep	High	Medium	Feature freeze after TP5 Week 10; anything else goes to post-capstone backlog	Open
ORM indecision stalls TP3 start	Medium	Medium	D-001 resolved (Prisma) ahead of the Week 4 target	Mitigated
[Added] Field connectivity may need phone-based bridging instead of a dedicated Gateway node (D-005)	Medium	High	Validate dedicated Gateway (Wi-Fi/cellular node, native Meshtastic MQTT module) in TP1 PoC before committing to it; mobile-app forking treated as out of scope given no macOS/Xcode access	Mitigated — D-005 resolved as a staged rollout (Stage A node-MQTT, Stage B basecamp bridge); app forking formally rejected
[Added] NestJS is explicitly listed as a skill only for Khoa (per the team skill matrix in 01-project-charter.md); the other 4 members list Spring Boot/plain Node.js/React	Medium	Medium	Sprint 1–2 pairing/ramp-up on 01-conventions/05-backend-conventions.md module conventions, mentored by Khoa, before Lane B/C backend stories are picked up solo	Open
[Added S3] SOS is announced by exactly one unacknowledged text packet. <code>TrekLinkSOSHelper::triggerSOS()</code> sends the "SOS - ..." text once with <code>want_ack = false</code> (<code>TrekLinkSOSHelper.cpp:157-160</code>); <code>tickBeacon</code> then retransmits position only . If that single text frame is lost over RF, the episode is invisible to the backend — the beacons that follow look like routine position reports.	Medium	Critical	Two layers, per D-008. <i>Platform side (built now):</i> cadence-anomaly detector — the beacon runs 5s for the first minute then 30s (<code>TrekLinkSOSHelper.cpp:181</code>), far denser than routine reporting, so $\geq N$ positions in window W raises a <i>suspected</i> episode at lower confidence (REQ-EVT-06, design.md §2.4). <i>Firmware side (now available, preferred):</i> retransmit the SOS discriminator with each beacon, and/or set <code>want_ack</code> , and/or introduce a real custom PortNum so SOS stops depending on a <code>printf</code> string. The platform mitigation is defence-in-depth and should be kept even after a firmware fix.	Open — firmware fix candidate #1

Risk	Likelihood	Impact	Mitigation	Status
[Added S3] Stage A (node MQTT, D-005) has no offline buffer, so charter §5's offline-recovery and priority-ordering NFRs and RQ1/RQ2 cannot be evaluated until Stage B lands	High	High	Build Stage A first. Stage B (basecamp bridge) is retained as scope the team lead will trade away if the supervisor agrees to reduce scope — in which case the offline NFRs and RQ1/RQ2 must be formally struck from the charter, not quietly dropped. Do not present Stage A as satisfying the offline NFR.	Open — needs supervisor decision
[Added S3] <code>treklink_v1_0</code> compiles MQTT out, so v1 units cannot act as the Stage A uplink node	Medium	High	v1 is out of the demo set (team lead, Session 3) — Stage A targets v2/v3/v4, all of which leave MQTT compiled in and carry Wi-Fi silicon. Were v1 ever needed, removing <code>-D MESHTASTIC_EXCLUDE_MQTT=1</code> is a one-line build-flag change now that D-008 permits it.	Closed
[Added S3] Stage A's JSON MQTT path omits <code>MeshPacket.priority</code> , making the SOS position packet indistinguishable from a routine one (D-007)	High	Medium	Backend episode correlation with a backward grace window (<code>design.md</code> §2.4). Two upgrades now available: switch to the <code>/2/e/</code> protobuf topic to recover <code>priority</code> , or emit an explicit SOS marker firmware-side (D-008).	Open — accepted for Stage A
[Added S3] <code>04-architecture-conventions.md</code> §3 still publishes the old <code>deviceId:sessionId:sequenceNumber</code> formula	High	Medium	§3 rewritten to the D-006 split key, with the D-008 firmware-sequence option noted as a layered upgrade.	Closed (Session 3)
[Added S3] Sessions 1–3 reasoned under a false "firmware is frozen" premise (D-008); several conclusions were shaped by a constraint that does not exist	High	Medium	D-008 logs the correction and lists every decision it reopens. Re-derive the affected conclusions next session before they harden into implementation.	Open — re-derivation pending
[Added S3] Firmware changes may earn no graded credit if the register's "firmware effort excluded from statistics" note holds (D-008 question 3)	Medium	Medium	Team lead confirms with supervisor. Until answered, keep firmware changes surgical and individually justified; prefer a platform-side fix at comparable cost.	Open — needs supervisor decision
[Added S4] SOS beacon retransmits (all but the very first position packet of an episode) are sent at <code>BACKGROUND</code> mesh priority, not <code>MAX</code> — <code>PositionModule::sendOurPosition()</code> sets priority by device role, and a handheld unit isn't <code>TRACKER / TAK_TRACKER</code> . Under mesh congestion, an ongoing emergency's beacon trail is currently the least-protected traffic class on the network after its opening packet.	Medium	High	Firmware-fix candidate under D-008: set <code>priority = MAX</code> (or <code>RELIABLE</code>) explicitly on the beacon send path rather than delegating to the generic broadcast method. Platform-side, the episode-correlation logic in <code>gateway-sync</code> already doesn't rely on <code>MeshPacket.priority</code> at all, so this doesn't block TP1/TP2 — it's a firmware-side robustness gap, not a backend defect.	Open — firmware fix candidate

Risk	Likelihood	Impact	Mitigation	Status
[Added S4] No backlog user story exists for the cadence-inferred SUSPECTED SOS episode path (gateway-sync/design.md §1.3/ §2.4, REQ-EVT-06) — the primary mitigation for the Critical single-unacknowledged-text-frame risk above has no corresponding Staff-facing workflow story yet	Medium	Medium	US-088 added (03-backlog/02-user-stories.md), assigned Khoa — Staff sees a Suspected episode visually distinct from Confirmed and can dismiss it with a lighter action than the full Resolved→Closed flow.	Mitigated (Session 4)

Contributing to TrekLink

The 2-minute version. Full detail lives in [01-conventions/](#), which is also the source of the [Developer Handbook PDF](#).

1. Orient yourself

- What are we building: [00-project-context/01-project-charter.md](#) ([01-project-charter.md](#))
- What week/sprint is it: [00-project-context/02-roadmap-and-milestones.md](#) ([02-roadmap-and-milestones.md](#))
- Anything blocking you: [00-project-context/03-decisions-and-risk-register.md](#) ([03-decisions-and-risk-register.md](#))

2. Where work lives

Jira is the single tracker ([TK project](#)). **GitHub Issues** hold daily reports and standalone bugs only. The **backlog** ([03-backlog/](#)) is read-only to members — it holds the EARS criteria you implement against. See [01-conventions/10-jira-tracking-and-workflow.md](#) ([10-jira-tracking-and-workflow.md](#)).

3. Before writing code

Every module goes through [specs/{module}/requirements.md](#) → [design.md](#) → [tasks.md](#) **before** implementation — see [01-conventions/02-spec-driven-development-workflow.md](#) ([02-spec-driven-development-workflow.md](#)). Blank templates in [02-templates/](#).

4. Branch, commit, PR

```
git fetch origin && git checkout dev && git pull origin dev
git checkout -b feat/TK-45-device-registration          # Jira key in the branch name

# ... work, per 01-conventions/03-operational-workflows.md ...
git commit -m "feat(TK-45): add device FSM transition guard"

npm test                                                # must be green BEFORE the PR
git fetch origin && git rebase origin/dev
git push -u origin feat/TK-45-device-registration --force-with-lease
gh pr create --base dev --title "feat(TK-45): device registration" --label "module:devices"
```

Then **move the Jira card to IN REVIEW** and **ping your reviewer in Zalo**. Full convention: [01-conventions/07-github-workflow-git-conventions.md](#) ([07-github-workflow-git-conventions.md](#)).

No direct pushes to any branch, ever — including the leader's.

5. Merging

Rebase & merge (single commit) or Squash & merge (multiple). **Merge commits are prohibited**. Delete the source branch, except `dev` → `main`. **Announce every merge to dev** in Zalo so everyone pulls and rebases.

6. Backend response contract (non-negotiable)

Every endpoint returns `{ result, isSuccess, statusCode, message }` — see [01-conventions/05-backend-conventions.md](#) ([05-backend-conventions.md](#)) §3.

7. Language

English only — code, commits, PRs, specs, docs, Jira cards. You may prompt your AI agent in Vietnamese; what it writes to the repo is English regardless.

8. Using an AI coding agent

Read `01-conventions/11-ai-first-doctrine-and-toolchain.md` (`11-ai-first-doctrine-and-toolchain.md`) once, fully. Install `prompt-orchestrator` and the TrekLink skills, open the `capstone/` parent folder (not a single repo), and run `/treklink-session` . The agent reads the conventions itself — you should never have to tell it to.

9. Definition of Done, every PR

- ☐ EARS criteria in `requirements.md` satisfied
- ☐ **Unit tests written and passing** — before the PR is opened
- ☐ Module boundaries respected (no cross-module repository access)
- ☐ Lint/typecheck clean
- ☐ `api-design/*.md` updated in the same PR if any endpoint changed
- ☐ Rebased on `dev` ; Jira card linked and moved to `IN REVIEW`
- ☐ `Model used:` declared in the PR